Informatik von Grund auf — Band 0

Die Logik des Schalters

Eine vollständige Einführung in die Grundlagen des Rechnens



Vom einzelnen Bit bis zur Maschinensprache
Jeder Schritt erklärt. Mit vollständig durchgerechneten Übungsaufgaben.

Stand der Forschung: August 2026

Informatik von Grund auf — Band 0
Die Logik des Schalters · Grundlagen des Rechnens

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Dauerhaft archiviert bei Zenodo: `https://doi.org/10.5281/zenodo.23773283`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 0: Die Logik des Schalters*. Fassung 1.0.1.
`https://doi.org/10.5281/zenodo.23773283`

Satz: LaTeX mit Palatino. Fassung 1.0.1, Oktober 2026. 226 Seiten.

Inhaltsverzeichnis

Vorwort **1**

I Das Bit **9**

1 Was Information ist **11**
1.1 Wo der Logarithmus herkommt 11
1.2 Der Informationsgehalt ungleicher Möglichkeiten 15

2 Zahlen aus Bits **27**
2.1 Negative Zahlen 29

3 Gleitkommazahlen **41**
3.1 Die Norm IEEE 754 42
3.2 Warum $0{,}1 + 0{,}2 \neq 0{,}3$ 45

4 Zeichen, Texte und alles Übrige **53**
4.1 Von ASCII zu Unicode 53
4.2 Bilder, Töne, alles andere 58

II Die Logik **65**

5 Die Boolesche Algebra **67**
5.1 Die Rechengesetze 69
5.2 De Morgan 70

6 Normalformen und Minimierung **79**
6.1 Minimierung 81

7 Gatter und Schaltnetze **91**
7.1 Ein Gatter genügt 92
7.2 Der Addierer 93
7.3 Multiplexer und die Recheneinheit 96

III Das Gedächtnis **105**

8 Wie Rückkopplung Gedächtnis erzeugt **107**
8.1 Das RS-Flipflop . . . 107
8.2 Der Takt . . . 110
8.3 Register und Speicher . . . 111

9 Endliche Automaten **117**
9.1 Was ein Automat kann . . . 118
9.2 Minimierung . . . 121

IV Die Maschine **127**

10 Der Von-Neumann-Rechner **129**
10.1 Die fünf Bauteile . . . 130
10.2 Der Befehlszyklus . . . 131

11 Maschinensprache und Assembler **141**
11.1 Der Assembler . . . 142
11.2 Sprünge, Schleifen, Unterprogramme . . . 143

12 Der Turm der Abstraktionen **153**
12.1 Vom Quelltext zum laufenden Programm . . . 156

V Das Programm **165**

13 Algorithmisches Denken **167**
13.1 Korrektheit . . . 168
13.2 Aufwand . . . 169

14 Der Selbstbezug **179**
14.1 Ein Programm, das sich selbst schreibt . . . 180
14.2 Die universelle Maschine . . . 182
14.3 Die Grenze . . . 184

VI Aktueller Stand **193**

15 Wo die Grundlagen heute stehen **195**
15.1 Das Ende der Skalierung . . . 196
15.2 Wohin es geht . . . 199

VII Anhang **205**

A Formelsammlung und Übersicht **207**
A.1 Die Kette vom Schalter zur Maschine 207
A.2 Zahlen und Kodierung 207
A.3 Aufwandsklassen 209
A.4 Die Landkarte des Bandes 211
A.5 Zeittafel 212
A.6 Glossar 215

B Kommentierte Literatur **217**

Vorwort: Warum zwei Zustände genügen

Ein Computer kann nur eines: unterscheiden, ob Strom fließt oder nicht. *Alles andere – jedes Bild, jeder Ton, jede Rechnung, jeder Text, jedes Modell des Wetters oder des Gehirns – ist auf diese eine Unterscheidung zurückgeführt.*

Die zentrale Behauptung dieses Bandes

> ***Aus einem einzigen Bauteil – einem Schalter, der einen anderen Schalter schalten kann – lässt sich jede berechenbare Funktion aufbauen.***

Das ist keine Metapher und keine Vereinfachung für Anfänger. *Es ist ein Satz, und dieser Band beweist ihn.*

Baustein	Was daraus wird	Kapitel
Schalter	**NAND-Gatter**	7
NAND	alle Booleschen Funktionen	6
Boolesche Funktionen	**Addierer, ALU**	7
+ Rückkopplung	**Gedächtnis**	8
+ Takt	**Automat**	9
+ Speicher für Befehle	**Universalrechner**	10

Sechs Zeilen vom Lichtschalter zur Maschine, die jedes Programm ausführen kann.
Dieser Band geht sie einzeln durch.

Warum ausgerechnet zwei

Man könnte fragen, warum Rechner binär arbeiten. Zehn Zustände wären der menschlichen Gewohnheit näher, drei wären theoretisch effizienter.

Die Antwort ist *physikalisch*, nicht mathematisch.

Grund	Erläuterung
Rauschabstand	Zwei Zustände lassen sich mit großem Abstand trennen; bei zehn liegen sie eng beieinander
Regeneration	Ein leicht verfälschtes Signal lässt sich auf den nächsten der zwei Werte zurückschieben
Bauteile	Ein Transistor ist von Natur aus ein Schalter, kein Zehnstufenregler
Algebra	Für zwei Werte existiert eine vollständige, einfache Algebra (Kapitel 5)

Der zweite Punkt ist der entscheidende. *Ein analoges Signal sammelt bei jeder Verarbeitungsstufe Fehler; nach hundert Stufen ist es unbrauchbar.* **Ein digitales Signal wird bei jeder Stufe wieder auf einen der beiden Sollwerte gezogen** – *der Fehler wächst nicht.*

> ***Das ist der eigentliche Grund für den Siegeszug des Digitalen**: nicht Genauigkeit, sondern **Fehlerfreiheit über beliebig viele Schritte**.* Eine analoge Schallplatte verliert bei jedem Abspielen; eine digitale Kopie ist nach tausend Generationen bitgleich.

Ternäre Rechner gab es wirklich

Von 1958 bis 1965 baute die Universität Moskau den Setun – *einen Rechner mit* ***drei*** *Zuständen* $(-1, 0, +1)$, **in ausgeglichener ternärer Darstellung.**

Die Bauart hat theoretische Vorzüge: Negative Zahlen brauchen keine Sonderbehandlung, und die Basis 3 *liegt näher an* $\mathrm{e} = 2{,}718$, *dem Optimum für die Speichereffizienz* (Kapitel 2).

Etwa fünfzig Exemplare wurden gebaut, und sie funktionierten gut. *Gescheitert ist die Bauart an der Verfügbarkeit von Bauteilen: Es gibt keinen dreistufigen Transistor.*

Ein Beispiel dafür, dass technische Entscheidungen selten aus theoretischer Überlegenheit folgen – sondern aus dem, was sich billig herstellen lässt.

Was dieses Buch will

Erstens: Die Kette lückenlos schließen. *Zwischen einem Transistor und einem laufenden Programm liegen etwa zehn Abstraktionsschichten.* **Die meisten Darstellungen springen über einige davon hinweg.** *Dieser Band lässt keine aus.*

> *Wer am Ende dieses Bandes angekommen ist, kann für jede Zeile eines Programms angeben, welche Schalter dabei umgelegt werden* – **nicht im Detail, aber im Prinzip, und ohne dass irgendwo Magie im Spiel wäre.**

Zweitens: Die Entscheidungen begründen, nicht bloß berichten. *Warum das Zweierkomplement und nicht Vorzeichen-Betrag? Warum ist* 0,1 *im Rechner nicht* 0,1*? Warum hat ein Byte acht Bit? Warum liegt das höchstwertige Bit links?* **Jede dieser Fragen hat eine Antwort, und meist eine interessante.**

Drittens: Den Selbstbezug von Anfang an sichtbar machen.

> *Der Gedanke, der die ganze Informatik trägt, ist der der* ***gespeicherten Anweisung****: Ein Programm ist Text, und Text ist Zahl, und Zahlen kann eine Maschine verarbeiten* – ***also kann ein Programm ein Programm verarbeiten, auch sich selbst.***

Kapitel 14 führt diesen Kreis zum ersten Mal vollständig aus – *mit einem Programm, das sich selbst ausdruckt, und mit einem ersten Blick auf die Grenze, an der die Sache umkippt.* **Die Grenze selbst ist Gegenstand von Band 2; der Begleitband widmet ihr das ganze Buch.**

Drei Warnungen

Erstens: Dies ist kein Programmierkurs.
Es kommt Code vor, aber nur so viel, wie zum Verständnis nötig ist – und meist in Assembler oder in einer bewusst schlichten Pseudosprache. **Wer eine Sprache lernen will, findet im Anhang Hinweise.**
Zweitens: Manches wird bewusst *zu einfach* dargestellt und später korrigiert.

Der Von-Neumann-Rechner in Kapitel 10 verarbeitet einen Befehl nach dem anderen. Ein realer Prozessor tut das seit dreißig Jahren nicht mehr – **Band 4 behandelt Pipelining und Nebenläufigkeit.**

Ich sage jedes Mal ausdrücklich, wo vereinfacht wird. *Eine Vereinfachung, die sich als solche zu erkennen gibt, ist ein Werkzeug; eine, die es nicht tut, ist ein Fehler.*
Drittens: Die Zahlen altern, die Prinzipien nicht.
Angaben zu Taktraten, Strukturgrößen und Speicherpreisen sind auf August 2026 bezogen und werden bald überholt sein. **Das Zweierkomplement, die Boolesche Algebra und der gespeicherte Befehl sind es nicht** – *sie stehen seit den 1940er Jahren und werden stehen bleiben.*

Zum Aufbau

- **Teil I** (Kap. 1–4): Was Information ist; wie man Zahlen, Brüche und Zeichen durch Bits darstellt – und wo die Darstellung an Grenzen stößt.
- **Teil II** (Kap. 5–7): Die Boolesche Algebra, die Minimierung von Ausdrücken und der Bau von Schaltnetzen bis hin zum Addierer und zur Recheneinheit.
- **Teil III** (Kap. 8–9): Wie aus Rückkopplung Gedächtnis wird; endliche Automaten als Modell jedes getakteten Systems.
- **Teil IV** (Kap. 10–12): Der Von-Neumann-Rechner, seine Maschinensprache und der Turm der Abstraktionsschichten.
- **Teil V** (Kap. 13–14): Algorithmisches Denken; der Selbstbezug und ein erster Blick auf die Grenze des Berechenbaren.
- **Teil VI** (Kap. 15): Aktueller Stand, August 2026.

Zweihundert Jahre vom Gedanken zur Maschine

1679	Leibniz	*De progressione dyadica*: das Binärsystem
1703	Leibniz	Veröffentlichung; Bezug auf das I Ging
1801	Jacquard	Lochkartensteuerung des Webstuhls
1837	Babbage	Entwurf der **Analytical Engine**
1843	**Ada Lovelace**	**Notes: das erste Programm, und der Gedanke der allgemeinen Symbolverarbeitung**
1847	**Boole**	*The Mathematical Analysis of Logic*
1854	Boole	*An Investigation of the Laws of Thought*
1886	Peirce	Logik durch elektrische Schaltkreise (unveröffentlicht)
1904	Fleming	Vakuumröhre
1936	**Turing**	*On Computable Numbers*: die Maschine und ihre Grenze
1936	Church	Lambda-Kalkül; dieselbe Grenze
1937	**Shannon**	**Masterarbeit: Boolesche Algebra = Relaisschaltungen**
1938	Zuse	Z1, mechanisch, binär, Gleitkomma
1941	Zuse	**Z3** – erster funktionsfähiger programmgesteuerter Rechner
1943	McCulloch, Pitts	Neuron als logisches Gatter
1945	von Neumann	*First Draft of a Report on the EDVAC*
1948	**Shannon**	*A Mathematical Theory of Communication*; das Wort „Bit“
1948	Manchester Baby	erster Rechner mit gespeichertem Programm
1947/48	Bardeen, Brattain, Shockley	**Transistor**
1958	Kilby, Noyce	integrierter Schaltkreis
1958–65	Brusenzow	**Setun**: ternärer Rechner
1965	Moore	Verdopplungsgesetz
1971	Intel	4004: erster Mikroprozessor auf einem Chip
1972	Ritchie	C; Unix in einer Hochsprache

Die Zeile 1937 verdient Hervorhebung. CLAUDE SHANNON, *damals 21 Jahre alt, schrieb eine Masterarbeit am MIT mit dem Titel „A Symbolic Analysis of Relay and Switching Circuits“.*

> *Ihr Inhalt in einem Satz:* ***Die Algebra, die George Boole 1847 für die Gesetze des Denkens erfunden hatte, beschreibt exakt das Verhalten von Relaisschaltungen.***

Damit war der Bau digitaler Schaltungen von einer Bastelei in eine Rechenaufgabe verwandelt. *Man konnte eine Schaltung nun* ***ausrechnen****, statt sie zu erraten – und man konnte beweisen, dass eine Vereinfachung korrekt ist.*

Die Arbeit gilt vielfach als die folgenreichste Masterarbeit, die je geschrieben wurde. **Sie ist 21 Seiten lang.**

Und die Zeile 1843 ist die merkwürdigste. ADA LOVELACE *übersetzte einen italienischen Aufsatz über Babbages Maschine und versah ihn mit Anmerkungen, die dreimal so lang waren wie das Original.*

> *In Anmerkung A steht der Gedanke, auf den vor ihr niemand gekommen war:* **Wenn die Maschine Symbole nach Regeln verarbeiten kann, muss sie nicht auf Zahlen beschränkt bleiben.** *Lovelace nennt ausdrücklich die Möglichkeit, damit* ***Musik*** *zu komponieren.*

Das ist der Gedanke der *universellen* Symbolverarbeitung, hundert Jahre vor Turing – *und er war zu einem Zeitpunkt formuliert, als die Maschine, von der er handelte, nicht existierte und nie gebaut werden sollte.*

Die Analytical Engine erhebt keinerlei Anspruch darauf,
irgendetwas hervorzubringen. Sie kann alles ausführen,
was wir ihr zu befehlen wissen.

— ADA LOVELACE (1815–1852), Anmerkung G.
Turing widersprach 1950 ausdrücklich.
Die Debatte ist bis heute offen.

Part I

Das Bit

1 Was Information ist

Die Grundfrage

Bevor man Information verarbeitet, muss man wissen, was man ***misst****. Und die Antwort ist überraschend:* ***Information misst nicht Bedeutung, sondern beseitigte Ungewissheit.***

Ein Gedankenexperiment. *Ich denke mir eine Zahl zwischen 1 und 16. Sie dürfen Ja-Nein-Fragen stellen.* **Wie viele brauchen Sie im schlimmsten Fall?**
Die schlechte Strategie: „Ist es die 1?“ „Ist es die 2?“ – *im schlimmsten Fall 15 Fragen.*
Die gute Strategie: *Halbieren.*

Frage	Inhalt	Antwort	Rest
1	Ist sie ≥ 9?	ja	$9 \ldots 16$
2	Ist sie ≥ 13?	nein	$9 \ldots 12$
3	Ist sie ≥ 11?	ja	$11, 12$
4	Ist sie ≥ 12?	nein	**11**

Vier Fragen genügen – und weniger geht nicht. **Denn jede Frage kann höchstens halbieren, und $2^4 = 16$.**

Definition 1.1. Ein **Bit** (von *binary digit*) ist die Informationsmenge, die eine einzelne Ja-Nein-Frage bei zwei gleich wahrscheinlichen Möglichkeiten beantwortet.

1.1 Wo der Logarithmus herkommt

Erst zählen, dann benennen

Vier Fragen reichen für sechzehn Möglichkeiten – das haben wir nachgezählt. Jetzt wollen wir den allgemeinen Zusammenhang, **und zwar so, dass am Ende nichts unerklärt bleibt.**

Schritt 1 – Wie viele Fälle kann man mit k Fragen unterscheiden?

Jede Frage hat zwei mögliche Antworten. Bei k Fragen nacheinander gibt es also

$$\underbrace{2 \cdot 2 \cdots 2}_{k \text{ Faktoren}} = 2^k$$

verschiedene Antwortfolgen – **das ist die Produktregel** (Mathematik-Band 15, Kapitel 1).

*Am Ende darf auf jede Antwortfolge höchstens **eine** Möglichkeit entfallen – sonst wäre die Sache nicht entschieden.* **Also:**

Mit k Ja-Nein-Fragen unterscheidet man höchstens 2^k Fälle.

k	2^k	k	2^k	k	2^k
1	2	5	32	9	512
2	4	6	64	10	1024
3	8	7	128	16	65 536
4	16	8	**256**	20	$\approx 10^6$

Schritt 2 – Die Frage umdrehen.

Bisher fragten wir: „Was leisten k Fragen?“ Jetzt umgekehrt: „Wie viele Fragen brauche ich für N Fälle?“

Gesucht ist also dasjenige k mit

$$2^k = N.$$

Für $N = 16$ liest man 4 in der Tabelle ab, für $N = 256$ die 8 – man geht sie einfach rückwärts. **Aber $N = 100$ steht nicht darin, und man möchte trotzdem eine Zahl nennen können.**

Schritt 3 – Der Name für diese Umkehrung.

Für genau diese Umkehrung hat die Mathematik einen Namen (Mathematik-Band 0, Kapitel 3): **den Logarithmus zur Basis 2.**

Definition 1.2. $\log_2 N$ ist **definiert** als diejenige Zahl k, für die $2^k = N$ gilt.

*Es ist keine neue Rechenart, sondern ein **Name für eine Frage**:* **„Mit welchem Exponenten muss ich 2 potenzieren, um N zu erhalten?“**

Gefragt	Schreibweise	Antwort
$2^{?} = 8$	$\log_2 8$	3
$2^{?} = 1024$	$\log_2 1024$	10
$2^{?} = 1$	$\log_2 1$	0
$2^{?} = 100$	$\log_2 100$	6,644
$2^{?} = \frac{1}{2}$	$\log_2 \frac{1}{2}$	-1

Schritt 4 – Die vierte Zeile verstehen.

Was soll $2^{6{,}644}$ bedeuten – ein gebrochener Exponent?

Er ist über Wurzeln erklärt (Mathematik-Band 0, Kapitel 3): $2^{1/2} = \sqrt{2}$, $2^{3/2} = \sqrt{2^3}$, *und für beliebige reelle Exponenten durch Grenzübergang.* **Man kann es aber auch schlicht nachprüfen:**

2^6	$= 64$	zu klein
$2^{6{,}5}$	$= 90{,}5$	noch zu klein
$2^{6{,}64}$	$= 99{,}7$	fast
$2^{6{,}644}$	$= 100{,}0$	✓
2^7	$= 128$	zu groß

Man braucht keinen Taschenrechner

$\log_2 N$ lässt sich auf jeden anderen Logarithmus zurückführen:

$$\log_2 N = \frac{\ln N}{\ln 2} = \frac{\ln N}{0{,}6931}.$$

Der Grund ist die Regel $\ln(2^k) = k \ln 2$*: Aus* $2^k = N$ *folgt* $k \ln 2 = \ln N$. ✓
Als Faustregel genügt: $\log_2 N \approx 3{,}32 \cdot \lg N$ **– also ungefähr die Zahl der Dezimalstellen mal** 3,32**.**

Eine Million hat sechs Nullen, also $\log_2 10^6 \approx 6 \cdot 3{,}32 = 19{,}9$ **– und tatsächlich ist** $2^{20} = 1\,048\,576$**.** ✓

Schritt 5 – Damit ist die Grundformel begründet.

$$2^k \geq N \iff k \geq \log_2 N.$$

Für N gleichwahrscheinliche Möglichkeiten braucht man $\log_2 N$ Bit.

Der Logarithmus ist hier nicht vom Himmel gefallen. Er ist die Umkehrung des Verdoppelns – und das Verdoppeln kam daher, dass jede Frage zwei Antworten hat. Mehr steckt nicht dahinter.

Möglichkeiten	Bit	Beispiel
2	1	Münzwurf
8	3	Himmelsrichtung (fein)
16	4	Hexadezimalziffer
256	8	**ein Byte**
65 536	16	
$4{,}3 \cdot 10^9$	32	IPv4-Adressen
$1{,}8 \cdot 10^{19}$	64	

Schritt 6 – Warum „Zahl der Fragen“ und „Zahl der Bits“ dasselbe ist.
Man schreibe für jede Antwort eine 1 (ja) oder eine 0 (nein). Aus der Antwortfolge wird ein Bitmuster:

Antwortfolge	Muster	gesuchte Zahl
ja, nein, ja, nein	1010	11
nein, nein, nein, nein	0000	1
ja, ja, ja, ja	1111	16

Jede Zahl liefert genau ein Muster, jedes Muster genau eine Zahl – eine ***Bijektion*** (Mathematik-Band 15, Kapitel 1). **„Vier Fragen“ und „vier Bit“ sind zwei Namen für dieselbe Sache.** ✓

Und damit ist bereits Kapitel 2 vorbereitet: *Die Halbierungsstrategie* ***erzeugt*** *die Binärdarstellung der gesuchten Zahl – Frage für Frage, Bit für Bit.*

Zurück zur Bedeutung

Information ist nicht Bedeutung

Der Satz „Morgen geht die Sonne auf“ enthält nach dieser Definition *fast keine* Information – *denn er beseitigt keine Ungewissheit.*
Der Satz „Morgen geht die Sonne *nicht* auf“ enthält sehr viel Information – *obwohl er mit hoher Wahrscheinlichkeit falsch ist.*

> ***Shannons Informationsbegriff ist bewusst bedeutungsfrei.*** *Er wollte wissen, wie viele Leitungen man für eine Telefonverbindung braucht – nicht, worüber gesprochen wird.*

Shannon selbst schrieb 1948 gleich im zweiten Absatz: *Die semantischen Aspekte der Kommunikation seien für das ingenieurtechnische Problem* ***ohne Belang****.*

Man hat ihm das oft zum Vorwurf gemacht. **Es ist aber genau die Beschränkung, die die Theorie überhaupt erst möglich machte** – *Band 7 der Reihe behandelt sie ausführlich.*

1.2 Der Informationsgehalt ungleicher Möglichkeiten

Die Formel wird nicht gesetzt, sondern gefunden

Bisher waren alle Möglichkeiten gleich wahrscheinlich. Was, wenn nicht?

Schritt 1 – Der Fall, den wir schon kennen.
Bei N gleichwahrscheinlichen Möglichkeiten hat jede einzelne die Wahrscheinlichkeit

$$p = \frac{1}{N}, \qquad \text{also} \qquad N = \frac{1}{p}.$$

Und wir haben ausgerechnet, dass man dafür $\log_2 N$ Bit braucht:

$$\log_2 N = \log_2 \frac{1}{p}.$$

> ***Das ist noch keine neue Definition, sondern nur eine Umschreibung des schon Bewiesenen.*** Wir haben lediglich N durch $\frac{1}{p}$ ersetzt.

Schritt 2 – Der Übergang zum ungleichen Fall.

Nun sei nicht mehr alles gleich wahrscheinlich. Was soll dann „der Informationsgehalt ***eines*** *Ereignisses“ heißen?*

Ein Beispiel, das die Antwort nahelegt. *Ein Würfel hat sechs Seiten. Ich frage nicht nach der Augenzahl, sondern nur:* „Ist es eine Sechs?“

Antwort	p	Wie viel ist geklärt?
„ja“	$\frac{1}{6}$	**alles** – ich kenne die Zahl
„nein“	$\frac{5}{6}$	**wenig** – fünf bleiben offen

*Die **seltene** Antwort trägt mehr Information als die häufige.* **Und zwar genau so viel, wie nötig wäre, um aus $\frac{1}{p}$ gleichwahrscheinlichen Fällen einen auszuwählen.**

Bei „ja“: $\log_2 6 = 2{,}585$ Bit – **die Sechs ist eine von sechs.** ✓
Bei „nein“: $\log_2 \frac{6}{5} = 0{,}263$ Bit – **die Menge ist von 6 auf 5 geschrumpft, also um den Faktor $\frac{6}{5}$.** ✓

Definition 1.3. Der **Informationsgehalt** eines Ereignisses mit Wahrscheinlichkeit p ist

$$I(p) := \log_2 \frac{1}{p} = -\log_2 p \quad \text{Bit}.$$

Schritt 3 – Die Probe: Ist die Verallgemeinerung zwingend?

Man könnte einwenden: Warum gerade diese Fortsetzung und keine andere? **Die Antwort ist, dass drei sehr schwache Forderungen sie bereits erzwingen.**

Forderung	Warum man sie stellt
$I(1) = 0$	Ein sicheres Ereignis klärt nichts
I fällt mit wachsendem p	Seltenes ist informativer
$I(p \cdot q) = I(p) + I(q)$	**Zwei unabhängige Nachrichten liefern zusammen die Summe**

Zur dritten Forderung. *Erfahre ich zwei unabhängige Dinge – etwa den Ausgang zweier Münzwürfe –, so ist die gemeinsame Wahrscheinlichkeit das **Produkt** $p \cdot q$* (Mathematik-Band 4, Kapitel 3). **Die Information soll sich aber *addieren*:** *zwei Würfe, zwei Bit.*

Gesucht ist also eine Funktion, die Produkte in Summen verwandelt – und genau das ist die kennzeichnende Eigenschaft des Logarithmus (Mathematik-Band 0, Kapitel 3).

Satz 1.4. *Erfüllt $I : (0,1] \to [0,\infty)$ die drei Forderungen, so gilt notwendig*

$$I(p) = c \cdot \log_2 \frac{1}{p}$$

für eine Konstante $c > 0$.

Der Beweis – in acht Schritten, ohne Lücke

Ich führe den Beweis vollständig aus, weil er der erste Punkt dieses Bandes ist, an dem eine Formel vom Himmel zu fallen scheint. **Sie tut es nicht.**

Schritt 1 – Die Wahrscheinlichkeit anders schreiben.
Jede Zahl p mit $0 < p \leq 1$ lässt sich darstellen als

$$p = 2^{-x} \qquad \text{mit genau einem } x \geq 0.$$

x	0	1	3
$p = 2^{-x}$	1	$\frac{1}{2}$	$\frac{1}{8}$

Wächst x von 0 ins Unendliche, so fällt 2^{-x} von 1 auf 0 und nimmt dabei ***jeden*** *Zwischenwert genau einmal an* (Mathematik-Band 3, Kapitel 6). **Die Zuordnung $x \leftrightarrow p$ ist also umkehrbar.** ✓
Schritt 2 – Eine Hilfsfunktion einführen.

$$f(x) := I\big(2^{-x}\big) \qquad (x \geq 0).$$

Was wird aus den drei Forderungen?

aus $I(1) = 0$	$f(0) = I(2^0) = I(1) = 0$
aus der Monotonie	f ist monoton **wachsend**: größeres x bedeutet kleineres p, also mehr Information
aus der Additivität	$f(x+y) = I(2^{-x-y}) = I(2^{-x} \cdot 2^{-y}) = I(2^{-x}) + I(2^{-y}) = f(x) + f(y)$

$$\boxed{f(x+y) = f(x) + f(y), \qquad f \text{ monoton wachsend}, \qquad f(0) = 0}$$

Das ist die ***Cauchysche Funktionalgleichung****.* **Wir zeigen nun ohne jeden weiteren Hilfssatz, dass daraus $f(x) = cx$ folgt.**

Schritt 3 – Ganze Zahlen.
Wir taufen den Wert an der Stelle 1:

$$c := f(1).$$

Dann liefert die Additivität

$$f(2) = f(1+1) = f(1) + f(1) = 2c,$$

$$f(3) = f(2+1) = f(2) + f(1) = 2c + c = 3c.$$

Vollständige Induktion (Mathematik-Band 0, Kapitel 14): *Gilt $f(n) = nc$, so*

$$f(n+1) = f(n) + f(1) = nc + c = (n+1)\,c. \qquad ✓$$

$$\Longrightarrow f(n) = n\,c \qquad \text{für alle } n \in \mathbb{N}.$$

Schritt 4 – Stammbrüche.
Sei $m \in \mathbb{N}$ fest. Wir zerlegen die Eins in m gleiche Teile:

$$c = f(1) = f\Big(\underbrace{\tfrac{1}{m} + \tfrac{1}{m} + \cdots + \tfrac{1}{m}}\Big).$$

*Eine andere Basis des Logarithmus entspricht einer anderen Wahl von c und damit nur einer anderen **Einheit**.*

Basis	Einheit	Wo gebräuchlich
2	**Bit**	Informatik
e	**Nat**	Theorie, Physik
10	**Hartley**, Ban	frühe Kryptoanalyse

Die Einheit „Ban“ stammt aus Bletchley Park – ALAN TURING und IRVING GOOD benutzten sie 1941 bei der Enigma-Entzifferung, benannt nach der Stadt Banbury, wo die dafür bedruckten Lochkarten hergestellt wurden.

Der Informationsgehalt in Zahlen

p	$I(p)$ in Bit	**Beispiel**
1	0	sicher
$\frac{1}{2}$	1	Münzwurf
$\frac{1}{6}$	2,585	Würfelaugenzahl
$\frac{1}{1000}$	9,966	
$\frac{1}{13\,983\,816}$	23,737	Lottotreffer

Die Entropie

Definition 1.5. Die **Entropie** einer Quelle mit den Wahrscheinlichkeiten $p_1, \ldots, p_n$ ist der **mittlere** Informationsgehalt

$$\boxed{H = \sum_{i=1}^{n} p_i \log_2 \frac{1}{p_i} = -\sum_i p_i \log_2 p_i}$$

*Sie gibt an, wie viele Bit man **im Mittel** braucht, um ein Zeichen der Quelle zu übertragen.*

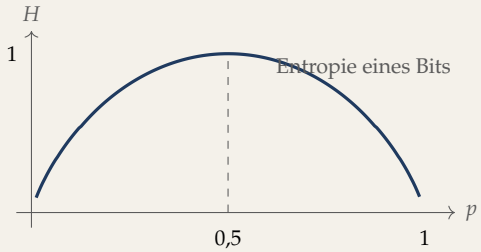
Zwei Extremfälle.
Alle gleich wahrscheinlich, $p_i = \frac{1}{n}$:

$$H = \sum_{i=1}^{n} \frac{1}{n} \log_2 n = \log_2 n \qquad \textbf{– maximal.}$$

Ein Ereignis sicher, $p_1 = 1$:

$$H = 1 \cdot \log_2 1 = 0 \qquad \textbf{– minimal.}$$

Die Entropie misst also Unvorhersagbarkeit. Eine Quelle, die immer dasselbe sendet, hat Entropie null; eine völlig zufällige hat die größtmögliche.



Die Kurve ist symmetrisch und hat ihr Maximum bei $p = \frac{1}{2}$. **Eine gezinkte Münze liefert weniger als ein Bit pro Wurf.**

p	H in Bit	**Deutung**
0,5	1,000	faire Münze
0,9	0,469	stark gezinkt
0,99	0,081	fast sicher
0,999	0,0114	

Das ist der Grund, warum Kompression funktioniert (Band 7): *Ein Text, dessen Zeichen ungleich verteilt sind, hat weniger Entropie als* $\log_2 n$ *Bit pro Zeichen – und lässt sich entsprechend kürzer kodieren.*

Das Wort „Bit“ und seine Herkunft

Shannon führte das Wort 1948 in einer Fußnote ein und schrieb es JOHN TUKEY *zu, der es als Kurzform von „binary digit“ vorgeschlagen hatte.*

Es ist ein doppelt geglückter Name – *denn „bit“ bedeutet im Englischen ohnehin „ein bisschen“.*

Zwei Bedeutungen von „Bit“

Das Wort wird für zwei verschiedene Dinge benutzt:

Speicherstelle	Ein physischer Platz, der 0 oder 1 halten kann
Informationsmenge	Die Menge Information, die eine Ja-Nein-Frage klärt

Ein 8-Bit-Speicher hat immer 8 Bit ***Kapazität*** **– aber wenn darin nur die Zahlen 0 bis 3 vorkommen, trägt er nur 2 Bit** ***Information*****.**

Der Unterschied ist derselbe wie zwischen dem Fassungsvermögen einer Kiste und dem, was tatsächlich drin ist.

Und das Byte? *Der Begriff stammt von* Werner Buchholz, *IBM, 1956 – absichtlich mit „y“ geschrieben, damit man ihn nicht mit „bit“ verwechselt.*

Dass ein Byte ***acht*** *Bit hat, ist keine Naturkonstante.* **Frühe Rechner hatten Bytes von 6, 7 oder 9 Bit.** *Die Acht setzte sich mit der IBM System/360 (1964) durch – weil sie eine Zweierpotenz ist, zwei Hexadezimalziffern fasst und für den damaligen Zeichensatz reichte.*

In Normen heißt die Einheit von genau acht Bit deshalb bis heute vorsichtshalber ***Oktett****.*

Aufgabe 1.1.

(a) Wie viele Fragen braucht man für ein Objekt aus N Möglichkeiten, wenn N keine Zweierpotenz ist? Berechnen Sie den Informationsgehalt eines Passworts.

(b) Bestimmen Sie die Entropie der deutschen Sprache und schätzen Sie, wie stark sich Text komprimieren lässt.

(c) Zwölf Kugeln, eine ist verschieden schwer. Bestimmen Sie die untere Schranke für die Zahl der Wägungen – und geben Sie ein Verfahren an.

Lösung Schritt für Schritt.

(a) Nicht-Zweierpotenzen und Passwörter.
Schritt 1 – Die allgemeine Schranke.
Bei N Möglichkeiten und Ja-Nein-Fragen kann man nach k Fragen höchstens 2^k Fälle unterscheiden. Also braucht man

$$2^k \geq N \Longrightarrow k \geq \log_2 N \Longrightarrow k_{\min} = \lceil \log_2 N \rceil \,.$$

Beispiel $N = 100$:

$$\log_2 100 = \frac{\ln 100}{\ln 2} = \frac{4{,}6052}{0{,}6931} = 6{,}644 \Longrightarrow k = 7.$$

Mit 6 Fragen unterscheidet man nur 64 Fälle, mit 7 aber 128. ✓

Der Unterschied zwischen Mittel und Maximum

Die Zahl $\log_2 100 = 6{,}644$ ist nicht bedeutungslos. *Sie ist die Zahl der Fragen, die man im **Mittel** braucht, wenn man **viele** solche Aufgaben hintereinander löst und die Fragen geschickt über die Aufgaben verteilt.*

Bei einer einzelnen Aufgabe braucht man 7 Fragen; bei n Aufgaben gleichzeitig etwa $6{,}644\,n$.

Das ist im Kern schon Shannons Quellenkodierungssatz (Band 7): *Die Entropie ist erreichbar, aber erst im Grenzwert langer Blöcke.*

Schritt 2 – Ein Passwort aus zufälligen Zeichen.
Bei einem Zeichensatz von 95 druckbaren ASCII-Zeichen und Länge ℓ:

$$N = 95^{\ell}, \qquad I = \ell \cdot \log_2 95 = \ell \cdot 6{,}570 \text{ Bit}.$$

ℓ	**Bit**	**Suchzeit bei 10^{12} Vers./s**
6	39,4	12 Minuten
8	52,6	76 Tage
10	65,7	1,9 Mio. Jahre
12	78,8	$1{,}7 \cdot 10^{10}$ Jahre

Schritt 3 – Der Vergleich mit Wörtern.
Vier zufällige Wörter aus einer Liste von $8192 = 2^{13}$ Wörtern:

$$I = 4 \cdot 13 = 52 \text{ Bit}.$$

Ebenso sicher wie acht zufällige Sonderzeichen – und um Größenordnungen leichter zu merken. Der Grund ist, dass die Sicherheit allein an der Zahl der **gleichwahrscheinlichen Möglichkeiten** hängt, nicht daran, wie kryptisch etwas aussieht.

Das Wort „zufällig“ trägt die ganze Last

Ein *selbst ausgedachter* Vierwortsatz hat weit weniger Entropie. *Menschen wählen keine gleichverteilten Wörter, sondern häufige, thematisch passende und grammatisch verbundene.*

Ebenso ist `P@ssw0rt!` *nicht 9 Zeichen × 6,57 Bit wert* – **es steht in jeder Angriffsliste und hat effektiv wenige Bit.**

Entropie ist eine Eigenschaft des *Erzeugungsverfahrens*, nicht des Ergebnisses. *Ein Würfel liefert echte Entropie; ein Gehirn, das sich „etwas Zufälliges“ ausdenkt, nicht.*

(b) Die Entropie des Deutschen.
Schritt 1 – Die naive Schätzung.
26 Buchstaben plus Leerzeichen, gleichverteilt gedacht:

$$H_0 = \log_2 27 = 4{,}755 \text{ Bit pro Zeichen.}$$

Schritt 2 – Die tatsächlichen Häufigkeiten.

Zeichen	p	Zeichen	p
Leer	0,151	N	0,087
E	0,147	I	0,065
S	0,058	R	0,060
A	0,054	T	0,053
…		Q	0,0002

Setzt man alle 27 Werte in die Entropieformel ein, so erhält man

$$H_1 \approx 4{,}1 \text{ Bit pro Zeichen.}$$

Schritt 3 – Die Abhängigkeiten berücksichtigen.

Deutsch ist stark strukturiert: Nach „Q“ kommt fast immer „U“, nach „SCH“ kommt ein Vokal, und ganze Wörter sind vorhersagbar. **Berücksichtigt man das, sinkt die Entropie weiter.**

Modell	Bit/Zeichen	Anteil von H_0
gleichverteilt	4,76	100 %
Einzelhäufigkeiten	4,1	86 %
Paare (Bigramme)	3,5	74 %
Dreiergruppen	3,0	63 %
ganzer Text	**1,0 … 1,5**	**ca. 25 %**

Schritt 4 – Shannons Ratespiel.
SHANNON *bestimmte 1951 die Entropie des Englischen mit einem verblüffend einfachen Verfahren: Er ließ Versuchspersonen den* ***nächsten Buchstaben raten*** *und zählte, wie viele Versuche sie brauchten.*

> *Wird der nächste Buchstabe meist im ersten Versuch erraten, ist die Entropie niedrig.* **Shannon erhielt 0,6 bis 1,3 Bit pro Zeichen.**

Schritt 5 – Die Konsequenz für die Kompression.

$$\text{Kompressionsfaktor} \approx \frac{8}{1{,}3} \approx 6.$$

Verfahren	Faktor	Bemerkung
Huffman (Zeichen)	$\approx 1{,}9$	nur Einzelhäufigkeiten
`gzip`	≈ 3	Wörterbuch + Huffman
`xz`, `bzip2`	≈ 4	
theoretische Grenze	≈ 6	Shannons Entropie

> *Moderne, auf Sprachmodellen beruhende Kompressoren kommen der Grenze näher als klassische Verfahren* – **was nicht überrascht: Ein Modell, das den nächsten Buchstaben gut vorhersagt, *ist* genau das, was Shannons Ratespiel gemessen hat.**

Der Zusammenhang ist exakt und nicht bloß analog: *Ein Kompressor und ein Vorhersagemodell sind mathematisch dasselbe* – **man kann jeden in den anderen überführen** (Band 7 und Band 13).

(c) Die zwölf Kugeln.

*Ein Klassiker, den man mit Informationstheorie **vorab** analysieren kann.*

Schritt 1 – Wie viele Fälle gibt es?
Die abweichende Kugel kann jede der 12 sein, und sie kann zu leicht oder zu schwer sein:

$$N = 12 \cdot 2 = 24 \text{ Fälle.}$$

Schritt 2 – Wie viel Information liefert eine Wägung?

*Eine Balkenwaage hat **drei** mögliche Ausgänge: links schwerer, rechts schwerer, Gleichgewicht.*

$$I_{\text{Wägung}} \leq \log_2 3 = 1{,}585 \text{ Bit.}$$

Schritt 3 – Die untere Schranke.

$$k \cdot \log_2 3 \geq \log_2 24 \Longrightarrow k \geq \frac{\log_2 24}{\log_2 3} = \log_3 24.$$

$$\log_3 24 = \frac{\ln 24}{\ln 3} = \frac{3{,}1781}{1{,}0986} = 2{,}893.$$

$$\boxed{k \geq 3}$$

***Drei Wägungen sind die untere Schranke – und sie reichen tatsächlich**, denn $3^3 = 27 \geq 24$. Es bleiben sogar drei Fälle Reserve.*

Schritt 4 – Warum die Reserve knapp ist.
*Mit **dreizehn** Kugeln wären es 26 Fälle, und $27 \geq 26$ – rechnerisch ginge es noch.* **Tatsächlich ist es aber unmöglich**, *weil man in der ersten Wägung nicht alle Kugeln auf die Waage bringen kann und die Aufteilung nicht gleichmäßig genug wird.*

*Die Informationsschranke ist **notwendig**, nicht **hinreichend**.* **Sie sagt, was ausgeschlossen ist – nicht, was möglich ist.**

Schritt 5 – Ein Verfahren.
Wir nummerieren die Kugeln 1 bis 12.
Erste Wägung: $\{1, 2, 3, 4\}$ **gegen** $\{5, 6, 7, 8\}$**.**

Gleichgewicht	Abweichler unter 9–12; 8 Fälle bleiben
Links schwerer	1–4 evtl. schwer *oder* 5–8 evtl. leicht; 8 Fälle
Rechts schwerer	symmetrisch; 8 Fälle

*In jedem Fall bleiben **acht** Möglichkeiten* – **und** $9 = 3^2 \geq 8$**, also reichen zwei weitere Wägungen.** ✓

Fall Gleichgewicht, im Detail.
Zweite Wägung: $\{9, 10, 11\}$ *gegen* $\{1, 2, 3\}$ *(bekannt gute Kugeln).*

Gleichgewicht	Es ist Kugel 12; dritte Wägung klärt schwer/leicht
Links schwerer	Eine von $9, 10, 11$ ist **schwer**; 3 Fälle
Rechts schwerer	Eine von $9, 10, 11$ ist **leicht**; 3 Fälle

Dritte Wägung im Fall „links schwerer“: 9 *gegen* 10. **Schwerer $\Longrightarrow$ diese; Gleichgewicht $\Longrightarrow$ es ist die 11.** ✓

> ***Die Lehre dieser Aufgabe:*** *Eine informationstheoretische Abschätzung kostet zwei Zeilen und sagt einem **vor** dem Nachdenken, ob eine Lösung überhaupt existieren kann.*

Dasselbe Argument liefert die berühmteste untere Schranke der Informatik: *Ein Sortierverfahren, das nur Vergleiche benutzt, muss unter $n!$ Anordnungen unterscheiden, also*

$$\log_2(n!) \approx n \log_2 n - 1{,}44\, n$$

Vergleiche durchführen. **Das ist die Schranke $\Omega(n \log n)$, an der kein Sortieralgorithmus vorbeikommt – Band 1 führt das aus.** Hillis[10] und Petzold[21] entwickeln den Informationsbegriff besonders anschaulich; Shannon[22] ist die Originalarbeit und erstaunlich gut lesbar.

2 Zahlen aus Bits

Das Stellenwertsystem, allgemein

Die Dezimaldarstellung ist eine Konvention, keine Eigenschaft der Zahlen.

Definition 2.1. Zur **Basis** $b \geq 2$ bedeutet die Ziffernfolge $z_{n-1}z_{n-2}\ldots z_1z_0$ mit $0 \leq z_i < b$ die Zahl

$$\sum_{i=0}^{n-1} z_i\, b^i.$$

Beispiel: $(1011)_2 = 1\cdot 8 + 0\cdot 4 + 1\cdot 2 + 1\cdot 1 = 11$. ✓

System	Basis	Ziffern	Wo
binär	2	0, 1	**Maschine**
oktal	8	0–7	Unix-Dateirechte
dezimal	10	0–9	Mensch
hexadezimal	16	0–9, A–F	**Programmierung**

Warum Hexadezimal so praktisch ist.

Weil $16 = 2^4$*, entspricht* ***genau eine*** *Hexadezimalziffer* ***genau vier*** *Bit.* **Die Umrechnung ist reines Ablesen, ohne jede Rechnung.**

$$(1011\,0110)_2 = (\texttt{B6})_{16}, \qquad \text{denn } (1011)_2 = \texttt{B},\ (0110)_2 = 6.$$

Hex	Bin	Hex	Bin	Hex	Bin	Hex	Bin
0	0000	4	0100	8	1000	C	1100
1	0001	5	0101	9	1001	D	1101
2	0010	6	0110	A	1010	E	1110
3	0011	7	0111	B	1011	F	1111

Diese sechzehn Zeilen sollte man auswendig können. **Sie ersparen für den Rest des Lebens jede Umrechnung.**

Die optimale Basis

Eine hübsche Nebenfrage: Welche Basis ist am sparsamsten?

Das Modell. *Eine Ziffernstelle zur Basis b kostet b Aufwand (man braucht b unterscheidbare Zustände). Um Zahlen bis N darzustellen, braucht man $\log_b N$ Stellen. Der Gesamtaufwand ist also*

$$K(b) = b \cdot \log_b N = b \cdot \frac{\ln N}{\ln b}.$$

Minimieren. *Wir leiten nach b ab (Mathematik-Band 3):*

$$K'(b) = \ln N \cdot \frac{\ln b - b \cdot \frac{1}{b}}{(\ln b)^2} = \ln N \cdot \frac{\ln b - 1}{(\ln b)^2}.$$

$$K'(b) = 0 \iff \ln b = 1 \iff b = \mathrm{e} = 2{,}71828.$$

b	$K(b)/\ln N$	**relativ zum Optimum**
2	2,885	+6,1 %
e	2,718	**0**
3	2,731	+0,5 %
4	2,885	+6,1 %
10	4,343	+59,8 %

Die Basis 3 ist der theoretischen Optimalbasis am nächsten – und schlägt die Basis 2 um etwa 6 Prozent. **Genau das war das Argument der Setun-Konstrukteure** (Vorwort).

Warum sich die Zwei trotzdem durchsetzte, steht im Vorwort: *Es gibt keinen dreistufigen Transistor, und sechs Prozent sind kein Argument gegen einen Faktor tausend beim Bauteilpreis.*

2.1 Negative Zahlen

Drei Verfahren, und warum eines gewann

Ein Speicherwort enthält nur Bits. Es gibt kein Minuszeichen – man muss eines erfinden.

Verfahren 1 – Vorzeichen und Betrag. *Das oberste Bit ist das Vorzeichen, der Rest der Betrag.*

$(0000\,0101)_2$	$+5$	$(1000\,0101)_2$	-5
$(0000\,0000)_2$	$+0$	$(1000\,0000)_2$	-0

Verfahren 2 – Einerkomplement. *Negation durch Umkippen aller Bits.*
Verfahren 3 – Zweierkomplement. *Alle Bits umkippen, dann 1 addieren.*

	Vorz./Betrag	Einerkompl.	Zweierkompl.
Zwei Nullen?	ja	ja	**nein**
Addierer	Sonderfälle	Übertrag-rücklauf	**normal**
Bereich (8 Bit)	$-127 \dots 127$	$-127 \dots 127$	$\mathbf{-128 \dots 127}$
Vergleich	Sonderfälle	Sonderfälle	**normal**

Die zweite Zeile hat die Entscheidung gebracht. *Beim Zweierkomplement kann derselbe Addierer vorzeichenbehaftete und vorzeichenlose Zahlen addieren – ohne jede Fallunterscheidung.*

Und das ist keine Kleinigkeit: *Man spart nicht bloß Programmzeilen, sondern* ***Transistoren.*** **Die Subtraktion wird zur Addition:**

$$a - b = a + (-b) = a + (\overline{b} + 1).$$

Ein Rechner braucht also gar kein Subtraktionswerk – **nur einen Inverter und ein Eingangsübertragsbit.**

Warum das Zweierkomplement funktioniert

Die Formel sieht willkürlich aus. Sie ist es nicht.

Satz 2.2. *Bei n Bit stellt das Zweierkomplement die Zahl x durch das Bitmuster der Restklasse*

$$x \bmod 2^n$$

dar. Der Wert eines Musters $z_{n-1} \dots z_0$ ist

$$\boxed{\mathit{Wert} = -z_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} z_i \, 2^i}$$

Das oberste Bit zählt negativ, alle anderen positiv. Mehr ist es nicht. Und daraus folgt alles Weitere.

Nachweis der Negationsregel.

Für jedes Bit gilt $z_i + \overline{z_i} = 1$. Für das ganze Wort:

$$x + \overline{x} = (111\ldots1)_2 = 2^n - 1.$$

$$\Longrightarrow \overline{x} + 1 = 2^n - x \equiv -x \pmod{2^n}. \qquad \checkmark$$

„Umkippen und eins addieren" ist also nichts anderes als „2^n minus die Zahl" – **und modulo 2^n ist das genau die Negation.**

Ein Beispiel mit 8 Bit.

$+5$	$(0000\,0101)_2$	
alle Bits kippen	$(1111\,1010)_2$	das ist -6
$+1$	$(1111\,1011)_2$	**das ist** -5

Probe mit der Wertformel:

$$-1 \cdot 128 + 64 + 32 + 16 + 8 + 0 + 2 + 1 = -128 + 123 = -5. \qquad \checkmark$$

Die Asymmetrie und ihre Folgen

Bei 8 Bit reicht der Bereich von -128 bis $+127$ – *es gibt **eine negative Zahl mehr** als positive.*

Der Grund: Von den 256 Mustern ist eines für die Null verbraucht, und die restlichen 255 teilen sich ungleich.

Daraus folgt ein bemerkenswerter Fehler:

$$-(-128) = -128.$$

Nachrechnen: $-128 = (1000\,0000)_2$; kippen ergibt $(0111\,1111)_2$; plus 1 ergibt $(1000\,0000)_2$ – **dasselbe Muster.** ✓

> ***Der Betrag einer Zahl kann also negativ sein.*** In C ist `abs(INT_MIN)` *undefiniertes Verhalten*, und in Java liefert `Math.abs(Integer.MIN_VALUE)` tatsächlich den negativen Wert zurück.

Das ist eine reale Fehlerquelle, *und sie hat schon Sicherheitslücken verursacht: Wer die Länge eines Puffers mit einer solchen Zahl prüft, kann die Prüfung umgehen.*

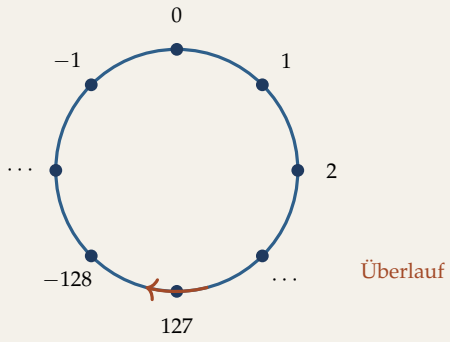
Überlauf

Was passiert, wenn das Ergebnis nicht mehr hineinpasst?

Bei 8 Bit vorzeichenbehaftet:

$$127 + 1 = (0111\,1111)_2 + (0000\,0001)_2 = (1000\,0000)_2 = -128.$$

Der Zahlenbereich ist ein Kreis, kein Strich. Zählt man über die größte Zahl hinaus, landet man bei der kleinsten.



Der teuerste Überlauf der Geschichte

Am 4. Juni 1996 explodierte die Ariane-5-Rakete beim Erstflug nach 37 Sekunden.
Die Ursache: Eine 64-Bit-Gleitkommazahl für die horizontale Geschwindigkeit wurde in eine 16-Bit-Ganzzahl umgewandelt. Der Wert passte nicht.

> *Der Code stammte unverändert aus der Ariane 4, wo der Wert nie so groß geworden war.* **Die Ariane 5 flog steiler.**

Der Schaden betrug etwa 370 Millionen Dollar. *Der ausgelöste Ausnahmezustand wurde als Hardwarefehler gedeutet, der Ersatzrechner führte dieselbe Rechnung aus und fiel eine halbe Sekunde später ebenfalls aus.*

> *Bemerkenswert: Die betroffene Routine wurde nach dem Start gar nicht mehr gebraucht.* **Sie lief nur weiter, weil man sie in der Ariane 4 aus Bequemlichkeit nicht abgeschaltet hatte.**

Zwei Sprachen, zwei Haltungen:

vorzeichenlos	Überlauf ist in C **definiert**: Rechnung modulo 2^n
vorzeichenbehaftet	Überlauf ist in C **undefiniertes Verhalten**

*Die zweite Zeile ist berüchtigt: Der Übersetzer darf annehmen, dass Überlauf **nie eintritt**, und darauf aufbauend optimieren.* **Eine Prüfung der Form `if (x + 1 < x)` kann er ersatzlos streichen** – *sie ist ja „unmöglich" wahr.*

Rust bricht bei Überlauf im Testmodus ab und rechnet im Freigabemodus modulo; Python hat unbegrenzt große Ganzzahlen und kennt das Problem nicht – **um den Preis, dass jede Addition langsamer ist.**

Aufgabe 2.1.

(a) Rechnen Sie zwischen den Systemen um und begründen Sie die Verfahren.

(b) Führen Sie eine vollständige Subtraktion im Zweierkomplement durch, mit Überlaufprüfung.

(c) Untersuchen Sie die Bitmuster-Tricks: Vorzeichen prüfen, mit 2 multiplizieren, Zweierpotenz erkennen.

Lösung Schritt für Schritt.

(a) Umrechnungen.
Teil 1 – Dezimal nach binär: 213.
*Verfahren: Fortgesetzt durch 2 teilen, Reste **von unten nach oben** lesen.*

Division	Ergebnis	Rest
213 : 2	106	**1**
106 : 2	53	**0**
53 : 2	26	**1**
26 : 2	13	**0**
13 : 2	6	**1**
6 : 2	3	**0**
3 : 2	1	**1**
1 : 2	0	**1**

$$213 = (1101\,0101)_2.$$

Warum das Verfahren stimmt.
Ist $z = \sum z_i 2^i$*, so ist* $z \bmod 2 = z_0$ *– der Rest bei Division durch 2 ist genau das unterste Bit.* **Und** $\lfloor z/2 \rfloor$ **schiebt alle Bits um eine Stelle nach rechts.** ✓
Probe:

$$128 + 64 + 16 + 4 + 1 = 213. \qquad ✓$$

Teil 2 – Über Hexadezimal, schneller.

$$213 : 16 = 13 \text{ Rest } 5 \Longrightarrow (\texttt{D5})_{16}.$$

Und $\texttt{D} = (1101)_2$, $5 = (0101)_2$*:*

$$(\texttt{D5})_{16} = (1101\,0101)_2. \qquad ✓$$

Zwei Divisionen statt acht. **Für größere Zahlen ist der Umweg über Hex immer schneller.**

Teil 3 – Binär nach dezimal, das Horner-Schema.
Statt Potenzen zu addieren, liest man von links und verdoppelt:

Bit	Rechnung	Zwischenwert
1	$0 \cdot 2 + 1$	1
1	$1 \cdot 2 + 1$	3
0	$3 \cdot 2 + 0$	6
1	$6 \cdot 2 + 1$	13
0	$13 \cdot 2 + 0$	26
1	$26 \cdot 2 + 1$	53
0	$53 \cdot 2 + 0$	106
1	$106 \cdot 2 + 1$	**213**

Nur Verdopplungen und Additionen – keine Potenzen. **Das ist das Hornerschema** (Mathematik-Band 5)*, und genau so rechnet auch die Maschine.* ✓

Teil 4 – Nachkommastellen.
Verfahren: fortgesetzt mit 2 ***multiplizieren****, Vorkommastellen* ***von oben nach unten*** *lesen. Für* 0,625*:*

$0{,}625 \cdot 2$	1,25	**1**
$0{,}25 \cdot 2$	0,5	**0**
$0{,}5 \cdot 2$	1,0	**1**

$$0{,}625 = (0{,}101)_2 = \tfrac{1}{2} + \tfrac{1}{8}. \qquad \checkmark$$

Für 0,1*:*

$0{,}1 \cdot 2$	0,2	0
$0{,}2 \cdot 2$	0,4	0
$0{,}4 \cdot 2$	0,8	0
$0{,}8 \cdot 2$	1,6	1
$0{,}6 \cdot 2$	1,2	1
$0{,}2 \cdot 2$	0,4	**0 – Wiederholung!**

$0{,}1 = (0{,}0\,\overline{0011})_2$ **– periodisch, unendlich.**

> ***Das ist der Grund für den bekanntesten Ärger der Programmierung.*** 0,1 ist im Binärsystem **keine endliche Zahl** – so wenig wie $\frac{1}{3}$ im Dezimalsystem. **Kapitel 3 zieht die Folgerungen.**

Allgemein: Ein Bruch $\frac{p}{q}$ *in gekürzter Form hat zur Basis* b *genau dann eine endliche Darstellung, wenn jeder Primfaktor von* q *auch* b *teilt* (Mathematik-Band 1). **Zur Basis 2 heißt das:** q **muss eine Zweierpotenz sein.**

(b) Subtraktion im Zweierkomplement.
Aufgabe: $52 - 91$ *mit 8 Bit.*
Schritt 1 – Beide Zahlen darstellen.

$$52 = (0011\,0100)_2, \qquad 91 = (0101\,1011)_2.$$

Probe: $32 + 16 + 4 = 52$ ✓, $64 + 16 + 8 + 2 + 1 = 91$ ✓
Schritt 2 – -91 **bilden.**

91	$(0101\,1011)_2$
Bits kippen	$(1010\,0100)_2$
$+1$	$(1010\,0101)_2$

Probe mit der Wertformel:

$$-128 + 32 + 4 + 1 = -91. \qquad \checkmark$$

Schritt 3 – Addieren.

	`0011 0100`
$+$	`1010 0101`
$=$	`1101 1001`

Spaltenweise nachgerechnet, von rechts:

Stelle	a	b	Übertrag ein	Summe / aus
0	0	1	0	1 / 0
1	0	0	0	0 / 0
2	1	1	0	0 / **1**
3	0	0	1	1 / 0
4	1	0	0	1 / 0
5	1	1	0	0 / **1**
6	0	0	1	1 / 0
7	0	1	0	1 / 0

Schritt 4 – Ergebnis deuten.

$$(1101\,1001)_2: \quad -128+64+16+8+1=-39.$$

$$\boxed{52-91=-39} \qquad \checkmark$$

Schritt 5 – Überlauf prüfen.

> ***Die Regel:*** *Ein vorzeichenbehafteter Überlauf liegt genau dann vor, wenn der Übertrag **in** das oberste Bit sich vom Übertrag **aus** dem obersten Bit unterscheidet.*

Übertrag in Bit 7	0	aus Tabelle
Übertrag aus Bit 7	0	
$0 \oplus 0$	$=0$	**kein Überlauf** ✓

Schritt 6 – Eine anschauliche Merkregel.

$+$ und $+$ ergibt $-$	**Überlauf**
$-$ und $-$ ergibt $+$	**Überlauf**
verschiedene Vorzeichen	nie Überlauf

Die dritte Zeile ist einleuchtend: Die Summe liegt betragsmäßig zwischen den beiden Summanden und kann den Bereich nicht verlassen. ✓

Schritt 7 – Ein Fall *mit* Überlauf.

$$100 + 50: \quad (0110\,0100)_2 + (0011\,0010)_2 = (1001\,0110)_2.$$

$$\text{Wert} = -128 + 16 + 4 + 2 = -106.$$

Zwei positive Zahlen, negatives Ergebnis – **Überlauf.** *Und* $150 - 256 = -106$ ✓ – **es wurde modulo 256 gerechnet.**

(c) Bitmuster-Tricks.
Trick 1 – Vorzeichen prüfen.

*Das oberste Bit **ist** das Vorzeichen. Ein Vergleich mit null ist unnötig:*

$$x < 0 \iff \text{Bit } n-1 \text{ ist gesetzt.}$$

Deshalb ist der Test auf Negativität in jeder Maschinensprache ein einziger Befehl – **man schaut nur ein Bit an.**

Trick 2 – Multiplikation und Division mit Zweierpotenzen.

Operation	**Bitoperation**	**Beispiel**
$x \cdot 2^k$	Linksschieben um k	$(0011)_2 \to (0110)_2$
$x : 2^k$ (positiv)	Rechtsschieben	$(0110)_2 \to (0011)_2$
$x \bmod 2^k$	UND mit $2^k - 1$	

Rechtsschieben negativer Zahlen ist heikel

Es gibt zwei verschiedene Rechtsschiebeoperationen:

logisch	schiebt Nullen nach; für vorzeichenlose Zahlen
arithmetisch	wiederholt das Vorzeichenbit; für vorzeichenbehaftete

Beispiel $-8 = (1111\,1000)_2$*:*

logisch $\gg 1$	$(0111\,1100)_2$	124 falsch
arithmetisch $\gg 1$	$(1111\,1100)_2$	-4 richtig

Und selbst das arithmetische Schieben ist keine Division:

$$-7 \gg 1 = (1111\,1100)_2 = -4, \qquad \text{aber } -7 : 2 = -3{,}5 \to -3.$$

Das Schieben rundet ***abwärts*** *($-3{,}5 \to -4$), die Division in C rundet* ***zur Null hin*** *($-3{,}5 \to -3$).*

Ein Übersetzer, der `x/2` durch `x>>1` ersetzt, muss das korrigieren – *und tut das mit zwei zusätzlichen Befehlen, die man im erzeugten Code sehen kann.*

Trick 3 – Ist x eine Zweierpotenz?

Der eleganteste Trick der Bitmanipulation:

$$\boxed{x \text{ ist Zweierpotenz} \iff x > 0 \text{ und } (x \,\&\, (x-1)) = 0}$$

Warum das stimmt.
Eine Zweierpotenz hat ***genau ein*** *gesetztes Bit. Subtrahiert man 1, so wird dieses Bit gelöscht und alle darunter gesetzt:*

x	x binär	$x-1$ binär	$x \,\&\, (x-1)$
8	$(1000)_2$	$(0111)_2$	$(0000)_2$ ✓
16	$(10000)_2$	$(01111)_2$	$(00000)_2$ ✓
12	$(1100)_2$	$(1011)_2$	$(1000)_2 \neq 0$
7	$(0111)_2$	$(0110)_2$	$(0110)_2 \neq 0$

Allgemein: Ist $x = (\ldots 1\underbrace{00\ldots0}_{k})_2$*, so ist* $x-1 = (\ldots 0\underbrace{11\ldots1}_{k})_2$*.* **Die beiden haben an der Stelle des gekippten Bits und darunter nirgends gemeinsam eine 1.** *Oberhalb sind sie gleich – und dort steht bei einer echten Zweierpotenz nichts.* ✓

> *Der Ausdruck* $x \,\&\, (x-1)$ *hat noch eine zweite Bedeutung:* ***Er löscht das unterste gesetzte Bit.*** Wiederholtes Anwenden zählt daher die gesetzten Bits – **der Kernighan-Trick**, benannt nach BRIAN KERNIGHAN.

Die Zahl der gesetzten Bits heißt ***Hamming-Gewicht*** *oder* ***Population Count****. Sie wird in der Kryptographie und bei Fehlerkorrekturcodes so oft gebraucht, dass moderne Prozessoren dafür einen eigenen Befehl haben* (`POPCNT`).

Ein hübsches Beispiel dafür, wie ein Programmiertrick zur Hardware wird.

Petzold[21] entwickelt die Zahldarstellungen mit großer Geduld; Hyde[12] und Bryant und O'Hallaron[2] behandeln die Fallstricke aus der Sicht des Programmierers; Knuth[15] ist die Referenz für alle Feinheiten der Arithmetik.

3 Gleitkommazahlen

Das Problem

Mit n Bit lassen sich genau 2^n verschiedene Dinge darstellen – nicht mehr. **Die reellen Zahlen sind aber überabzählbar** (Mathematik-Band 10, Kapitel 6). *Schon in jedem noch so kleinen Intervall liegen unendlich viele.*

Ein Rechner kann also nur *endlich viele* Zahlen darstellen. *Die Frage ist nicht „ob man Fehler macht", sondern „welche Zahlen wählt man aus, und wie groß wird der Fehler".*

Zwei Grundentwürfe.

	Festkomma	**Gleitkomma**
Prinzip	Komma an fester Stelle	Komma wandert mit
Abstände	**überall gleich**	**relativ gleich**
Bereich	klein	**riesig**
Genauigkeit	absolut konstant	**relativ konstant**
Verwendung	Finanzsoftware, DSP	**alles Wissenschaftliche**



Beim Gleitkomma stehen die darstellbaren Zahlen nahe null dicht und weit draußen dünn. Der **relative** Fehler bleibt dabei konstant – und das ist bei Messwerten und Naturkonstanten genau das Richtige.

Die Grundidee ist die wissenschaftliche Schreibweise (Mathematik-Band 0, Kapitel 3):

$$299\,792\,458 = 2{,}99792458 \cdot 10^8.$$

Man speichert die ***Mantisse*** 2,99792458 *und den* ***Exponenten*** 8 *getrennt* – **nur eben zur Basis 2.**

3.1 Die Norm IEEE 754

Der Aufbau

Definition 3.1. Eine Gleitkommazahl nach IEEE 754 besteht aus drei Feldern:

$$(-1)^s \cdot 1{,}m \cdot 2^{e-B}$$

mit dem **Vorzeichenbit** s, der **Mantisse** m und dem **Exponenten** e; B heißt **Bias**.



`float`: 32 Bit insgesamt

Typ	Bit	Exponent	Mantisse	Bias
`half`	16	5	10	15
`float`	32	8	23	**127**
`double`	64	11	52	**1023**
`quad`	128	15	112	16 383

Drei Entwurfsentscheidungen, jede mit Grund.
Erstens: das versteckte Bit.

> *Normalisiert man die Mantisse so, dass genau eine Stelle vor dem Komma steht, so ist diese Stelle im Binärsystem* ***immer eine 1*** – denn die einzige andere Ziffer wäre 0, und dann wäre die Zahl nicht normalisiert.

Also braucht man sie nicht zu speichern. *Das schenkt ein Bit Genauigkeit umsonst –* **daher das** $1{,}m$ **in der Formel.**

Im Dezimalsystem ginge dieser Trick nicht: Dort kann die führende Ziffer 1 bis 9 sein. **Ein weiterer Vorteil der Basis 2.**

Zweitens: der Bias statt des Zweierkomplements.
Der Exponent könnte im Zweierkomplement stehen. Stattdessen speichert man $e + B$ als vorzeichenlose Zahl.

echter Exponent	gespeichert	Bitmuster
-126	1	$(0000\,0001)_2$
0	127	$(0111\,1111)_2$
$+127$	254	$(1111\,1110)_2$

Der Vorteil: *Größere Gleitkommazahlen haben auch als* ***Bitmuster*** *größere Werte.* **Man kann zwei positive Gleitkommazahlen mit dem gewöhnlichen Ganzzahlvergleich vergleichen** – *ohne die Felder zu zerlegen.*

Deshalb steht das Vorzeichen ganz links, dann der Exponent, dann die Mantisse – **genau in der Reihenfolge ihrer Wichtigkeit für den Vergleich.**
Drittens: reservierte Muster für Sonderfälle.

Exponent	Mantisse	Bedeutung
alle 0	$= 0$	± 0 (zwei Nullen!)
alle 0	$\neq 0$	**subnormale Zahl**
alle 1	$= 0$	$\pm\infty$
alle 1	$\neq 0$	**NaN** (*not a number*)
sonst	beliebig	normale Zahl

Unendlich und NaN sind Rechenwerte

Das ist eine der klügsten Entscheidungen der Norm. Statt bei einer Division durch null das Programm abzubrechen, liefert IEEE 754 einen **Wert**, mit dem weitergerechnet werden kann.

$1/0$	$+\infty$	$\infty + 1$	∞
$-1/0$	$-\infty$	$\infty - \infty$	**NaN**
$1/\infty$	0	$0/0$	**NaN**
$\sqrt{-1}$	**NaN**	$\infty \cdot 0$	**NaN**

NaN hat eine merkwürdige Eigenschaft:

$$\text{NaN} \neq \text{NaN}.$$

Ein Wert, der nicht einmal sich selbst gleich ist.

Warum das so sein muss

NaN steht für „das Ergebnis einer Rechnung, die kein Ergebnis hat“. *Wäre NaN sich selbst gleich, so wäre* $\sqrt{-1} = 0/0$ **– was offenkundig Unsinn ist.** *Zwei verschiedene undefinierte Rechnungen dürfen nicht als gleich gelten.*

> *Das ergibt einen praktischen Test:* **`x != x` ist wahr genau dann, wenn `x` ein NaN ist** – der kürzeste NaN-Test, den es gibt.

Die Folgen sind allerdings unangenehm. *Sortierverfahren, die Vergleiche benutzen, geraten in Schwierigkeiten: Bei NaN sind* $<$, $>$ *und* $=$ ***alle drei*** *falsch.* **Ein Sortieralgorithmus, der das nicht bedenkt, kann in eine Endlosschleife laufen oder das Feld beschädigen.**

In der Java-Bibliothek war das bis Version 7 eine dokumentierte Falle; heute gibt es einen ausdrücklichen Sortierschlüssel, der NaN definiert einordnet.

Und die zwei Nullen?
Es gilt $+0 = -0$ *beim Vergleich, aber die Bitmuster sind verschieden.* **Das ist kein Versehen:**

$$\frac{1}{+0} = +\infty, \qquad \frac{1}{-0} = -\infty.$$

Das Vorzeichen der Null bewahrt die Information, ***von welcher Seite*** *man sich der Null genähert hat* – **was bei Grenzwerten und in der Funktionentheorie den Unterschied macht** (Mathematik-Band 12).

3.2 Warum $0{,}1 + 0{,}2 \neq 0{,}3$

Der berühmteste Stolperstein

Gibt man in nahezu jeder Programmiersprache `0.1 + 0.2` *ein, so lautet die Antwort*

$$0{,}30000000000000004.$$

Das ist kein Fehler der Sprache und kein Fehler des Prozessors. *Es folgt zwingend aus dem, was wir in Kapitel 2 gerechnet haben.*
Schritt 1 – $0{,}1$ **ist binär periodisch.**

$$0{,}1 = (0{,}0\,\overline{0011})_2 = (0{,}000110011001100\ldots)_2$$

Nach $\frac{p}{q}$ *mit* $q = 10 = 2 \cdot 5$ *und dem Primfaktor 5, der die Basis 2 nicht teilt* (Mathematik-Band 1) **– die Darstellung bricht nie ab.**

Schritt 2 – Also wird gerundet.
Bei `double` stehen 52 Mantissenbit zur Verfügung. Der gespeicherte Wert ist

$$0{,}1000000000000000055511151231257827\ldots$$

genauer:

$$\frac{3602879701896397}{2^{55}}.$$

Schritt 3 – Auch $0{,}2$ wird gerundet.

$$0{,}2 \to \frac{3602879701896397}{2^{54}} = 0{,}2000000000000000111022302462515 65\ldots$$

Schritt 4 – Die Summe.

$$0{,}1 + 0{,}2 \to 0{,}3000000000000000444089209850062616\ldots$$

Schritt 5 – Und $0{,}3$ selbst?

$$0{,}3 \to 0{,}2999999999999999888977697537484 35\ldots$$

Die beiden Werte sind verschieden. Die Summe der gerundeten Werte ist nicht der gerundete Wert der Summe – und der Unterschied beträgt genau ein Bit an der letzten Stelle.

Was daraus folgt

Erstens: Gleitkommazahlen nie auf Gleichheit prüfen.

$$\text{statt } \texttt{a == b} \qquad \text{besser} \qquad |a - b| < \varepsilon.$$

*Und ε sollte **relativ** gewählt werden,* **denn der Abstand zweier benachbarter Gleitkommazahlen wächst mit ihrer Größe:**

$$|a - b| < \varepsilon \cdot \max(|a|, |b|).$$

Zweitens: Die Addition ist nicht assoziativ.

$$(a + b) + c \neq a + (b + c) \qquad \text{im Allgemeinen.}$$

Beispiel in `double`*:*

$$(10^{20} + (-10^{20})) + 1 = 1, \qquad 10^{20} + ((-10^{20}) + 1) = 0.$$

Im zweiten Fall geht die 1 beim Addieren zu 10^{20} verloren – sie liegt weit unterhalb der letzten Mantissenstelle. ✓

> ***Das ist der Grund, warum ein Übersetzer Gleitkommaausdrücke nicht einfach umsortieren darf*** – und warum paralleles Summieren auf verschiedenen Rechnern verschiedene Ergebnisse liefern kann, obwohl beide korrekt rechnen.

Drittens: Für Geld nimmt man kein Gleitkomma.
Man rechnet in ganzzahligen Cents oder benutzt einen Dezimaltyp. **Sonst summieren sich die Rundungsfehler** *– und bei Buchhaltung ist ein Cent Abweichung ein Fehler, kein Rundungsrest.*

Maschinengenauigkeit und Auslöschung

Definition 3.2. Die **Maschinengenauigkeit** ε ist der Abstand von 1 zur nächstgrößeren darstellbaren Zahl.

Typ	ε	Dezimalst.	größte Zahl
`half`	$9{,}8 \cdot 10^{-4}$	≈ 3	65 504
`float`	$1{,}2 \cdot 10^{-7}$	≈ 7	$3{,}4 \cdot 10^{38}$
`double`	$2{,}2 \cdot 10^{-16}$	≈ 16	$1{,}8 \cdot 10^{308}$

Es gilt $\varepsilon = 2^{-p}$, wobei p die Zahl der Mantissenbit einschließlich des versteckten ist – **bei `double` also $2^{-53} = 1{,}11 \cdot 10^{-16}$.**

Damit ist $1 + \varepsilon/2 = 1$ in Maschinenarithmetik – **eine Zahl, die kleiner als die Maschinengenauigkeit ist, verschwindet beim Addieren spurlos.**

Auslöschung – der gefährlichste Effekt

Subtrahiert man zwei fast gleiche Zahlen, so löschen sich die führenden Stellen aus, und die Rundungsfehler rücken nach vorn.
Beispiel mit 6 gültigen Dezimalstellen:

	exakt	**gerundet**
a	1,2345678	1,23457
b	1,2345600	1,23456
$a - b$	0,0000078	0,00001

Der relative Fehler springt von 10^{-6} auf 28 %. **Sechs gültige Stellen sind auf eine geschrumpft.**

Die Abhilfe ist meist algebraisch. *Statt*

$$\sqrt{x+1} - \sqrt{x}$$

rechnet man mit der dritten binomischen Formel

$$\sqrt{x+1} - \sqrt{x} = \frac{(x+1) - x}{\sqrt{x+1} + \sqrt{x}} = \frac{1}{\sqrt{x+1} + \sqrt{x}}.$$

Die zweite Form enthält keine Subtraktion fast gleicher Zahlen – **und liefert für große x volle Genauigkeit. ✓**

Das ist der Kern der numerischen Stabilität – *Mathematik-Band 5 behandelt sie ausführlich.* **Merksatz: Mathematisch gleichwertige Formeln sind numerisch keineswegs gleichwertig.**

Aufgabe 3.1.

(a) Zerlegen Sie das Bitmuster einer `float`-Zahl vollständig und rekonstruieren Sie den Wert.

(b) Bestimmen Sie, ab wann eine `float` nicht mehr jede ganze Zahl darstellen kann.

(c) Analysieren Sie den Patriot-Missile-Vorfall von 1991.

Lösung Schritt für Schritt.

(a) Ein `float` zerlegen.

Gegeben sei das Bitmuster $(\texttt{C1 48 00 00})_{16}$.

Schritt 1 – In Bits übersetzen.

$$\texttt{C} = (1100)_2,\ \texttt{1} = (0001)_2,\ \texttt{4} = (0100)_2,\ \texttt{8} = (1000)_2$$

$$(1100\,0001\,0100\,1000\,0000\,0000\,0000\,0000)_2$$

Schritt 2 – Felder trennen.

Feld	Bit	Inhalt
Vorzeichen	1	$(1)_2$
Exponent	8	$(1000\,0010)_2$
Mantisse	23	$(100\,1000\,0000\ldots0)_2$

Schritt 3 – Vorzeichen.

$$s = 1 \Longrightarrow (-1)^1 = -1 \Longrightarrow \textbf{negativ}.$$

Schritt 4 – Exponent.

$$(1000\,0010)_2 = 128 + 2 = 130.$$

$$e = 130 - 127 = 3.$$

Schritt 5 – Mantisse.

Die gespeicherten Bits sind $100\,1000\ldots$*; das versteckte Bit kommt davor:*

$$1{,}m = (1{,}1001000\ldots0)_2 = 1 + \tfrac{1}{2} + \tfrac{1}{16} = 1{,}5625.$$

Denn die gesetzten Bits stehen an Position 1 und 4 nach dem Komma: $2^{-1} = 0{,}5$ und $2^{-4} = 0{,}0625$. ✓

Schritt 6 – Zusammensetzen.

$$\text{Wert} = -1 \cdot 1{,}5625 \cdot 2^3 = -1{,}5625 \cdot 8$$

$$\boxed{= -12{,}5}$$

Schritt 7 – Gegenprobe von der anderen Seite.

$$12{,}5 = (1100{,}1)_2 = (1{,}1001)_2 \cdot 2^3.$$

Vorzeichen	negativ	$s = 1$
Exponent	$3 + 127 = 130$	$(1000\,0010)_2$
Mantisse	$(1001)_2$ (ohne führende 1)	$(1001\,0\ldots)_2$

Zusammengesetzt ergibt das genau $(\texttt{C1480000})_{16}$. ✓

(b) Wo die ganzen Zahlen ausgehen.
Schritt 1 – Die Überlegung.

> *Eine* `float` *hat 24 gültige Mantissenbit (23 gespeicherte plus das versteckte).* **Damit lassen sich alle ganzen Zahlen bis 2^{24} exakt darstellen** – *denn sie passen in 24 Bit.*

Schritt 2 – Der erste Ausfall.

$$2^{24} = 16\,777\,216.$$

Diese Zahl selbst ist darstellbar ($(1)_2$ mit 24 Nullen, also $1{,}0 \cdot 2^{24}$). ✓
Die nächste Zahl $2^{24} + 1$ *bräuchte 25 Bit* – **sie ist es nicht.**

$$\boxed{16\,777\,216 + 1 = 16\,777\,216 \quad \text{in } \texttt{float}}$$

Schritt 3 – Wie es weitergeht.

Bereich	**Abstand**	**darstellbar**
$0 \ldots 2^{24}$	1	**alle ganzen Zahlen**
$2^{24} \ldots 2^{25}$	2	nur die geraden
$2^{25} \ldots 2^{26}$	4	nur Vielfache von 4
$2^n \ldots 2^{n+1}$	2^{n-23}	

Schritt 4 – Für `double`.

$$2^{53} = 9\,007\,199\,254\,740\,992 \approx 9 \cdot 10^{15}.$$

> ***Das ist eine praktisch wichtige Zahl.*** JavaScript kennt nur `double` als Zahlentyp; deshalb ist dort 2^{53} die größte sicher verwendbare Ganzzahl – **und heißt in der Sprache ausdrücklich `MAX_SAFE_INTEGER`.**

Schritt 5 – Eine reale Folge.
Datenbank-Identifikatoren sind heute oft 64-Bit-Ganzzahlen und überschreiten 2^{53}. **Werden sie über eine JSON-Schnittstelle an ein JavaScript-Programm übertragen, verändern sie sich stillschweigend.**

Twitter geriet 2011 genau in dieses Problem und musste jede Nachrichten-Kennung zusätzlich als ***Zeichenkette*** *mitliefern* – **das Feld heißt bis heute `id_str`.**

(c) Der Patriot-Vorfall.
Schritt 1 – Was geschah.
Am 25. Februar 1991 verfehlte eine Patriot-Abwehrrakete in Dhahran, Saudi-Arabien, eine anfliegende Scud-Rakete. Diese traf eine Unterkunft der US-Armee. **28 Soldaten starben, etwa 100 wurden verletzt.**
Schritt 2 – Die Ursache.
Die interne Uhr zählte Zehntelsekunden seit dem Einschalten als Ganzzahl. Zur Berechnung der Zielposition musste daraus eine Zeit in Sekunden werden:

$$t = \text{Ticks} \cdot 0{,}1.$$

Der Faktor $0{,}1$ *wurde in einem 24-Bit-Festkommaregister gespeichert* – **und** $0{,}1$ **ist binär periodisch** (Kapitel 2).

Schritt 3 – Der Fehler pro Tick.

$$0{,}1 = (0{,}0001100110011001100110\,0110\ldots)_2$$

Nach 24 Bit wurde abgeschnitten. Der abgeschnittene Rest beträgt etwa

$$\Delta \approx 9{,}5 \cdot 10^{-8}.$$

Schritt 4 – Der Fehler nach 100 Betriebsstunden.

$$100\ \text{h} = 360\,000\ \text{s} = 3\,600\,000\ \text{Ticks}.$$

$$\text{Zeitfehler} = 3\,600\,000 \cdot 9{,}5 \cdot 10^{-8}\ \text{s} \approx 0{,}34\ \text{s}.$$

Schritt 5 – Was ein Drittel einer Sekunde bedeutet.
Eine Scud fliegt mit etwa $1676\ \frac{\text{m}}{\text{s}}$*:*

$$0{,}34\ \text{s} \cdot 1676\ \tfrac{\text{m}}{\text{s}} \approx 570\ \text{m}.$$

Das Suchfenster des Radars lag 570 Meter neben dem tatsächlichen Ziel.
Die anfliegende Rakete wurde als Fehlmessung verworfen.

Schritt 6 – Was die Sache besonders bitter macht.

Der Fehler war bekannt	Israelische Einheiten hatten ihn zwei Wochen zuvor gemeldet
Die Korrektur existierte	Sie traf einen Tag nach dem Angriff ein
Die Behelfsanweisung lautete	„Nicht zu lange am Stück betreiben“
Die Batterie lief	rund **100 Stunden**

Die eigentliche Lehre

Es war kein Rechenfehler im engeren Sinn. *Jede einzelne Multiplikation war so genau, wie das Format es zuließ.*

> *Der Fehler entstand durch **Akkumulation**: ein winziger, systematischer Fehler, dreieinhalb Millionen Mal wiederholt.* **Rundungsfehler heben sich nicht immer auf** – *wenn sie stets in dieselbe Richtung gehen, addieren sie sich.*

Die Abhilfe wäre einfach gewesen: *Man zählt Ticks als Ganzzahl und rechnet **einmal** am Ende in Sekunden um* – **statt bei jedem Schritt eine fehlerbehaftete Multiplikation zu wiederholen.**

> *Derselbe Gedanke steckt hinter der Empfehlung, Geldbeträge in Cents zu führen* (oben). **Man verschiebt die Ungenauigkeit an eine Stelle, an der sie sich nicht mehr fortpflanzen kann.**

Zur Person, der wir die Norm verdanken. WILLIAM KAHAN *setzte IEEE 754 in den späten 1970er Jahren gegen erheblichen Widerstand durch* – **jeder Hersteller hatte damals sein eigenes, meist schlechteres Gleitkommaformat.**

> *Kahan erhielt 1989 den Turing-Preis. Sein Argument war nicht Genauigkeit, sondern **Reproduzierbarkeit**: dieselbe Rechnung soll auf jeder Maschine dasselbe Ergebnis liefern.* **Vor 754 war das nicht so** – *und wissenschaftliche Ergebnisse waren zwischen Rechnern nicht vergleichbar.*

Goldberg[7] ist der klassische Aufsatz zum Thema und Pflichtlektüre; Overton[19] und Muller u. a.[17] führen tiefer; Feynman[6] behandelt die Arithmetik von der physikalischen Seite.

4 Zeichen, Texte und alles Übrige

Die Grundeinsicht

Ein Bitmuster hat für sich genommen keine Bedeutung. Dieselben acht Bit sind je nach Deutung eine Zahl, ein Buchstabe, ein Grauwert oder ein Maschinenbefehl.

Das Muster $(0100\,0001)_2 = (41)_{16} = 65$	
als Ganzzahl	65
als Zeichen (ASCII)	`'A'`
als Grauwert	dunkles Grau (von 255)
als x86-Befehl	`INC ECX`
als Farbanteil	25,5 % Rot

Die Bedeutung liegt nicht im Bit, sondern in der Verabredung.

Das ist mehr als eine technische Feststellung. **Es ist der Grund, warum ein Rechner universell sein kann:** *Weil ein Bitmuster alles bedeuten darf, kann dieselbe Maschine Texte setzen, Bilder rechnen und Programme ausführen.*

Und es ist der Keim des Selbstbezugs (Kapitel 14): *Wenn Befehle Zahlen sind und Zahlen Daten, dann sind Programme Daten* – **und ein Programm kann ein Programm lesen.**

4.1 Von ASCII zu Unicode

ASCII, 1963

Sieben Bit, 128 Zeichen. Die Anordnung ist keineswegs willkürlich.

Bereich	Hex	Inhalt
0–31	00–1F	Steuerzeichen
32	20	Leerzeichen
48–57	30–39	**Ziffern 0–9**
65–90	41–5A	**A–Z**
97–122	61–7A	**a–z**
127	7F	DEL

Drei Entwurfsentscheidungen, die sich bis heute auswirken.
Erstens: Die Ziffern beginnen bei $(30)_{16}$.

$$\text{Ziffernwert} = \text{Code} - 48.$$

Und in Hexadezimal ist $(30)_{16} + n$ *einfach* $3n$ **– man liest den Ziffernwert direkt in der zweiten Hexstelle ab.** ✓
Zweitens: Groß- und Kleinbuchstaben unterscheiden sich in *genau einem* Bit.

$$\texttt{'A'} = (100\,0001)_2, \qquad \texttt{'a'} = (110\,0001)_2.$$

Nur Bit 5 unterscheidet sich, denn $97 - 65 = 32 = 2^5$.

$$\text{Kleinschreiben} = \text{ODER mit } (20)_{16}, \qquad \text{Großschreiben} = \text{UND mit } (\texttt{DF})_{16}.$$

Ein einziger Maschinenbefehl statt einer Fallunterscheidung. Das war 1963 ein ernsthaftes Argument – Rechenzeit war teuer.

Drittens: $(7F)_{16}$ für `DEL`.
Auf Lochstreifen konnte man ein Zeichen nicht löschen – man konnte nur ***alle Löcher stanzen****.* $(111\,1111)_2$ **ist genau das –** *und bedeutet deshalb „ignoriere dieses Zeichen“.*

Ein Detail aus der Lochstreifentechnik, das sich in jede moderne Tastatur vererbt hat.

Das achte Bit und die Codepage-Hölle

ASCII benutzt nur 7 Bit; ein Byte hat 8. *Die 128 freien Plätze wurden ab den 1980er Jahren von jedem Land anders belegt.*

Kodierung	Position 228	Region
Latin-1	ä	Westeuropa
Latin-2	ä	Osteuropa
KOI8-R	kyrillisch	Russland
CP437	griech. Sigma	DOS
Mac Roman	Doppelkreuz	Apple

Ein Text trug seine Kodierung nicht mit sich. Wer sie nicht kannte, sah Kauderwelsch – im Japanischen entstand dafür sogar ein eigenes Wort: *mojibake*, „Zeichenverwandlung“.

Und es war schlimmer als Ärger: *Chinesisch, Japanisch und Koreanisch passen nicht in 256 Plätze.* **Es entstanden Mehrbyte-Kodierungen mit eigenen, zueinander unverträglichen Regeln.**

Unicode: ein Platz für jedes Zeichen

Die Idee von 1988: ***Man trennt die Frage „welches Zeichen“ von der Frage „wie speichere ich es“.***

Codepoint	Eine Nummer für jedes Zeichen der Welt, geschrieben `U+0041`
Kodierung	Wie diese Nummer in Bytes zerlegt wird

Der Raum umfasst 1 114 112 **Positionen** ($17 \cdot 2^{16}$), *von denen Stand 2026 etwa* 155 000 *belegt sind.*
Die wichtigste Kodierung ist UTF-8 (Ken Thompson und Rob Pike, 1992, angeblich auf einer Papierserviette in einem Diner entworfen).

Bereich	Bytes	Muster
U+0000–007F	1	0xxxxxxx
U+0080–07FF	2	110xxxxx 10xxxxxx
U+0800–FFFF	3	1110xxxx 10xxxxxx 10xxxxxx
U+10000–10FFFF	4	11110xxx + 3×10xxxxxx

Die Eigenschaften, die UTF-8 gewinnen ließen:

ASCII-verträglich	Jeder ASCII-Text ist bereits gültiges UTF-8
selbstsynchronisierend	Am Bitmuster erkennt man, ob ein Byte ein Anfang ist
keine Nullbytes	C-Zeichenketten funktionieren weiter
Reihenfolge erhalten	Bytevergleich sortiert wie Codepointvergleich

Die zweite Zeile ist die technisch klügste. Beginnt ein Byte mit `10`, so ist es ein Folgebyte; sonst ein Anfang. **Springt man mitten in einen Text, findet man in höchstens drei Schritten den nächsten Zeichenanfang** – *bei anderen Mehrbyte-Kodierungen muss man von vorn lesen.*

Stand 2026 sind über 98 Prozent aller Webseiten in UTF-8 kodiert – **eine der wenigen vollständig gewonnenen Normungsschlachten der Informatik.**

Warum Text schwieriger ist, als er aussieht

Unicode löste das Kodierungsproblem und legte dabei frei, wie kompliziert Schrift wirklich ist.

Erstens: Ein Zeichen ist nicht ein Codepoint.

`e´` = `U+00E9` oder `e` + `U+0301` (Akut).

Beide sehen gleich aus und sind verschiedene Bytefolgen. **Ein Vergleich Byte für Byte scheitert.**

*Die Lösung heißt **Normalisierung**: Vor jedem Vergleich bringt man den Text in eine von vier Normalformen (NFC, NFD, NFKC, NFKD).* **Wer das vergisst, baut Anmeldeverfahren, bei denen dasselbe Passwort mal passt und mal nicht.**
Zweitens: Ein Zeichen ist nicht ein Buchstabe.

Beispiel	Codepoints	wahrgenommen
`ä` (zerlegt)	2	1 Zeichen
Familien-Emoji	bis 7	1 Bild
Devanagari-Silbe	3–4	1 Silbe
Flaggen-Emoji	2	1 Flagge

*Die Einheit, die ein Mensch als „ein Zeichen" sieht, heißt **Graphemcluster*** – **und die Regeln dafür füllen ein eigenes Normdokument.**

Drittens: Es gibt keine sprachunabhängige Sortierung.

Deutsch	`ae`-Umlaut sortiert wie `a` (DIN 5007-1)
Telefonbuch	Umlaut sortiert wie `ae` (DIN 5007-2)
Schwedisch	Umlaut steht **hinter** `z`
Türkisch	i mit und ohne Punkt sind **verschieden**

Das türkische I

Im Türkischen hat `i` als Großbuchstaben ein I mit Punkt, und `I` hat ein i ohne Punkt.
Ein Programm, das eine Zeichenkette „in Großbuchstaben" umwandelt, um sie zu vergleichen, macht auf einem türkisch eingestellten Rechner aus `"file"` ein Wort mit gepunktetem grossen I – **und findet die Datei nicht mehr.**

> *Der Fehler ist so verbreitet, dass er einen eigenen Namen hat: das **Turkish-I-Problem**.* **Er hat unter anderem in .NET-Anwendungen und in Android reale Ausfälle verursacht.**

Die Regel lautet: *Für Vergleiche niemals die **sprachabhängige** Umwandlung benutzen* – **sondern eine ausdrücklich als „invariant" gekennzeichnete.**

Ein Musterfall dafür, dass „Text" keine einfache Datenstruktur ist, sondern kulturelle Konventionen mit sich trägt.

4.2 Bilder, Töne, alles andere

Dasselbe Prinzip

Für jede Art von Daten gilt derselbe Dreischritt: **abtasten, quantisieren, kodieren.**

Schritt	Bild	Ton
Abtasten	Bildpunkte (Pixel)	Zeitpunkte (44 100/s)
Quantisieren	8 Bit je Farbkanal	16 Bit je Abtastwert
Kodieren	PNG, JPEG	FLAC, MP3

Warum ausgerechnet 44 100 Abtastungen pro Sekunde?

Satz 4.1 (Abtasttheorem; Nyquist 1928, Shannon 1949)**.** *Ein Signal, das keine Frequenzen oberhalb $f_{\max}$ enthält, ist durch Abtastung mit der Rate*

$$f_s > 2\, f_{\max}$$

***vollständig** bestimmt – es lässt sich exakt rekonstruieren.*

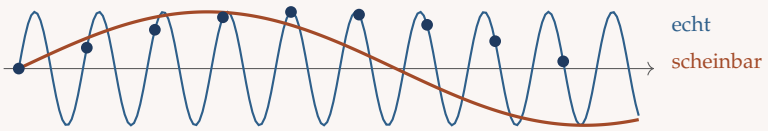
> *Das ist ein bemerkenswerter Satz:* ***Aus endlich vielen Zahlen pro Sekunde lässt sich ein kontinuierliches Signal* exakt *zurückgewinnen*** – vorausgesetzt, es war vorher bandbegrenzt.

Das menschliche Gehör reicht bis etwa 20 kHz. Also braucht man mehr als 40 000 Abtastungen pro Sekunde. **Die 44 100 kamen daher, dass die ersten Digitalaufnahmen auf Videobändern gespeichert wurden** – *und* $44\,100 = 3 \cdot 5^2 \cdot 7^2 \cdot 12$ *passte zu den Zeilenzahlen sowohl des PAL- als auch des NTSC-Systems.*

Eine Zahl, die heute jeden Musikkonsum bestimmt und aus der Fernsehtechnik der 1970er Jahre stammt.

Was passiert, wenn man zu langsam abtastet

Frequenzen oberhalb der halben Abtastrate erscheinen als *falsche, niedrigere* Frequenzen. *Man nennt das **Aliasing**.*



*Die Abtastpunkte passen auf **beide** Kurven – man kann sie nicht unterscheiden.*

> *Man sieht dasselbe im Film, wenn Wagenräder rückwärts zu laufen scheinen* – **die Bildrate von 24 Bildern pro Sekunde tastet die Drehung zu langsam ab.**

Die Abhilfe ist ein Antialiasingfilter *vor* der Abtastung, *der alle zu hohen Frequenzen entfernt.* **Nachher geht es nicht mehr** – *die Information ist dann unwiederbringlich falsch.*

Aufgabe 4.1.

(a) Kodieren und dekodieren Sie UTF-8 von Hand und weisen Sie die Selbstsynchronisation nach.

(b) Berechnen Sie den Speicherbedarf von Bild und Ton und vergleichen Sie mit realen Formaten.

(c) Warum ist `"Straße".toUpperCase()` ein Problem? Untersuchen Sie Textlängen.

Lösung Schritt für Schritt.

(a) UTF-8 von Hand.
Teil 1 – Kodieren: `ä`, also `U+00E4`.
Schritt 1 – Bereich bestimmen.

$$(\texttt{E4})_{16} = 228, \qquad 128 \le 228 \le 2047.$$

*Also **zwei** Bytes im Muster* `110xxxxx 10xxxxxx`.

Schritt 2 – Codepoint binär.

$$228 = (1110\,0100)_2 \Longrightarrow \text{auf 11 Bit: } (000\,1110\,0100)_2.$$

Elf Bit, weil das Zweibytemuster $5 + 6 = 11$ *Nutzbits hat.* ✓
Schritt 3 – Aufteilen.

	Nutzbits	Byte	
obere 5	$(00011)_2$	$(110\,00011)_2$	$=$ $(\texttt{C3})_{16}$
untere 6	$(100100)_2$	$(10\,100100)_2$	$=$ $(\texttt{A4})_{16}$

$$\boxed{\texttt{ä} = (\texttt{C3})_{16}\,(\texttt{A4})_{16}}$$

Deshalb erscheint ein falsch als Latin-1 gelesener UTF-8-Text mit dem berüchtigten `Ã¤` **– denn** $(\texttt{C3})_{16}$ **ist in Latin-1 das Zeichen Ã und** $(\texttt{A4})_{16}$ **das Zeichen ¤. ✓**

Teil 2 – Dekodieren: $(\texttt{F0})_{16}\,(\texttt{9F})_{16}\,(\texttt{98})_{16}\,(\texttt{80})_{16}$**.**
Schritt 1 – Erstes Byte lesen.

$$(\texttt{F0})_{16} = (1111\,0000)_2.$$

Es beginnt mit `11110` **– vier Bytes, drei Nutzbits im ersten.**
Schritt 2 – Nutzbits sammeln.

Byte	binär	Präfix	Nutzbits
$(\texttt{F0})_{16}$	$(11110\,000)_2$	`11110`	$(000)_2$
$(\texttt{9F})_{16}$	$(10\,011111)_2$	`10`	$(011111)_2$
$(\texttt{98})_{16}$	$(10\,011000)_2$	`10`	$(011000)_2$
$(\texttt{80})_{16}$	$(10\,000000)_2$	`10`	$(000000)_2$

Schritt 3 – Zusammenhängen.

$$(000\;011111\;011000\;000000)_2 = (0\,0001\,1111\,0110\,0000\,0000)_2.$$

In Vierergruppen von rechts: $(1)_2\,(1111)_2\,(0110)_2\,(0000)_2\,(0000)_2$

$$= (\texttt{1F600})_{16}.$$

Das ist `U+1F600`*, das grinsende Gesicht. ✓*

Teil 3 – Die Selbstsynchronisation nachweisen.

Behauptung: An jedem einzelnen Byte erkennt man seine Rolle.

Beginnt mit	Rolle	Wertebereich
`0`	Einzelbyte	$(\texttt{00})_{16}$–$(\texttt{7F})_{16}$
`10`	**Folgebyte**	$(\texttt{80})_{16}$–$(\texttt{BF})_{16}$
`110`, `1110`, `11110`	**Startbyte**	$(\texttt{C2})_{16}$–$(\texttt{F4})_{16}$

Die Bereiche überschneiden sich nicht. Springt man an eine beliebige Stelle, so geht man rückwärts, bis ein Byte **nicht** mit `10` beginnt – und höchstens drei Schritte später ist man am Zeichenanfang.

Der praktische Wert ist erheblich: *Man kann eine Datei an beliebiger Stelle aufteilen und parallel verarbeiten, man kann rückwärts suchen, und ein einzelnes verlorenes Byte zerstört nur **ein** Zeichen* – **nicht den ganzen Rest der Datei.**

(b) Speicherbedarf.
Teil 1 – Ein Bild.
Ein Foto mit 6000×4000 *Bildpunkten, drei Farbkanäle zu je 8 Bit:*

$$6000 \cdot 4000 \cdot 3 = 72\,000\,000 \text{ Byte} = 68{,}7 \text{ MiB}.$$

Format	Größe	Verfahren
unkomprimiert	68,7 MiB	—
PNG	$\approx$ 40 MiB	**verlustfrei**
JPEG (Q90)	$\approx$ 8 MiB	verlustbehaftet
JPEG (Q50)	$\approx$ 2 MiB	sichtbare Artefakte

Der Faktor 30 bei JPEG ist nur möglich, weil Information* weggeworfen *wird – und zwar gezielt solche, die das Auge schlecht wahrnimmt: feine Farbunterschiede stärker als feine Helligkeitsunterschiede.

Das ist ein wesentlicher begrifflicher Unterschied:

verlustfrei	Original exakt rekonstruierbar; Grenze ist die Entropie (Kapitel 1)
verlustbehaftet	Original **nicht** rekonstruierbar; Grenze ist die Wahrnehmung

Teil 2 – Ton.
CD-Qualität: 44 100 Abtastungen pro Sekunde, 16 Bit, 2 Kanäle:

$$44\,100 \cdot 2 \cdot 2 = 176\,400 \ \tfrac{\text{Byte}}{\text{s}} = 1{,}41 \ \tfrac{\text{Mbit}}{\text{s}}.$$

Für 74 Minuten:

$$176\,400 \cdot 4440 \text{ s} = 783 \text{ MB}.$$

Und genau das ist die Kapazität einer Audio-CD. **Die 74 Minuten waren die Vorgabe; die 783 MB ergaben sich daraus.** ✓

> *Die verbreitete Erzählung, die Spielzeit sei gewählt worden, damit Beethovens Neunte auf eine Scheibe passt, ist wahrscheinlich eine nachträgliche Legende –* **Sony und Philips selbst haben sie mehrfach unterschiedlich dargestellt.**

Teil 3 – Text zum Vergleich.
Ein 300-seitiges Buch, etwa 2000 Zeichen pro Seite:

$$600\,000 \text{ Zeichen} \approx 0{,}6 \text{ MB}.$$

Inhalt	Größe	Verhältnis
Ein ganzes Buch	0,6 MB	1
Eine Minute Musik	10 MB	17×
Ein Foto	8 MB	13×
Eine Minute Video (HD)	100 MB	170×

> ***Text ist erstaunlich billig.*** Die gesamte deutschsprachige Wikipedia passt komprimiert in wenige Gigabyte – **weniger als ein Spielfilm in hoher Auflösung.**

(c) Textlängen und das scharfe S.
Schritt 1 – Das Problem.

```
"Straße".toUpperCase() = "STRASSE".
```

Aus sechs Zeichen werden ***sieben****.*

> ***Die Umwandlung in Großbuchstaben ist keine zeichenweise Abbildung.*** Sie kann die Länge verändern – und sie ist **nicht umkehrbar**: Aus `"STRASSE"` lässt sich `"Straße"` nicht zurückgewinnen, denn `"Strasse"` wäre ebenso richtig.

Schritt 2 – Warum das reale Fehler verursacht.

Puffer zu klein	Wer n Zeichen einliest und n Zeichen Platz reserviert, läuft über
Längenprüfung	„höchstens 20 Zeichen“ kann nach der Umwandlung verletzt sein
Datenbankfeld	Feste Feldlänge reicht plötzlich nicht

Seit 2017 gibt es zwar das offizielle große Eszett grosse Eszett (`U+1E9E`), aber die Standardumwandlung liefert weiterhin `SS` – **aus Rücksicht auf bestehende Texte.**

Schritt 3 – Was ist dann die „Länge“ eines Textes?

Betrachten wir die Zeichenkette `"Straße"` + *Familien-Emoji:*

Maß	**Wert**	**wo benutzt**
Bytes (UTF-8)	$7 + 25$	Speicher, Netz
Codepoints	$6 + 7$	Python 3, Go
UTF-16-Einheiten	$6 + 11$	**Java, JavaScript, C#**
Graphemcluster	$6 + 1$	**was der Mensch sieht**
Spaltenbreite	$6 + 2$	Terminalanzeige

> ***Fünf verschiedene, jeweils sinnvolle Antworten auf die Frage „wie lang ist dieser Text?“*** – und Programmiersprachen geben unterschiedliche davon zurück, ohne das immer deutlich zu machen.

Schritt 4 – Die berüchtigte dritte Zeile.

Java und JavaScript entstanden Anfang der 1990er Jahre, als Unicode noch 16 Bit umfasste. Ein `char` *war ein Zeichen.*

1996 wurde der Raum auf 1 114 112 *Positionen erweitert.* **Zeichen oberhalb von `U+FFFF` werden seither durch *zwei* 16-Bit-Einheiten dargestellt** (**Surrogatpaare**).

Folge: In JavaScript liefert `"<Emoji>".length` *den Wert* **2**. **Und wer eine solche Zeichenkette in der Mitte teilt, erhält zwei ungültige Bruchstücke.**

> *Das ist ein Musterbeispiel für eine **Abstraktion, die undicht geworden ist**: Eine Entwurfsentscheidung von 1993 – „16 Bit reichen für alle Zeichen“ – wirkt sich dreißig Jahre später in jedem Browser aus, weil man sie nicht mehr ändern kann, ohne bestehenden Code zu brechen.*

Die Lehre für den ganzen Band: *Jede Kodierung ist eine* ***Verabredung****, und Verabredungen veralten.* **Was bleibt, ist die Einsicht, dass Bits erst durch eine Deutung Bedeutung erhalten** – *und dass man diese Deutung stets ausdrücklich mitführen muss.* Petzold[21] entwickelt die Kodierungen historisch; Bryant und O'Hallaron[2] behandeln die Datenrepräsentation systematisch; Kernighan und Ritchie[13] zeigen, wie eng C an das Bytemodell gebunden ist.

Part II

Die Logik

5 Die Boolesche Algebra

Booles Idee

GEORGE BOOLE *veröffentlichte 1847 eine Schrift mit dem Titel „The Mathematical Analysis of Logic". Sein Gedanke war ungeheuerlich:* **Man könne die Gesetze des Denkens *ausrechnen* wie eine Gleichung.**

Der Trick besteht darin, Aussagen die Werte 0 und 1 zu geben – *falsch und wahr* – **und dann Rechenregeln dafür aufzustellen.**

Definition 5.1. Eine **Boolesche Algebra** ist eine Menge $\mathbb{B} = \{0, 1\}$ mit drei Verknüpfungen:

Name	Zeichen	Bedeutung
UND	$a \wedge b$, $a \cdot b$	beide wahr
ODER	$a \vee b$, $a + b$	mindestens eines wahr
NICHT	$\overline{a}$, $\neg a$	Umkehrung

Die vollständigen Wertetafeln:

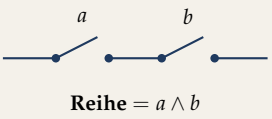
a	b	$a \wedge b$	$a \vee b$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

a	$\overline{a}$
0	1
1	0

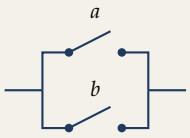
Man beachte: ***Das ODER ist einschließend.*** $1 \vee 1 = 1$. Das umgangssprachliche „entweder oder" ist eine andere Verknüpfung – das **XOR**, das wir gleich behandeln.

Shannons Übersetzung: Schalter

Das Entscheidende ist nicht die Algebra, sondern ihre Deutung.



Reihe $= a \wedge b$



Parallel $= a \vee b$

> ***Zwei Schalter in Reihe leiten nur, wenn beide geschlossen sind*** *– das ist das UND.* **Zwei Schalter parallel leiten, wenn mindestens einer geschlossen ist** *– das ist das ODER.*

Das ist Shannons Entdeckung von 1937 in zwei Bildern. *Damit war eine hundert Jahre alte, für rein philosophisch gehaltene Algebra plötzlich das Werkzeug der Elektrotechnik.*

Zwei Männer, die dasselbe fanden

George Boole (1815–1864) war Autodidakt. *Sohn eines Schusters, ohne Universitätsabschluss, brachte er sich Griechisch, Latein und Mathematik selbst bei und wurde 1849 – ohne je studiert zu haben – Professor in Cork.*

> *Sein Hauptwerk „An Investigation of the Laws of Thought“ (1854) galt zu Lebzeiten als sonderbare Grenzarbeit zwischen Mathematik und Philosophie.* **Praktische Anwendungen sah niemand.**

Boole starb mit 49 an einer Lungenentzündung, nachdem er im Regen zur Vorlesung gegangen war und seine Frau ihn – einer damaligen Heillehre folgend – mit kaltem Wasser übergossen hatte.

Claude Shannon (1916–2001) war 21, als er die Verbindung fand.

> *Er hatte im Nebenfach Philosophie eine Vorlesung über Boolesche Logik gehört – damals eine Seltenheit – und arbeitete zugleich an Vannevar Bushs „Differential Analyzer“, einem Analogrechner mit hunderten von Relais.*

Die Verbindung fiel ihm auf, weil er als Einziger beides kannte.

> *Shannon blieb zeitlebens für seine Verspieltheit bekannt: Er baute eine jonglierende Maschine, ein schachspielendes Gerät, eine „Ultimate Machine“, deren einzige Funktion darin bestand, sich selbst abzuschalten –* **und fuhr auf den Fluren der Bell Labs Einrad.**

5.1 Die Rechengesetze

Was gilt, und was überraschend ist

Gesetz	für UND	für ODER
Kommutativ	$a \wedge b = b \wedge a$	$a \vee b = b \vee a$
Assoziativ	$(a \wedge b) \wedge c = a \wedge (b \wedge c)$	entsprechend
Neutral	$a \wedge 1 = a$	$a \vee 0 = a$
Extrem	$a \wedge 0 = 0$	$a \vee 1 = 1$
Idempotenz	$a \wedge a = a$	$a \vee a = a$
Komplement	$a \wedge \overline{a} = 0$	$a \vee \overline{a} = 1$
Distributiv	$a \wedge (b \vee c)$	$= (a \wedge b) \vee (a \wedge c)$
Distributiv (2)	$a \vee (b \wedge c)$	$= (a \vee b) \wedge (a \vee c)$
Absorption	$a \wedge (a \vee b) = a$	$a \vee (a \wedge b) = a$
Doppelnegation	$\overline{\overline{a}} = a$	

Zwei Gesetze verdienen Aufmerksamkeit, weil sie in der gewöhnlichen Arithmetik *falsch* sind.
Erstens die Idempotenz: $a \wedge a = a$ und $a \vee a = a$.

In den Zahlen gilt $2 + 2 = 4$, *nicht* 2. **Hier gilt** $1 \vee 1 = 1$ – *denn „wahr oder wahr" ist eben wahr, nicht „doppelt wahr".*

Zweitens das *zweite* Distributivgesetz:

$$a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c).$$

Übersetzt in Zahlen hieße das $a + b \cdot c = (a + b) \cdot (a + c)$ **– und das ist offenkundig falsch.** *In der Booleschen Algebra ist es richtig.*

Der Nachweis durch vollständige Wertetafel.

a	b	c	$b \wedge c$	$a \vee (b \wedge c)$	$a \vee b$	$a \vee c$	$(a \vee b) \wedge (a \vee c)$
0	0	0	0	**0**	0	0	**0**
0	0	1	0	**0**	0	1	**0**
0	1	0	0	**0**	1	0	**0**
0	1	1	1	**1**	1	1	**1**
1	0	0	0	**1**	1	1	**1**
1	0	1	0	**1**	1	1	**1**
1	1	0	0	**1**	1	1	**1**
1	1	1	1	**1**	1	1	**1**

Spalte 5 und Spalte 8 stimmen in allen acht Zeilen überein. ✓

Das ist zugleich das Grundverfahren dieses Kapitels: Bei n Variablen hat die Wertetafel 2^n Zeilen. Für kleine n ist das ein **vollständiger, mechanischer Beweis** – man muss nur alle Fälle hinschreiben.

Und für große n?

Bei 30 Variablen hat die Tafel $2^{30} \approx 10^9$ Zeilen – *noch machbar.* **Bei 100 Variablen sind es 10^{30}** – *aussichtslos.*

> *Die Frage, ob es ein grundsätzlich besseres Verfahren gibt, um zu prüfen, ob ein Boolescher Ausdruck erfüllbar ist, heißt* ***SAT-Problem*** **– und sie ist das erste als NP-vollständig erkannte Problem** (Cook 1971).

Ob es effizient lösbar ist, ist die Frage „$\mathrm{P} = \mathrm{NP}$?“ – *das bekannteste offene Problem der Informatik und Gegenstand von Band 3.*

Bemerkenswert: Die harmloseste Struktur dieses Bandes – eine Tabelle aus Nullen und Einsen – führt auf direktem Weg zum größten offenen Problem des Fachs.

5.2 De Morgan

Satz 5.2 (Gesetze von De Morgan, 1847)**.**

$$\overline{a \wedge b} = \overline{a} \vee \overline{b}, \qquad \overline{a \vee b} = \overline{a} \wedge \overline{b}$$

Das wichtigste Gesetz der digitalen Technik

In Worten: *„Nicht (A und B)“ ist dasselbe wie „nicht A oder nicht B“.*

*Anschaulich: Wenn es nicht wahr ist, dass **beide** Bedingungen gelten, dann muss **mindestens eine** verletzt sein.*

Beweis durch Wertetafel.

a	b	$a \wedge b$	$\overline{a \wedge b}$	$\overline{a}$	$\overline{b}$	$\overline{a} \vee \overline{b}$
0	0	0	**1**	1	1	**1**
0	1	0	**1**	1	0	**1**
1	0	0	**1**	0	1	**1**
1	1	1	**0**	0	0	**0**

Spalte 4 und Spalte 7 stimmen überein. ✓

Die praktische Merkregel.

Beim Hineinziehen eines Negationsstrichs kippt jede Verknüpfung um: UND wird ODER, ODER wird UND – und jede Variable wird negiert.

Beispiel aus der Programmierpraxis:

$$\neg(x > 0 \ \wedge \ x < 10) \iff x \leq 0 \ \vee \ x \geq 10.$$

Man beachte, dass auch die Vergleiche selbst umgekehrt werden – **das ist der häufigste Fehler beim Negieren von Bedingungen.**

Die technische Bedeutung ist noch größer: *De Morgan erlaubt es, jede Schaltung allein aus NAND-Gattern zu bauen*, **oder allein aus NOR-Gattern** *– was Kapitel 6 ausführt und der eigentliche Grund dafür ist, warum ein einziges Bauteil genügt.*

Augustus De Morgan

Augustus De Morgan (1806–1871) veröffentlichte seine Gesetze im selben Jahr wie Boole seine Schrift – *1847, offenbar unabhängig, und die beiden standen in freundschaftlichem Briefwechsel.*

De Morgan war auf einem Auge blind, verweigerte aus Gewissensgründen jede kirchliche Bindung und lehnte deshalb eine Ehrendoktorwürde ab.

Von ihm stammt auch der Begriff vollständige Induktion (Mathematik-Band 0, Kapitel 14) **in seiner heutigen Form.**

Und eine hübsche Marginalie: De Morgan bemerkte, dass er im Jahr x alt war, wobei x^2 das Jahr war – **er war 43 im Jahr 1849, und** $43^2 = 1849$**.** ✓

Weitere Verknüpfungen

Bei zwei Eingängen gibt es $2^4 = 16$ mögliche Funktionen – denn die Wertetafel hat 4 Zeilen, und jede kann 0 oder 1 liefern.

Name	Zeichen	Ausgabe 1 bei	Bemerkung
UND	$a \wedge b$	11	
ODER	$a \vee b$	01, 10, 11	
XOR	$a \oplus b$	01, 10	**entweder oder**
NAND	$a \barwedge b$	00, 01, 10	**universell**
NOR	$a \barvee b$	00	**universell**
Äquivalenz	$a \leftrightarrow b$	00, 11	$= \overline{a \oplus b}$
Implikation	$a \rightarrow b$	00, 01, 11	$= \overline{a} \vee b$

Das XOR verdient besondere Beachtung.

$$a \oplus b = (a \vee b) \wedge \overline{(a \wedge b)} = a\overline{b} \vee \overline{a}b.$$

Ungleichheitstest	$a \oplus b = 1 \iff a \neq b$
Addition ohne Übertrag	Die Summenstelle des Halbaddierers (Kap. 7)
Selbstinvers	$(a \oplus b) \oplus b = a$
Parität	XOR über alle Bits = Parität

Die dritte Zeile ist die folgenreichste.
Beweis: $b \oplus b = 0$ und $a \oplus 0 = a$, also nach dem Assoziativgesetz

$$(a \oplus b) \oplus b = a \oplus (b \oplus b) = a \oplus 0 = a. \qquad \checkmark$$

Verschlüsseln und Entschlüsseln sind dieselbe Operation. Das ist die Grundlage des **One-Time-Pad** – des einzigen beweisbar unbrechbaren Verschlüsselungsverfahrens (Band 8).

Und dieselbe Eigenschaft liefert einen berühmten Trick: *Man kann zwei Variablen **ohne Hilfsvariable** vertauschen.*

$$a \oplus\!= b; \qquad b \oplus\!= a; \qquad a \oplus\!= b;$$

Schritt	Inhalt von a	Inhalt von b
Start	x	y
$a \oplus= b$	$x \oplus y$	y
$b \oplus= a$	$x \oplus y$	$y \oplus x \oplus y = x$
$a \oplus= b$	$x \oplus y \oplus x = y$	x

Vertauscht. ✓

Der Trick ist heute schädlich

Er versagt, wenn a und b *dieselbe* Speicherstelle sind – *dann löscht der erste Schritt den Wert auf null.*

*Außerdem erzeugt er drei voneinander abhängige Operationen, die ein moderner Prozessor **nicht** parallel ausführen kann* – **während der gewöhnliche Tausch mit Hilfsvariable schneller ist.**

Ein Musterbeispiel für einen Trick, der 1970 klug war und heute nur noch schadet: *Damals war Speicher knapp, heute sind Register billig und Abhängigkeiten teuer.*

Aufgabe 5.1.

(a) Vereinfachen Sie Boolesche Ausdrücke und weisen Sie die Vereinfachung nach.

(b) Zeigen Sie, dass NAND allein genügt, um alle Verknüpfungen darzustellen.

(c) Wie viele Boolesche Funktionen von n Variablen gibt es? Und wie viele davon sind „einfach“?

Lösung Schritt für Schritt.

(a) Vereinfachen.
Aufgabe 1: $f = a\bar{b} \vee ab$.
Schritt 1 – Distributivgesetz rückwärts (ausklammern).

$$a\bar{b} \vee ab = a \wedge (\bar{b} \vee b).$$

Schritt 2 – Komplementgesetz.

$$\bar{b} \vee b = 1.$$

Schritt 3 – Neutrales Element.

$$a \wedge 1 = a.$$

$$\boxed{f = a}$$

Anschaulich: Wenn a wahr ist, ist der Ausdruck wahr – gleichgültig, was b tut. ✓

Probe durch Wertetafel:

a	b	$a\overline{b}$	ab	f	a
0	0	0	0	**0**	**0**
0	1	0	0	**0**	**0**
1	0	1	0	**1**	**1**
1	1	0	1	**1**	**1**

✓

Aufgabe 2: $g = \overline{\overline{a} \vee \overline{b}} \vee \overline{\overline{a} \vee b}$.
Schritt 1 – De Morgan auf beide Klammern.

$$\overline{\overline{a} \vee \overline{b}} = \overline{\overline{a}} \wedge \overline{\overline{b}} = a \wedge b.$$

$$\overline{\overline{a} \vee b} = \overline{\overline{a}} \wedge \overline{b} = a \wedge \overline{b}.$$

Schritt 2 – Einsetzen.

$$g = ab \vee a\overline{b}.$$

Schritt 3 – Wie in Aufgabe 1.

$$\boxed{g = a}$$

Ein Ausdruck mit fünf Negationen und drei Verknüpfungen entpuppt sich als das einfache a. **So etwas passiert bei automatisch erzeugten Schaltungen ständig** *– und ist der Grund, warum Kapitel 6 systematische Minimierungsverfahren behandelt.*

Aufgabe 3 – Das Konsensustheorem: $h = ab \vee \overline{a}c \vee bc$.
Behauptung: *Der letzte Term ist überflüssig.*
Schritt 1 – Den Term aufblähen.

$$bc = bc \wedge 1 = bc \wedge (a \vee \overline{a}) = abc \vee \overline{a}bc.$$

Schritt 2 – Einsetzen.

$$h = ab \vee \overline{a}c \vee abc \vee \overline{a}bc.$$

Schritt 3 – Absorption zweimal anwenden.

$$ab \vee abc = ab \wedge (1 \vee c) = ab.$$

$$\overline{a}c \vee \overline{a}bc = \overline{a}c \wedge (1 \vee b) = \overline{a}c.$$

$$\boxed{h = ab \vee \overline{a}c}$$

Die anschauliche Begründung: *Ist bc wahr, so ist $b = c = 1$. Ist dann $a = 1$, greift ab; ist $a = 0$, greift $\bar{a}c$.* **In beiden Fällen ist der Ausdruck ohnehin wahr** – *der Term bc trägt nichts bei.*

Und trotzdem baut man ihn manchmal ein

Der Konsensusterm ist logisch überflüssig, aber technisch wertvoll.
In einer realen Schaltung brauchen Signale Zeit. Wechselt a von 1 auf 0 bei $b = c = 1$, so kann der Term ab bereits abgefallen sein, während $\bar{a}c$ noch nicht angestiegen ist **– für einen Augenblick liegt am Ausgang eine 0, obwohl das Ergebnis konstant 1 sein müsste.**

*Man nennt das einen **Hazard** oder Störimpuls.*

Der Konsensusterm *bc* **überbrückt diese Lücke**, *weil er während des gesamten Übergangs stabil 1 liefert.* **Man fügt ihn also *absichtlich* wieder ein.**

*Ein schönes Beispiel dafür, dass **logische Gleichwertigkeit nicht technische Gleichwertigkeit** bedeutet* – die Boolesche Algebra kennt keine Zeit, die Schaltung schon.

(b) NAND genügt.

Definition 5.3. Eine Menge von Verknüpfungen heißt **funktional vollständig**, wenn sich jede Boolesche Funktion allein durch sie ausdrücken lässt.

Schritt 1 – Was NAND ist.

$$a \barwedge b := \overline{a \wedge b}.$$

Schritt 2 – NICHT aus NAND.

$$a \barwedge a = \overline{a \wedge a} = \bar{a}.$$

Wegen der Idempotenz $a \wedge a = a$. ✓

Schritt 3 – UND aus NAND.

$$(a \barwedge b) \barwedge (a \barwedge b) = \overline{\overline{a \wedge b}} = a \wedge b.$$

Ein NAND, dann ein NICHT – und das NICHT ist selbst ein NAND. ✓

Schritt 4 – ODER aus NAND.
Nach De Morgan gilt

$$a \vee b = \overline{\bar{a} \wedge \bar{b}} = \bar{a} \barwedge \bar{b}.$$

Und $\bar{a} = a \barwedge a$:

$$a \vee b = (a \barwedge a) \barwedge (b \barwedge b).$$

Schritt 5 – Die Bilanz.

Funktion	aus NAND	Gatter
NICHT	$a \barwedge a$	1
UND	$(a \barwedge b) \barwedge (a \barwedge b)$	2
ODER	$(a \barwedge a) \barwedge (b \barwedge b)$	3
XOR	(siehe Kapitel 7)	4

Schritt 6 – Warum das die zentrale Aussage des Bandes ist.

*Da sich **jede** Boolesche Funktion durch UND, ODER und NICHT darstellen lässt* (Kapitel 6) *und diese drei sich durch NAND darstellen lassen:*

Aus NAND-Gattern allein lässt sich **jede** Boolesche Funktion bauen.

Dasselbe gilt für NOR – **und für sonst keine einzelne der sechzehn zweistelligen Funktionen.**

Der praktische Wert ist gewaltig: *Eine Chipfabrik muss nur **einen** Gattertyp beherrschen und optimieren.* **In CMOS-Technik braucht ein NAND vier Transistoren, ein UND dagegen sechs** – *weil UND intern als NAND mit nachgeschaltetem Inverter gebaut wird.*

> ***Das negierte Gatter ist das billigere.*** Deshalb sind reale Schaltungen voll von NAND und NOR, während UND und ODER die teureren Sonderfälle sind – genau umgekehrt zur Anschauung.

Der Nachweis, dass NAND allein genügt, stammt von Henry Sheffer *(1913); die Verknüpfung heißt deshalb auch* ***Sheffer-Strich****.* Charles Sanders Peirce *hatte dasselbe bereits 1880 gefunden, aber nicht veröffentlicht.*

(c) Wie viele Boolesche Funktionen gibt es?
Schritt 1 – Abzählen.
Eine Funktion von n Variablen ist durch ihre Wertetafel festgelegt. Die Tafel hat 2^n *Zeilen, und in jeder steht 0 oder 1.*

$$\text{Anzahl} = 2^{2^n}$$

Nach der Produktregel (Mathematik-Band 15, Kapitel 1).

n	2^n Zeilen	**Funktionen**	**Vergleich**
1	2	4	
2	4	16	
3	8	256	
4	16	65 536	
5	32	$4{,}3 \cdot 10^9$	
6	64	$1{,}8 \cdot 10^{19}$	Sekunden seit Urknall
8	256	10^{77}	**Atome im Universum:** 10^{80}

Die Zahl wächst doppelt exponentiell. Schon bei acht Eingängen gibt es fast so viele Funktionen wie Atome im beobachtbaren Universum.

Schritt 2 – Die entscheidende Folgerung.
Eine Schaltung mit g Gattern lässt sich durch eine Beschreibung von etwa $c \cdot g \log g$ Bit festlegen. Es gibt also höchstens etwa $2^{c\,g \log g}$ verschiedene Schaltungen mit g Gattern. Damit alle 2^{2^n} Funktionen darstellbar sind, muss gelten

$$2^{c\,g \log g} \geq 2^{2^n} \Longrightarrow g \gtrsim \frac{2^n}{n}.$$

Satz 5.4 (Shannon, 1949)**.** ***Fast jede*** *Boolesche Funktion von n Variablen benötigt mindestens etwa $\frac{2^n}{n}$ Gatter.*

Es gibt also Funktionen, deren Schaltungen exponentiell groß sein müssen – und zwar nicht ein paar Ausnahmen, sondern **fast alle**.

Und hier wird es merkwürdig.

Ein Beweis, der nichts zeigt

Shannons Argument ist ein Abzählargument (Mathematik-Band 15, Kapitel 14): *Es gibt mehr Funktionen als kleine Schaltungen, also müssen manche Funktionen große Schaltungen brauchen.*

> ***Aber es nennt keine einzige solche Funktion.*** Man weiß, dass fast alle Funktionen schwer sind – und kennt bis heute **keine einzige explizite Funktion**, für die eine superlineare untere Schranke bewiesen wäre.

Stand August 2026 ist die beste bekannte untere Schranke für eine explizit angebbare Funktion in n Variablen von der Größenordnung $3{,}1\,n$ – **gegenüber der bewiesenen Existenz von Funktionen der Größenordnung $\frac{2^n}{n}$.**

> *Die Lücke zwischen „fast alle sind schwer“ und „diese hier ist schwer“ ist astronomisch* – **und sie ist genau der Grund, warum $\mathrm{P} \neq \mathrm{NP}$ so schwer zu beweisen ist** (Band 3).

> Paul Erdős *nannte solche Situationen das Paradox der probabilistischen Methode: Der Zufall findet Objekte, die wir nicht konstruieren können* (Mathematik-Band 15, Kapitel 14). **In der Komplexitätstheorie ist das kein Kuriosum, sondern die zentrale Schwierigkeit.**

Petzold[21] und Hillis[10] führen die Boolesche Algebra anschaulich ein; Kohavi und Jha[16] ist die klassische Referenz; Sipser[23] stellt den Bezug zur Komplexitätstheorie her.

6 Normalformen und Minimierung

Von der Tabelle zur Formel

> *Bisher gingen wir von einer Formel aus und rechneten die Tabelle aus.* **Jetzt drehen wir es um:** *Gegeben ist, was die Schaltung tun soll – gesucht ist eine Formel dafür.*

Das ist die eigentliche Aufgabe des Schaltungsentwurfs, *und sie ist erstaunlich mechanisch lösbar.*

Satz 6.1 (Existenz der Normalform)**.** *Zu **jeder** Booleschen Funktion gibt es einen Ausdruck, der sie darstellt – und zwar in einer festen, aus der Wertetafel ablesbaren Form.*

Das Verfahren an einem Beispiel.
Gesucht sei die Funktion „genau zwei der drei Eingänge sind 1“:

a	b	c	f	**Minterm**
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	**1**	$\overline{a}\,b\,c$
1	0	0	0	
1	0	1	**1**	$a\,\overline{b}\,c$
1	1	0	**1**	$a\,b\,\overline{c}$
1	1	1	0	

Die Idee in einem Satz:

> ***Für jede Zeile mit Ausgabe 1 baut man einen Term, der genau in dieser Zeile wahr ist.*** *Dann verknüpft man alle diese Terme mit ODER.*

Definition 6.2. Ein **Minterm** ist ein UND über **alle** Variablen, jede einmal, negiert oder nicht.
Die **disjunktive Normalform** (DNF) ist das ODER aller Minterme zu den Zeilen mit Ausgabe 1.

Für unser Beispiel:

$$f = \bar{a}bc \vee a\bar{b}c \vee ab\bar{c}$$

Warum das stimmt.
Ein Minterm wird genau dann 1, wenn jede seiner Variablen den passenden Wert hat – **er ist also „wahr in genau einer Zeile".** *Das ODER ist wahr, wenn mindestens ein Minterm wahr ist –* **also genau in den ausgewählten Zeilen.** ✓

> ***Damit ist die zentrale Behauptung des Vorworts zur Hälfte bewiesen:*** Jede Boolesche Funktion lässt sich durch UND, ODER und NICHT darstellen – **und diese drei lassen sich aus NAND bauen** (Kapitel 5).

Aus NAND-Gattern lässt sich **jede** Boolesche Funktion bauen.

Die zweite Normalform

Man kann auch von den Nullen ausgehen.

Definition 6.3. Ein **Maxterm** ist ein ODER über alle Variablen.
Die **konjunktive Normalform** (KNF) ist das UND aller Maxterme zu den Zeilen mit Ausgabe **0**, wobei jede Variable **umgekehrt** eingesetzt wird.

Für die Zeile $a = 0, b = 0, c = 0$ mit $f = 0$ lautet der Maxterm $(a \vee b \vee c)$ – **er ist genau in dieser Zeile falsch.** *Das UND aller solchen Terme ist genau dort falsch, wo f falsch ist.* ✓

	DNF	**KNF**
gebaut aus	Zeilen mit **1**	Zeilen mit **0**
Aufbau	ODER von UND	UND von ODER
Term wahr wenn	eine Zeile passt	keine Zeile verletzt
gut, wenn	wenige Einsen	wenige Nullen

> *Man wählt in der Praxis die kürzere: Hat die Tafel drei Einsen und fünf Nullen, nimmt man die DNF.* **Beide beschreiben dieselbe Funktion.**

Die KNF ist zugleich die Sprache des SAT-Problems (Kapitel 5): *„Gibt es eine Belegung, die alle Klauseln erfüllt?"* **Jeder SAT-Löser erwartet seine Eingabe in KNF.**

6.1 Minimierung

Warum die Normalform nicht das Ende ist

Die DNF ist immer richtig, aber fast nie klein.

Beispiel: Die Funktion „mindestens zwei von drei“ hat vier Einsen, also vier Minterme:

$$f = \bar{a}bc \vee a\bar{b}c \vee ab\bar{c} \vee abc.$$

Das sind 4 UND-Gatter zu je 3 Eingängen und ein ODER-Gatter zu 4 Eingängen – **plus drei Inverter.**

Es geht viel kürzer:

$$f = ab \vee ac \vee bc.$$

Nachweis durch Ausklammern.
Wir schreiben den letzten Minterm dreimal hin – erlaubt wegen der Idempotenz $x \vee x = x$:

$$f = \bar{a}bc \vee a\bar{b}c \vee ab\bar{c} \vee \underbrace{abc \vee abc \vee abc}_{=abc}.$$

Nun paarweise zusammenfassen:

$$ab\bar{c} \vee abc = ab(\bar{c} \vee c) = ab$$
$$a\bar{b}c \vee abc = ac(\bar{b} \vee b) = ac$$
$$\bar{a}bc \vee abc = bc(\bar{a} \vee a) = bc$$

$$\boxed{f = ab \vee ac \vee bc} \qquad \checkmark$$

Der Kern jeder Minimierung steckt in dieser einen Regel:

$$xy \vee x\bar{y} = x.$$

*Zwei Terme, die sich in **genau einer** Variablen unterscheiden, lassen sich zusammenfassen – diese Variable fällt weg.*

Die Ersparnis in Zahlen:

	DNF	**minimal**	**Ersparnis**
UND-Gatter	4	3	
Eingänge gesamt	12	6	**50 %**
Inverter	3	**0**	**100 %**

Das KV-Diagramm

Das Zusammenfassen von Hand wird schnell unübersichtlich. MAURICE KARNAUGH *fand 1953 (aufbauend auf* EDWARD VEITCH*, 1952) eine Darstellung, in der man die zusammenfassbaren Terme* ***sieht****.*

Die Idee: Man ordnet die Wertetafel so an, dass benachbarte Felder sich in genau *einem* Bit unterscheiden.

$ab \backslash cd$	$cd = 00$	$cd = 01$	$cd = 11$	$cd = 10$
~~$ab = 00$~~	0	1	1	0
$ab = 01$	0	1	1	0

ein Block = ein Term

Die Spaltenreihenfolge ist $00, 01, 11, 10$ **– nicht** $00, 01, 10, 11$**.**

Das ist der ***Gray-Code****: Von einem Eintrag zum nächsten ändert sich* ***genau ein*** *Bit.* **Deshalb sind benachbarte Felder immer zusammenfassbar.**

binär	Gray	geänderte Stelle
000	000	—
001	001	letzte
010	011	mittlere
011	010	letzte
100	110	erste
101	111	letzte
110	101	mittlere
111	100	letzte

Die Regeln für das Ablesen:

1. Blöcke aus Einsen bilden, so **groß** wie möglich.
2. Die Größe muss eine **Zweierpotenz** sein: 1, 2, 4, 8.
3. Blöcke dürfen sich **überlappen**.
4. Das Diagramm ist **umlaufend** – links und rechts, oben und unten sind benachbart.
5. Jeder Block liefert einen Term aus den Variablen, die im Block **konstant** sind.

Regel 4 ist die überraschendste: Das Diagramm ist eigentlich ein ***Torus*** (Mathematik-Band 8) – **man klebt links an rechts und oben an unten.**

Die Grenze des Verfahrens

Über sechs Variablen hinaus ist das KV-Diagramm nicht mehr zeichenbar.

Für größere Fälle gibt es das Verfahren von QUINE *und* MCCLUSKEY *(1956), das denselben Gedanken tabellarisch und damit programmierbar macht.*

Aber auch das hilft nur begrenzt: *Die Minimierung Boolescher Ausdrücke ist ein NP-schweres Problem* (genauer: Σ_2^p-vollständig). **Reale Werkzeuge benutzen daher Heuristiken wie `Espresso` (Berkeley, 1984), die gute, aber nicht garantiert optimale Lösungen liefern.**

Das ist eine wiederkehrende Erfahrung der Informatik: ***Das Optimum ist unerreichbar, das Gute genügt.*** Ein moderner Chip enthält Milliarden Gatter – niemand kann prüfen, ob es ein paar Prozent weniger täten.

Der Gray-Code hat eigene Verdienste

Er wurde nicht für KV-Diagramme erfunden, sondern für ein praktisches Problem.

Das Problem: Winkelgeber. *Eine Scheibe trägt ein Bitmuster, das die Drehstellung angibt. Beim Übergang von* $(0111)_2$ *auf* $(1000)_2$ *müssen* ***vier*** *Bit gleichzeitig kippen.*

Gleichzeitig gibt es aber nicht – die Sensoren lesen minimal versetzt. **Für einen Augenblick kann jedes der 16 Muster erscheinen,** *also auch* $(1111)_2 = 15$ *statt der richtigen 7 oder 8.*

Im Gray-Code ändert sich immer nur *ein* Bit – *ein Zwischenzustand ist entweder der alte oder der neue Wert, nie ein völlig falscher.*

FRANK GRAY *von den Bell Labs meldete das 1947 zum Patent an, für die Pulscodemodulation im Fernsprechnetz.* **Der Code selbst ist älter** – ÉMILE BAUDOT *benutzte ihn 1878 im Telegrafen, und* LOUIS GROS *beschrieb ihn 1872 in einer Analyse des Geduldsspiels „Baguenaudier“.*

Die Umrechnung ist bemerkenswert einfach:

$$\text{Gray} = \text{Binär} \oplus (\text{Binär} \gg 1).$$

Probe für $6 = (110)_2$*:*

$$(110)_2 \oplus (011)_2 = (101)_2 = \text{Gray-Code von 6.} \qquad \checkmark$$

Und stimmt mit der Tabelle oben überein. ✓

Aufgabe 6.1.

(a) Entwerfen Sie einen Siebensegment-Dekoder für die Ziffer und minimieren Sie.

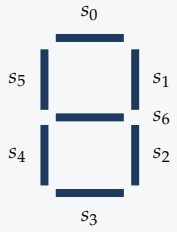
(b) Minimieren Sie eine Funktion mit vier Variablen im KV-Diagramm, mit und ohne Umlauf.

(c) Untersuchen Sie „don't care"-Belegungen und ihren Nutzen.

Lösung Schritt für Schritt.

(a) Der Siebensegment-Dekoder.

Aufgabe: Aus einer 4-Bit-Zahl $(abcd)_2$ (Werte 0–9) sollen die sieben Segmente einer Ziffernanzeige angesteuert werden.



Schritt 1 – Die Wertetafel für Segment s_0 (oben).
Bei welchen Ziffern leuchtet der obere Balken?

Ziffer	a	b	c	d	s_0	Ziffer	a	b	c	d	s_0
0	0	0	0	0	**1**	5	0	1	0	1	**1**
1	0	0	0	1	0	6	0	1	1	0	**1**
2	0	0	1	0	**1**	7	0	1	1	1	**1**
3	0	0	1	1	**1**	8	1	0	0	0	**1**
4	0	1	0	0	0	9	1	0	0	1	**1**

Nur bei 1 und 4 bleibt der obere Balken dunkel. ✓

Schritt 2 – Die DNF wäre lang.
Acht Einsen ergeben acht Minterme zu je vier Variablen – **32 Gattereingänge.**
Schritt 3 – Über die Nullen gehen.

*Es gibt nur **zwei** Nullen. Also lohnt sich die KNF – oder, gleichwertig, man beschreibt das Komplement:*

$$\overline{s_0} = \overline{a}\,\overline{b}\,\overline{c}\,d \;\vee\; \overline{a}\,b\,\overline{c}\,\overline{d}.$$

(Ziffer 1 ist $(0001)_2$*, Ziffer 4 ist* $(0100)_2$*.)*
Schritt 4 – Zusammenfassen.
Die beiden Terme unterscheiden sich in ***zwei*** *Variablen (b und d) – sie lassen sich nicht direkt zusammenfassen.* **Aber man kann ausklammern:**

$$\overline{s_0} = \overline{a}\,\overline{c}\left(\overline{b}d \vee b\overline{d}\right) = \overline{a}\,\overline{c}\,(b \oplus d).$$

Schritt 5 – Negieren.

$$\boxed{s_0 = \overline{\overline{a}\,\overline{c}\,(b \oplus d)} = a \vee c \vee \overline{b \oplus d}}$$

Nach De Morgan. ✓
Schritt 6 – Die Bilanz.

Fassung	Gattereingänge	Gatter
DNF	32	9
über das Komplement	8	3

> ***Ein Faktor vier****, gewonnen allein dadurch, dass man von der kürzeren Seite der Tabelle ausgeht.* **Und das XOR fiel als natürliche Struktur heraus, nicht als Trick.**

(b) Ein KV-Diagramm mit vier Variablen.
Gegeben sei f *mit Einsen bei den Mintermnummern* $0, 2, 5, 7, 8, 10, 13, 15$*.*
Schritt 1 – Das Diagramm füllen.

ab \ *cd*	00	01	11	10
00	**1**	0	0	**1**
01	0	**1**	**1**	0
11	0	**1**	**1**	0
10	**1**	0	0	**1**

Schritt 2 – Den großen Block ablesen (rot).
Die mittleren beiden Spalten ($cd = 01$ *und* 11*) und die mittleren beiden Zeilen (*$ab = 01$ *und* 11*).*

Variable	im Block	Beitrag
a	0 und 1	wechselt – fällt weg
b	**immer 1**	liefert b
c	0 und 1	wechselt – fällt weg
d	**immer 1**	liefert d

$$\text{Term} = b\,d.$$

Schritt 3 – Die vier Eckfelder (grün).

Die vier Ecken sind im Torusmodell ***ein einziger Block der Größe 4*** – oben klebt an unten, links an rechts. **Das ist Regel 4.**

Variable	Werte in den Ecken	Beitrag
a	$0, 0, 1, 1$	fällt weg
b	**immer 0**	liefert $\overline{b}$
c	$0, 1, 0, 1$	fällt weg
d	**immer 0**	liefert $\overline{d}$

$$\text{Term} = \overline{b}\,\overline{d}.$$

Schritt 4 – Ergebnis.

$$\boxed{f = bd \vee \overline{b}\,\overline{d}}$$

Schritt 5 – Erkennen, was das ist.

$$bd \vee \overline{bd} = \overline{b \oplus d} = (b \leftrightarrow d).$$

Die Funktion prüft schlicht, ob b ***und*** d ***gleich sind*** – und hängt von a und c überhaupt nicht ab. **Das sah man den acht Mintermen nicht an.**

Schritt 6 – Was der Umlauf gebracht hat.

	ohne Umlauf	mit Umlauf
Blöcke	4 (je Größe 2)	**2** (je Größe 4)
Terme	4	**2**
Gattereingänge	12	**6**

Wer die Umlaufregel nicht kennt, erhält eine doppelt so große Schaltung – **sie ist nicht richtiger und nicht falscher, nur teurer.**

(c) Don't care.

Definition 6.4. Eine **don't care**-Belegung ist eine Eingangskombination, die **nie vorkommt** oder deren Ausgabe **gleichgültig** ist. Man schreibt $\times$ statt 0 oder 1.

Schritt 1 – Wo sie herkommen.
Beim Siebensegment-Dekoder aus Teil (a) sind die Eingaben $(1010)_2$ *bis* $(1111)_2$ *– also 10 bis 15 – keine Ziffern.*

Was die Anzeige dort tut, ist gleichgültig. **Sechs der sechzehn Zeilen sind frei.**

Schritt 2 – Der Gewinn.

Beim Bilden der Blöcke darf man ein $\times$ als ***1*** *zählen, wenn es den Block vergrößert* – **und als 0, wenn nicht.** *Man wählt für jedes $\times$ frei, was besser passt.*

Schritt 3 – Am Beispiel.
Für Segment s_0 hatten wir

$$\overline{s_0} = \overline{a}\,\overline{c}\,(b \oplus d).$$

Setzt man die sechs unbenutzten Belegungen alle auf $s_0 = 1$ (also $\overline{s_0} = 0$), so ändert sich nichts. Setzt man sie geschickt, so wird es kürzer:
Die Nullen von s_0 liegen bei $(0001)_2$ und $(0100)_2$. Erlaubt man zusätzlich $(1001)_2$ und $(1100)_2$ als Nullen (don't care), so lassen sich Paare bilden:

$$\overline{s_0} = \overline{b}\,\overline{c}\,d \vee b\,\overline{c}\,\overline{d}.$$

Die Variable a ist verschwunden – der Term wurde um ein Viertel kürzer. ✓

Schritt 4 – Die Ersparnis allgemein.

Anteil don't care	**typ. Ersparnis**	**Beispiel**
0 %	—	vollständige Funktion
$\approx$ 20 %	10–20 %	
$\approx$ 40 %	30–50 %	**BCD-Dekoder**

Don’t care heißt nicht „egal“

Der Ausgang ist nach der Minimierung *nicht* undefiniert – *er hat einen ganz bestimmten Wert, nur hat ihn niemand vorgeschrieben.*

> *Beim Siebensegment-Dekoder zeigt die Anzeige bei Eingabe 12 irgendein Muster.* **Welches, hängt davon ab, wie das Minimierungswerkzeug die Blöcke gelegt hat** – *und kann sich beim nächsten Übersetzungslauf ändern.*

Das wird gefährlich, wenn die Annahme „kommt nie vor“ falsch ist.

> *Genau das ist das Muster hinter dem* ***undefinierten Verhalten*** *in C* (Kapitel 2): **Der Übersetzer darf annehmen, dass gewisse Fälle nicht eintreten, und optimiert entsprechend.** *Treten sie doch ein, kann alles passieren.*

> *In sicherheitskritischer Hardware verzichtet man deshalb oft auf don’t care und legt jeden Fall ausdrücklich fest* – **man zahlt ein paar Prozent Fläche für die Gewissheit, dass nichts Unerwartetes geschieht.**

Kohavi und Jha[16] behandeln Normalformen und Minimierung erschöpfend; Nisan und Schocken[18] lassen den Leser die Schaltungen selbst bauen; Tanenbaum[24] ordnet sie in die Rechnerarchitektur ein.

7 Gatter und Schaltnetze

Vom Ausdruck zur Schaltung

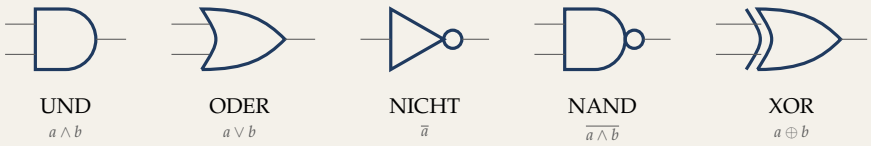
> *Bisher war die Boolesche Algebra Mathematik. Jetzt wird sie Blech:* ***Jeder Boolesche Ausdruck lässt sich als Schaltung bauen, und jede Schaltung berechnet einen Booleschen Ausdruck.***

Das ist Shannons Entdeckung von 1937 (Vorwort), *und sie ist die Brücke zwischen den Kapiteln 5 und 10.*

Definition 7.1. Ein **Gatter** ist ein Bauteil, das aus einem oder mehreren Eingangssignalen ein Ausgangssignal berechnet.
Ein **Schaltnetz** (*combinational circuit*) ist eine kreisfreie Zusammenschaltung von Gattern. Sein Ausgang hängt **nur** von den momentanen Eingängen ab – es hat **kein Gedächtnis**.

> *Das Wort „kreisfrei" ist wesentlich.* **Sobald ein Ausgang auf einen Eingang zurückgeführt wird, entsteht etwas grundsätzlich anderes** – *nämlich Gedächtnis, und davon handelt Kapitel 8.*

Die Symbole.



a	b	UND	ODER	NAND	NOR	XOR
0	0	0	0	**1**	**1**	0
0	1	0	1	**1**	0	**1**
1	0	0	1	**1**	0	**1**
1	1	**1**	**1**	0	0	0

7.1 Ein Gatter genügt

Satz 7.2 (Funktionale Vollständigkeit von NAND)**.** *Jede Boolesche Funktion lässt sich allein mit NAND-Gattern aufbauen.*

Der Beweis in drei Zeilen

Wir zeigen, dass sich NICHT, UND und ODER aus NAND bauen lassen. Da diese drei nach Kapitel 6 bereits jede Funktion erzeugen, ist damit alles gezeigt.

Zeile 1 – NICHT.

$$a \barwedge a = \overline{a \wedge a} = \overline{a}.$$

Man legt beide Eingänge zusammen. ✓
Zeile 2 – UND.

$$\overline{(a \barwedge b)} = \overline{\overline{a \wedge b}} = a \wedge b.$$

Also: ein NAND, dann ein weiteres als Inverter – **zwei Gatter.** ✓
Zeile 3 – ODER.
Nach de Morgan (Kapitel 5):

$$a \vee b = \overline{\overline{a} \wedge \overline{b}} = \overline{a} \barwedge \overline{b} = (a \barwedge a) \barwedge (b \barwedge b).$$

Drei Gatter. ✓

***Damit ist die zentrale Behauptung des Vorworts bewiesen**: Aus einem einzigen Bauteiltyp lässt sich jede Boolesche Funktion aufbauen – und damit, wie die folgenden Kapitel zeigen, ein vollständiger Rechner.*

Funktion	NAND-Gatter	Bemerkung
NICHT	1	Eingänge verbinden
UND	2	
ODER	3	
XOR	4	
NOR	4	

Auch NOR ist allein vollständig – *man rechnet es genauso nach.* **Und das sind die *einzigen* beiden zweistelligen Funktionen mit dieser Eigenschaft.**

HENRY SHEFFER *bewies das 1913, weshalb NAND auch* ***Sheffer-Strich*** *heißt* – **allerdings hatte CHARLES SANDERS PEIRCE dasselbe schon 1880 gefunden und nicht veröffentlicht.**

Warum die Industrie das ernst nimmt

Real werden Schaltungen tatsächlich überwiegend aus NAND und NOR gebaut – *nicht aus Sparsamkeit im Entwurf, sondern aus physikalischen Gründen.*

> *In CMOS-Technik ist ein NAND-Gatter aus **vier** Transistoren aufgebaut, ein UND-Gatter dagegen aus **sechs** –* denn ein UND ist ein NAND plus Inverter. **Das invertierende Gatter ist das billigere.**

Der Grund liegt in der Bauart der Transistoren: Ein CMOS-Gatter besteht aus einem Netzwerk, das den Ausgang nach oben zieht, und einem, das ihn nach unten zieht. **Diese Anordnung invertiert von Natur aus.** *Eine nichtinvertierende Funktion kostet eine zusätzliche Stufe.*

Ein hübsches Beispiel dafür, dass die „natürlichen" Grundoperationen der Physik nicht die des menschlichen Denkens sind.

7.2 Der Addierer

Der Halbaddierer

Wir bauen jetzt die Addition – aus nichts als Gattern.

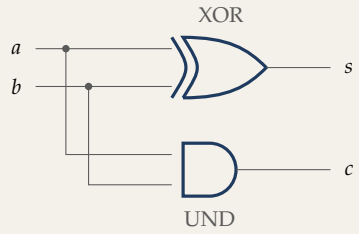
Aufgabe: *Zwei Bits a und b addieren.* **Das Ergebnis kann 0, 1 oder 2 sein und braucht daher *zwei* Ausgangsbits:** *die **Summe** s und den **Übertrag** c (von* carry*).*

a	b	c	s	**Bedeutung**
0	0	0	0	$0+0=0$
0	1	0	1	$0+1=1$
1	0	0	1	$1+0=1$
1	1	**1**	0	$1+1=(10)_2$

Die Spalten ablesen.
Die Spalte s ist genau die XOR-Tabelle. Die Spalte c ist genau die UND-Tabelle.

$$\boxed{s = a \oplus b, \qquad c = a \wedge b}$$

> ***Das ist der ganze Halbaddierer: zwei Gatter.*** Die Addition, die uns als Grundrechenart erscheint, ist aus der Sicht der Maschine eine Nebenerscheinung der Logik.



Der Volladdierer

Für mehrstellige Zahlen genügt der Halbaddierer nicht: Ab der zweiten Stelle kommt ein ***Übertrag von rechts*** *hinzu.*

Drei Eingänge a, b, c_{in}; zwei Ausgänge s, c_{out}.

a	b	c_{in}	c_{out}	s	**Summe**
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	0	1	1
0	1	1	1	0	2
1	0	0	0	1	1
1	0	1	1	0	2
1	1	0	1	0	2
1	1	1	1	1	3

Die Summe ablesen.
s ist genau dann 1, wenn eine ***ungerade*** *Anzahl der Eingänge 1 ist* – **das ist die Verallgemeinerung von XOR:**

$$s = a \oplus b \oplus c_{\text{in}}.$$

Den Übertrag ablesen.
c_{out} ist genau dann 1, wenn ***mindestens zwei*** *Eingänge 1 sind* – **die Mehrheitsfunktion:**

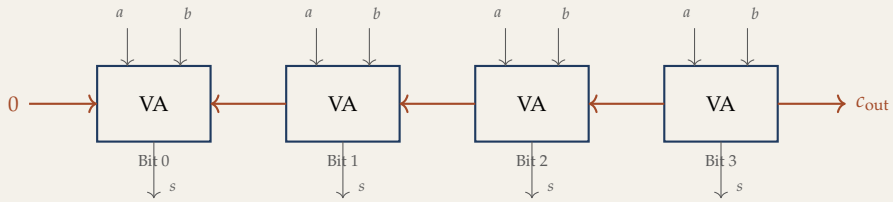
$$c_{\text{out}} = (a \wedge b) \vee (a \wedge c_{\text{in}}) \vee (b \wedge c_{\text{in}}).$$

> *Man kann den Volladdierer auch aus* ***zwei Halbaddierern*** *und einem ODER bauen:* **Zuerst $a + b$, dann das Ergebnis $+ c_{\text{in}}$;** *die beiden Überträge werden verodert.* **Daher der Name „Halb"addierer.**

Nachweis, dass das dasselbe liefert: Zwei Überträge können nie gleichzeitig auftreten – **denn wenn der erste Halbaddierer einen Übertrag erzeugt, ist seine Summe 0, und der zweite kann dann keinen erzeugen.** ✓

Der Ripple-Carry-Addierer

Für n-Bit-Zahlen schaltet man n Volladdierer hintereinander und leitet den Übertrag jeweils weiter.



*Der Name kommt daher, dass der Übertrag wie eine Welle (*ripple*) von rechts nach links durchläuft.*

Und genau darin liegt das Problem

Der Ausgang steht erst fest, wenn der Übertrag *alle* Stufen durchlaufen hat.

Bei 64 Bit sind das 64 hintereinandergeschaltete Gatterlaufzeiten. **Die Verzögerung wächst *linear* mit der Wortbreite** – *und sie bestimmt, wie hoch der Prozessor getaktet werden darf.*

Zahlenbeispiel: Bei 20 Pikosekunden pro Gatterstufe und 2 Stufen pro Volladdierer:

$$64 \cdot 2 \cdot 20\,\text{ps} = 2560\,\text{ps} = 2{,}56\,\text{ns}.$$

Das entspräche einer Höchsttaktrate von $\frac{1}{2{,}56\,\text{ns}} = 391\ \text{MHz}$ – **lächerlich wenig für einen modernen Prozessor.**

Die Abhilfe heißt Carry-Lookahead: *Man berechnet die Überträge nicht nacheinander, sondern **parallel** aus allen Eingängen zugleich.*
*Die Idee: Eine Stelle **erzeugt** einen Übertrag, wenn* $a_i \wedge b_i$*, und sie **leitet ihn weiter**, wenn* $a_i \oplus b_i$*:*

$$g_i := a_i \wedge b_i, \qquad p_i := a_i \oplus b_i, \qquad c_{i+1} = g_i \vee (p_i \wedge c_i).$$

Setzt man das ineinander ein, entsteht ein Ausdruck, der c_{i+1} ***direkt*** *aus allen* a_j, b_j *berechnet – ohne Kette.*

Verfahren	Verzögerung	Gatteraufwand
Ripple-Carry	$O(n)$	$O(n)$
Carry-Lookahead	$O(\log n)$	$O(n \log n)$

***Hier begegnet uns zum ersten Mal der Grundkonflikt der Informatik**: Zeit gegen Platz. Man kauft Geschwindigkeit mit Bauteilen.* **Dieser Konflikt wird uns durch die ganze Reihe begleiten** – *in Band 3 als Komplexitätstheorie, in Band 4 als Cachehierarchie.*

7.3 Multiplexer und die Recheneinheit

Der Multiplexer – das „Wenn“ der Hardware

Definition 7.3. Ein **Multiplexer** (MUX) wählt anhand eines Steuersignals s eines von mehreren Eingangssignalen aus:

$$y = (\overline{s} \wedge a) \vee (s \wedge b).$$

s	y	**in einer Programmiersprache**
0	a	`y = s ? b : a;`
1	b	

Der Multiplexer ist die Hardwareentsprechung der Fallunterscheidung. Jedes `if` eines Programms landet letztlich entweder in einem Sprungbefehl (Kapitel 11) oder in einem Multiplexer.

Und er ist selbst funktional vollständig.
Setzt man in $y = (\overline{s} \wedge a) \vee (s \wedge b)$ feste Werte ein:

$a = 0, b = 1$	$y = s$	Durchleitung
$a = 1, b = 0$	$y = \overline{s}$	**NICHT**
$a = 0, b = x$	$y = s \wedge x$	**UND**
$a = x, b = 1$	$y = s \vee x$	**ODER**

Damit lässt sich ebenfalls jede Boolesche Funktion aufbauen – **und genau darauf beruhen die frei programmierbaren Logikbausteine (FPGA), deren Grundzelle eine kleine Wertetabelle mit Multiplexer ist.**

Die arithmetisch-logische Einheit

*Die **ALU** ist das Rechenwerk des Prozessors. Sie kann mehrere Operationen und wählt zwischen ihnen anhand eines Steuersignals* – **sie ist also ein Bündel von Schaltnetzen plus Multiplexer.**

Steuerbits	Operation	Realisierung
00	$a + b$	Addierer
01	$a - b$	Addierer mit $\overline{b}$, $c_{\text{in}} = 1$
10	$a \wedge b$	UND-Gatter, bitweise
11	$a \vee b$	ODER-Gatter, bitweise

Die zweite Zeile verdient Beachtung. *Subtraktion braucht* ***kein eigenes Werk*** **– man invertiert den zweiten Operanden und setzt den Eingangsübertrag auf 1:**

$$a + \overline{b} + 1 = a + (-b) = a - b,$$

genau nach der Zweierkomplementregel aus Kapitel 2. ✓

> ***Hier zahlt sich die Entscheidung für das Zweierkomplement bar aus:*** *Ein einziges XOR-Gatter pro Bit und eine Steuerleitung verwandeln den Addierer in einen Subtrahierer.*

Denn $b_i \oplus 1 = \overline{b_i}$ *und* $b_i \oplus 0 = b_i$ **– das XOR wirkt als steuerbarer Inverter.** ✓

Die Statusbits. *Zusätzlich liefert die ALU vier Merker, die für Sprungbefehle gebraucht werden:*

Z	*zero*	Ergebnis ist null (NOR über alle Bits)
N	*negative*	oberstes Bit gesetzt
C	*carry*	Übertrag aus der obersten Stelle
V	*overflow*	$c_{n-1} \oplus c_n$ (Kapitel 2)

Der Vergleich `a < b` *wird intern als Subtraktion ausgeführt, deren Ergebnis man wegwirft* **– man interessiert sich nur für die Merker.**

Aufgabe 7.1.

(a) Bauen Sie einen Volladdierer vollständig aus NAND-Gattern und zählen Sie sie.

(b) Entwerfen Sie eine Schaltung, die prüft, ob ein 4-Bit-Wort durch 3 teilbar ist.

(c) Bestimmen Sie die Verzögerung eines 32-Bit-Addierers in beiden Bauarten und rechnen Sie die maximale Taktrate aus.

Lösung Schritt für Schritt.

(a) Volladdierer aus NAND.
Schritt 1 – XOR aus NAND.
Wir brauchen zuerst XOR. Die bekannte Zerlegung:

$$a \oplus b = (a \vee b) \wedge \overline{(a \wedge b)}.$$

Setzt man $t := a \mathbin{\overline{\wedge}} b$, *so ist* $t = \overline{a \wedge b}$, *und man rechnet:*

$$a \oplus b = (a \mathbin{\overline{\wedge}} t) \mathbin{\overline{\wedge}} (b \mathbin{\overline{\wedge}} t).$$

Schritt 2 – Die Formel nachprüfen.
Wir gehen alle vier Fälle durch.

a	b	t	$a \mathbin{\overline{\wedge}} t$	$b \mathbin{\overline{\wedge}} t$	**Ergebnis**
0	0	1	1	1	$\overline{1 \wedge 1} = \mathbf{0}$
0	1	1	1	0	$\overline{1 \wedge 0} = \mathbf{1}$
1	0	1	0	1	**1**
1	1	0	1	1	**0**

Genau die XOR-Tabelle. ✓ **Aufwand: 4 NAND-Gatter** *(eines für* t*, zwei für die Klammern, eines für die Verknüpfung).*

Schritt 3 – Die Summe.

$$s = (a \oplus b) \oplus c_{\text{in}}.$$

Zwei XOR-Blöcke: $2 \cdot 4 = 8$ **NAND-Gatter.**
Schritt 4 – Der Übertrag geschickt.
Man kann den Übertrag aus den Zwischenergebnissen der XOR-Blöcke gewinnen. Sei $u := a \oplus b$. *Dann:*

$$c_{\text{out}} = (a \wedge b) \vee (u \wedge c_{\text{in}}).$$

Warum das stimmt: Ist $u = 0$, *so sind* a *und* b *gleich, und der Übertrag ist genau dann 1, wenn beide 1 sind. Ist* $u = 1$, *so ist genau eines der beiden 1, und ein Übertrag entsteht genau dann, wenn zusätzlich* $c_{in} = 1$ *ist.* ✓
In NAND-Form (nach de Morgan):

$$c_{\text{out}} = \overline{\overline{(a \wedge b)} \wedge \overline{(u \wedge c_{\text{in}})}} = (a \mathbin{\overline{\wedge}} b) \mathbin{\overline{\wedge}} (u \mathbin{\overline{\wedge}} c_{\text{in}}).$$

Drei NAND-Gatter – **und** $a \mathbin{\overline{\wedge}} b$ **ist bereits im ersten XOR-Block als** t **vorhanden.** ✓

Schritt 5 – Die Bilanz.

Teil	NAND	Bemerkung
erstes XOR	4	liefert u und t
zweites XOR	4	liefert s
Übertrag	2	t wiederverwendet
Summe	**10**	

Zehn Gatter für eine einstellige Addition. Ein 64-Bit-Addierer braucht davon 64 Stück – **640 Gatter, oder in CMOS etwa 2560 Transistoren.**

Zum Vergleich: Ein moderner Prozessor enthält etwa 10^{10} *Transistoren.* **Der gesamte Addierer macht davon weniger als ein Millionstel aus** – *das meiste geht in Cache, Sprungvorhersage und Parallelität.*

(b) Teilbarkeit durch 3.
Schritt 1 – Die Wertetabelle aufstellen.
Wir suchen $f(x_3x_2x_1x_0) = 1$ *genau dann, wenn der Wert durch 3 teilbar ist.*

Wert	binär	f	Wert	binär	f
0	0000	**1**	8	1000	0
1	0001	0	9	1001	**1**
2	0010	0	10	1010	0
3	0011	**1**	11	1011	0
4	0100	0	12	1100	**1**
5	0101	0	13	1101	0
6	0110	**1**	14	1110	0
7	0111	0	15	1111	**1**

Schritt 2 – Die Minterme.

$$f = \sum m(0, 3, 6, 9, 12, 15).$$

Schritt 3 – Minimieren mit dem KV-Diagramm (Kapitel 6).
Man trägt die sechs Einsen ein und sucht Blöcke.

$x_3x_2 \backslash x_1x_0$	00	01	11	10
00	**1**	0	**1**	0
01	0	0	0	**1**
11	**1**	0	**1**	0
10	0	**1**	0	0

Das Muster ist auffällig zerstreut. Es gibt keine Zweierblöcke außer über die Ränder hinweg – **die Funktion lässt sich kaum vereinfachen.**

*Immerhin bilden m_0, m_3, m_{12}, m_{15} einen Viererblock über die Spalten- **und** Zeilenränder hinweg:* $\overline{(x_3 \oplus x_2)} \wedge \overline{(x_1 \oplus x_0)}$.

Schritt 4 – Die elegante Lösung.

Statt zu minimieren, denkt man um. **Modulo 3 gilt:**

$$2^0 \equiv 1, \quad 2^1 \equiv 2, \quad 2^2 \equiv 1, \quad 2^3 \equiv 2 \pmod 3.$$

Die Potenzen wechseln zwischen 1 und 2, also zwischen $+1$ und -1.

$$x \equiv (x_0 - x_1 + x_2 - x_3) \pmod 3.$$

Das ist genau die alternierende Quersumme – dieselbe Regel wie die Elferregel im Dezimalsystem (Mathematik-Band 1, Kapitel 2), *nur dass hier* $2 \equiv -1 \bmod 3$ *ist statt* $10 \equiv -1 \bmod 11$.

Also: x ist durch 3 teilbar genau dann, wenn

$$(x_0 + x_2) - (x_1 + x_3) \in \{-2,\ 0,\ 2\}.$$

Mit $u := x_0 + x_2$ und $v := x_1 + x_3$ (jeweils 0, 1 oder 2) heißt das $u = v$.

Schritt 5 – Als Schaltung.

$$f = \left(\overline{u_1 \oplus v_1}\right) \wedge \left(\overline{u_0 \oplus v_0}\right),$$

wobei u_1u_0 die Zweibitdarstellung von u ist – **man baut zwei Halbaddierer und einen Zweibit-Vergleicher.**

Für 4 Bit ist das kaum billiger als die direkte Lösung. **Für 32 Bit ist es der Unterschied zwischen machbar und unmöglich** – *die Wertetabelle hätte dort 4,3 Milliarden Zeilen.*

Die Lehre: *Minimierungsverfahren arbeiten auf der Wertetabelle und wachsen exponentiell. Ein struktureller Einfall ersetzt sie* – **und diesen Einfall kann kein Algorithmus liefern.**

(c) Verzögerung und Taktrate.
Schritt 1 – Die Annahmen.

Gatterlaufzeit τ	20 ps
Wortbreite n	32 Bit
Stufen pro Volladdierer	2

Schritt 2 – Ripple-Carry.

$$T = n \cdot 2\tau = 32 \cdot 2 \cdot 20\,\text{ps} = 1280\,\text{ps} = 1{,}28\,\text{ns}.$$

$$f_{\max} = \frac{1}{1{,}28\,\text{ns}} = 781\,\text{MHz}.$$

Schritt 3 – Carry-Lookahead.
Bei baumartigem Aufbau mit Verzweigungsgrad 4:

$$\text{Stufen} = 2 \cdot \log_4 32 = 2 \cdot 2{,}5 = 5.$$

Zusammen mit der Erzeugung der g_i, p_i und der Endsumme etwa 8 Stufen:

$$T = 8 \cdot 20\,\text{ps} = 160\,\text{ps}.$$

$$f_{\max} = \frac{1}{160\,\text{ps}} = 6{,}25\,\text{GHz}.$$

Schritt 4 – Der Vergleich.

Bauart	**Verzögerung**	**Taktrate**	**Gatter**
Ripple-Carry	1280 ps	781 MHz	≈ 320
Carry-Lookahead	160 ps	6,25 GHz	≈ 1200
Verhältnis	**8×**	**8×**	3,8×

Achtfache Geschwindigkeit für den knapp vierfachen Aufwand – ein sehr guter Handel, und deshalb hat jeder reale Prozessor einen Lookahead-Addierer.

Schritt 5 – Warum reale Prozessoren trotzdem nicht bei 6 GHz stehen bleiben.
Der Addierer ist nicht der einzige Weg durch den Prozessor. Der ***kritische Pfad*** *– die langsamste Kette zwischen zwei Registern – geht meist durch andere Teile.*

*Und die Grenze ist heute nicht die Laufzeit, sondern die **Verlustleistung**:*

$$P \approx C \cdot U^2 \cdot f.$$

*Die Leistung wächst **linear** mit dem Takt und **quadratisch** mit der Spannung* **– und höhere Takte brauchen höhere Spannungen, also wächst sie faktisch kubisch.** *Bei etwa 5 GHz wird ein Chip nicht mehr gekühlt.*

Das ist der Grund für die Wende von 2005: *Statt schnellerer Kerne baut man seither **mehr** Kerne* – **und schiebt damit das Problem von der Physik zur Programmierung, wo es bis heute nicht gelöst ist.** *Band 5 behandelt die Folgen.* Nisan und Schocken[18] lassen den Leser einen vollständigen Rechner aus NAND-Gattern aufbauen; Patterson und Hennessy[20] behandeln Addierer und ALU im Detail; Hillis[10] erklärt die Grundidee ohne jede Formel.

Part III

Das Gedächtnis

8 Wie Rückkopplung Gedächtnis erzeugt

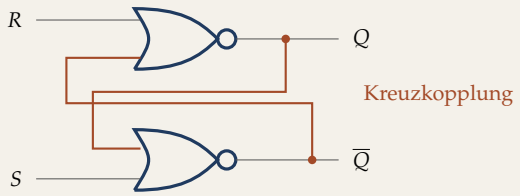
Der entscheidende Schritt

> *Alle bisherigen Schaltungen waren **kreisfrei**: Das Signal lief von den Eingängen zu den Ausgängen, und der Ausgang hing nur davon ab, was gerade anlag.* **Ein solches Schaltnetz kann nichts behalten.**

Wir tun jetzt das Verbotene: Wir führen einen Ausgang auf einen Eingang zurück.

*Damit entsteht ein **Kreis** – und mit ihm etwas qualitativ Neues.*

Das einfachste Beispiel: zwei über Kreuz geschaltete NOR-Gatter.



*Diese Schaltung heißt **RS-Flipflop** oder **RS-Latch*** (von *reset* und *set*).

8.1 Das RS-Flipflop

Wie es sich verhält

Wir gehen die Fälle einzeln durch. **Erinnerung: NOR liefert genau dann 1, wenn *beide* Eingänge 0 sind.**

Fall 1 – $S = 1$, $R = 0$ **(Setzen).**
Das untere Gatter hat eine 1 am Eingang, also ist sein Ausgang $\overline{Q} = 0$.
Das obere Gatter hat dann $R = 0$ *und* $\overline{Q} = 0$ *– beide Eingänge 0 – also ist* $Q = 1$.

$$\boxed{Q = 1}$$

Fall 2 – $S = 0$, $R = 1$ **(Rücksetzen).**

Symmetrisch: $Q = 0$ und $\overline{Q} = 1$. ✓
Fall 3 – $S = 0$**,** $R = 0$ **(Halten).**

> *Jetzt wird es interessant. Beide Eingänge sind 0, also hängt jedes Gatter* ***nur*** *vom Ausgang des anderen ab.*

War vorher $Q = 1$*, so ist* $\overline{Q} = 0$*; das obere Gatter sieht zweimal 0 und bleibt bei* $Q = 1$*.*
Der Zustand hält sich selbst. ✓
War vorher $Q = 0$*, so ist* $\overline{Q} = 1$*; das obere Gatter sieht eine 1 und bleibt bei* $Q = 0$*.*
Ebenfalls stabil. ✓

> ***Es gibt also zwei stabile Zustände, und die Schaltung bleibt in dem, in den man sie gebracht hat.*** Das ist Gedächtnis – ein gespeichertes Bit.

Fall 4 – $S = 1$**,** $R = 1$ **(verboten).**
Beide Gatter haben eine 1 am Eingang, also sind beide Ausgänge 0:

$$Q = 0 \quad \text{und} \quad \overline{Q} = 0.$$

Das widerspricht der Benennung: $\overline{Q}$ *soll das Gegenteil von* Q *sein.*

Der wirklich gefährliche Teil

Das eigentliche Problem tritt beim *Verlassen* dieses Zustands auf.
*Gehen S und R **gleichzeitig** auf 0, so versuchen beide Gatter zugleich, auf 1 zu gehen. Jedes sieht dabei den alten Wert des anderen und kippt wieder zurück.*

> *Die Schaltung kann in eine **Schwingung** geraten oder in einen Zwischenzustand, in dem die Spannung weder 0 noch 1 entspricht.* **Man nennt das Metastabilität.**

Und Metastabilität ist nicht bloß eine Verzögerung – sie ist grundsätzlich unbeschränkt.
*Man kann beweisen: Es gibt **keine** obere Schranke für die Zeit, nach der ein metastabiler Zustand aufgelöst ist.* **Nur die *Wahrscheinlichkeit*, dass er noch andauert, fällt exponentiell:**

$$P(t) \approx \mathrm{e}^{-t/\tau}.$$

Wartezeit	P (bei $\tau = 50$ ps)	**mittlere Zeit bis Fehler**
0,5 ns	$4{,}5 \cdot 10^{-5}$	Sekunden
1 ns	$2 \cdot 10^{-9}$	Stunden
2 ns	$4 \cdot 10^{-18}$	Jahrmilliarden

Man kann Metastabilität also nicht ausschließen, nur unwahrscheinlich machen. *Der übliche Weg ist ein **Synchronisierer** aus zwei hintereinandergeschalteten Flipflops – er gibt dem ersten eine ganze Taktperiode Zeit, sich zu entscheiden.*

> *Das ist eine der wenigen Stellen, an denen die Informatik der Physik nicht ausweichen kann.* **Jede Schnittstelle zwischen zwei unabhängigen Taktbereichen trägt dieses Restrisiko** – *man rechnet es aus und legt es fest, aber man beseitigt es nicht.*

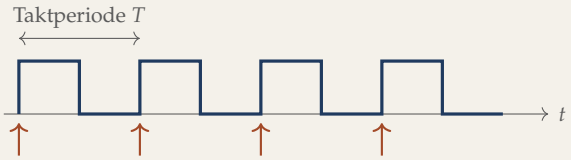
S	R	Q_{neu}	**Bedeutung**
0	0	Q_{alt}	**halten**
1	0	1	setzen
0	1	0	rücksetzen
1	1	—	**verboten**

8.2 Der Takt

Warum ein Rechner einen Herzschlag braucht

> *Das RS-Flipflop reagiert **sofort** auf jede Änderung seiner Eingänge. In einer großen Schaltung ist das unbrauchbar: Signale laufen verschieden schnell, und Zwischenwerte flackern über die Leitungen.*

*Man nennt diese Zwischenwerte **Hasards*** (glitches)*. Sie sind kurz und verschwinden von selbst – aber ein Flipflop, das genau dann hinschaut, speichert Unsinn.*
Die Lösung ist der Takt: *ein Signal, das regelmäßig zwischen 0 und 1 wechselt und allen Speicherelementen sagt, **wann** sie hinschauen dürfen.*



steigende Flanken – hier wird übernommen

Definition 8.1. Ein **taktflankengesteuertes D-Flipflop** übernimmt den Wert seines Dateneingangs D genau im Moment der steigenden Taktflanke und hält ihn bis zur nächsten.

> ***Das ist das Standardbauelement jedes Rechners.*** Der Buchstabe D steht für *data* oder *delay* – denn das Flipflop verzögert sein Eingangssignal um genau einen Takt:
>
> $$Q(n+1) = D(n).$$

Wie man es aus zwei Latches baut.
Man schaltet zwei RS-Latches hintereinander, das erste bei Takt $= 0$ durchlässig, das zweite bei Takt $= 1$. **So ist zu keinem Zeitpunkt ein durchgehender Weg vom Eingang zum Ausgang offen** – *daher der Name **Master-Slave-Flipflop**.*

Die Bauart löst zugleich das Problem des verbotenen Zustands: Aus einem einzigen D erzeugt man $S = D$ und $R = \overline{D}$, die nie beide 1 sein können. ✓

Setup- und Hold-Zeit

Ein Flipflop verlangt, dass sein Eingang um die Taktflanke herum *ruhig* ist:

Setup-Zeit t_{su}	Der Wert muss **vor** der Flanke schon stabil anliegen
Hold-Zeit t_h	Er muss **nach** der Flanke noch kurz stabil bleiben

Wird eine der beiden Bedingungen verletzt, kann Metastabilität eintreten.

Daraus folgt die zentrale Gleichung des Rechnerentwurfs:

$$\boxed{T_{\text{Takt}} \;\geq\; t_{cq} + t_{\text{Logik,max}} + t_{su}}$$

– Ausgangsverzögerung des Flipflops, plus längster Weg durch die Logik, plus Setup-Zeit.

Der längste Weg zwischen zwei Flipflops bestimmt die Taktrate des ganzen Chips. Er heißt **kritischer Pfad**, und ihn zu verkürzen ist die Hauptarbeit jedes Hardwareentwurfs.

Beispiel: Bei $t_{cq} = 30\,ps$, $t_{Logik} = 200\,ps$ *und* $t_{su} = 25\,ps$ *ergibt sich*

$$T \geq 255\,\text{ps} \Longrightarrow f_{\max} = 3{,}92\,\text{GHz}. \qquad \checkmark$$

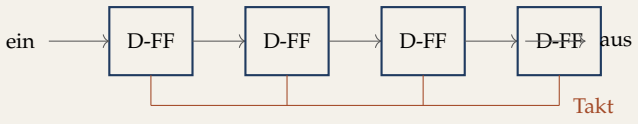
8.3 Register und Speicher

Von einem Bit zu vielen

Definition 8.2. Ein **Register** ist eine Gruppe von n D-Flipflops mit gemeinsamem Takt. Es speichert ein n-Bit-Wort.

Mehr ist ein Register nicht – n Flipflops nebeneinander. **Ein 64-Bit-Prozessorregister besteht aus 64 solchen Zellen.**

Das Schieberegister. *Verbindet man den Ausgang jedes Flipflops mit dem Eingang des nächsten, so wandert der Inhalt bei jedem Takt um eine Stelle weiter.*



Damit realisiert man das Links- und Rechtsschieben aus Kapitel 2 – **und damit die Multiplikation mit Zweierpotenzen.**

Der Zähler. *Ein Register mit Addierer, der bei jedem Takt 1 addiert.* **Er wird uns in Kapitel 10 als Befehlszähler wiederbegegnen – dem Bauteil, das dafür sorgt, dass ein Programm voranschreitet.**

Speicherart	Bauart	Zugriffszeit
Register	D-Flipflops	< 1 Takt
SRAM (Cache)	6 Transistoren/Bit	wenige Takte
DRAM (Hauptsp.)	1 Transistor + Kondensator	≈ 200 Takte
Flash (SSD)	Ladung in isoliertem Gate	$\approx 10^5$ Takte

Der Preis der Kapazität ist die Geschwindigkeit. Ein DRAM-Bit kostet einen Transistor statt sechs – dafür verliert der Kondensator seine Ladung und muss tausende Male pro Sekunde **aufgefrischt** werden.

*Das Wort **dynamisch** in DRAM meint genau das.* **Und es ist der Grund, warum der Hauptspeicher so viel langsamer ist als der Prozessor** – *ein Missverhältnis, das Band 4 unter dem Namen **Memory Wall** behandelt.*

Aufgabe 8.1.

(a) Analysieren Sie das RS-Flipflop aus NAND-Gattern und vergleichen Sie es mit der NOR-Variante.

(b) Entwerfen Sie einen 3-Bit-Zähler und geben Sie seine Zustandsfolge an.

(c) Bauen Sie eine Ampelsteuerung als getaktetes Schaltwerk.

Lösung Schritt für Schritt.

(a) Das NAND-Flipflop.
Schritt 1 – Die Schaltung.
Man ersetzt die beiden NOR-Gatter durch NAND-Gatter, sonst nichts.
Schritt 2 – Das Verhalten durchgehen.

*Erinnerung: NAND liefert genau dann 0, wenn **beide** Eingänge 1 sind.*

*Fall $\overline{S} = 0$: Das obere Gatter hat eine 0 am Eingang, also ist sein Ausgang **1**, gleichgültig was sonst anliegt.*

$$\Longrightarrow Q = 1.$$

Fall $\overline{R} = 0$: $\overline{Q} = 1$, also $Q = 0$. ✓
Fall $\overline{S} = \overline{R} = 1$: Beide Gatter hängen nur voneinander ab – **Haltezustand.** ✓
Fall $\overline{S} = \overline{R} = 0$: Beide Ausgänge werden 1 – **der verbotene Zustand.**
Schritt 3 – Der Vergleich.

	NOR-Variante	**NAND-Variante**
Aktiv bei	**1**	**0**
Eingänge heißen	S, R	$\overline{S}$, $\overline{R}$
Verboten	$S = R = 1$	$\overline{S} = \overline{R} = 0$
Ruhezustand	$0, 0$	$1, 1$

Beide sind dieselbe Schaltung in dualer Form – man erhält die eine aus der anderen, indem man alle Signale invertiert. **Das ist der Dualitätssatz der Booleschen Algebra** (Kapitel 5), *diesmal in Blech.*

Schritt 4 – Warum das praktisch bedeutsam ist.
Da NAND das billigere CMOS-Gatter ist (Kapitel 7), **wird in der Praxis fast immer die NAND-Variante gebaut.** *Die Eingänge heißen dann **low-aktiv** und werden in Schaltplänen mit einem Querstrich oder einem Kreis gekennzeichnet.*

Wer sich je gefragt hat, warum in der Digitaltechnik so viele Signale „verkehrt herum" laufen – **das ist der Grund.**

(b) Ein 3-Bit-Zähler.
Schritt 1 – Die gewünschte Folge.

Takt	Zustand	binär	nächster
0	0	000	001
1	1	001	010
2	2	010	011
3	3	011	100
4	4	100	101
5	5	101	110
6	6	110	111
7	7	111	**000**

Schritt 2 – Wann kippt welches Bit?

Man sieht es der Tabelle an: ***Ein Bit kippt genau dann, wenn alle Bits rechts davon 1 sind.*** Das ist genau die Übertragsregel der Addition von 1.

$$q_0' = \overline{q_0}, \qquad q_1' = q_1 \oplus q_0, \qquad q_2' = q_2 \oplus (q_1 \wedge q_0).$$

Schritt 3 – Probe.
Von $(011)_2$ *(Wert 3) aus:*

$$q_0' = \overline{1} = 0, \quad q_1' = 1 \oplus 1 = 0, \quad q_2' = 0 \oplus (1 \wedge 1) = 1.$$
$$\Longrightarrow (100)_2 = 4. \qquad \checkmark$$

Von $(111)_2$ *(Wert 7):*

$$q_0' = 0, \quad q_1' = 1 \oplus 1 = 0, \quad q_2' = 1 \oplus 1 = 0 \Longrightarrow (000)_2. \qquad \checkmark$$

Der Überlauf ergibt sich von selbst – **genau wie beim Zahlenkreis aus Kapitel 2.**

Schritt 4 – Zwei Bauarten.

	asynchron	**synchron**
Takt	Ausgang treibt nächstes FF	**alle FF am selben Takt**
Aufwand	minimal	Logik nötig
Verzögerung	$O(n)$, kumuliert	konstant
Zwischenzustände	**ja, kurzzeitig falsch**	nein

Der asynchrone Zähler lügt kurzzeitig

Beim Übergang von $(011)_2$ *nach* $(100)_2$ *kippen die Bits* ***nacheinander****:*

$$011 \to 010 \to 000 \to 100.$$

Für wenige Nanosekunden zeigt der Zähler die Werte 2 und 0 an – **Werte, die er nie annehmen sollte.**

Liest eine andere Schaltung genau dann mit, so erhält sie Unsinn. *Deshalb baut man in Prozessoren ausschließlich synchrone Zähler* – **der zusätzliche Logikaufwand ist der Preis für Verlässlichkeit.**

(c) Die Ampelsteuerung.
Schritt 1 – Die Zustände festlegen.

Nr.	Anzeige	Kodierung	Dauer
0	Rot	00	lang
1	Rot-Gelb	01	kurz
2	Grün	11	lang
3	Gelb	10	kurz

> ***Die Kodierung ist mit Bedacht gewählt:*** *Von Zustand zu Zustand ändert sich genau* ***ein*** *Bit* ($00 \to 01 \to 11 \to 10 \to 00$). **Das ist der Gray-Code.**

Der Vorteil: Es gibt keine Zwischenzustände. Kippten zwei Bits gleichzeitig und eines minimal früher, so erschiene kurz ein falscher Zustand – **bei einer Ampel wäre das „Grün" statt „Rot".**

Derselbe Code wird in Drehgebern und Positionssensoren verwendet, aus genau demselben Grund.

Schritt 2 – Die Übergangsfunktion.
Mit $z_1 z_0$ als Zustand und t als Signal „Zeit abgelaufen":

z_1	z_0	t	z_1'	z_0'
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	1
1	1	0	1	1
1	1	1	1	0
1	0	0	1	0
1	0	1	0	0

Schritt 3 – Die Gleichungen ablesen.
Man liest die Einsen der Spalte z_1' ab (Zeilen 4, 5, 6, 7):

$$z_1' = (\overline{z_1} \wedge z_0 \wedge t) \vee (z_1 \wedge z_0 \wedge \overline{t}) \vee (z_1 \wedge z_0 \wedge t) \vee (z_1 \wedge \overline{z_0} \wedge \overline{t}).$$

Die mittleren beiden fassen sich zu $z_1 \wedge z_0$ zusammen (Kapitel 6):

$$z_1' = (\overline{z_1} \wedge z_0 \wedge t) \vee (z_1 \wedge z_0) \vee (z_1 \wedge \overline{z_0} \wedge \overline{t}).$$

Für z_0' (Zeilen 2, 3, 4, 5):

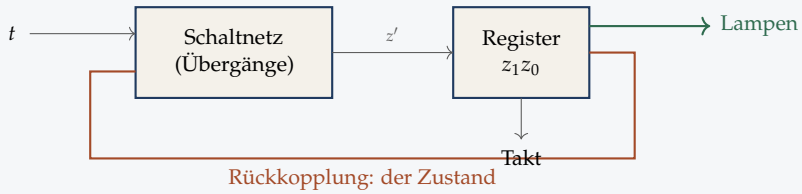
$$z_0' = (\overline{z_1} \wedge \overline{z_0} \wedge t) \vee (\overline{z_1} \wedge z_0) \vee (z_1 \wedge z_0 \wedge \bar{t}).$$

Schritt 4 – Die Ausgangsfunktion.

Lampe	Formel	Zustände
Rot	$\overline{z_1}$	0 und 1
Gelb	$z_1 \oplus z_0$	1 und 3
Grün	$z_1 \wedge z_0$	nur 2

Probe für Zustand 1 ($z_1 z_0 = 01$): Rot $= \bar{0} = 1$ ✓, Gelb $= 0 \oplus 1 = 1$ ✓, Grün $= 0 \wedge 1 = 0$ ✓ – **Rot-Gelb.** ✓

Schritt 5 – Der Aufbau.



Das ist die allgemeine Bauform jedes getakteten Systems: *ein Schaltnetz, dessen Ausgang über ein Register auf seinen Eingang zurückgeführt wird.*

Man nennt es Schaltwerk oder endlichen Automaten – *und das ist der Gegenstand des nächsten Kapitels.*

Bemerkenswert ist, wie wenig zwischen der Ampel und dem Prozessor liegt: **Beide sind Schaltnetz plus Register plus Rückkopplung.** *Der Unterschied ist die Zahl der Zustände – vier gegen $2^{\text{einige Milliarden}}$.* Kohavi und Jha[16] behandeln Flipflops und Schaltwerke systematisch; Patterson und Hennessy[20] zeigen den Weg zum Prozessor; Bryant und O'Hallaron[2] erklären, warum Metastabilität den Programmierer betrifft.

9 Endliche Automaten

Das Modell hinter allem Getakteten

*Die Ampel aus Kapitel 8 war ein Sonderfall. Wir abstrahieren jetzt davon und erhalten das erste echte **Berechnungsmodell** dieser Reihe.*

Definition 9.1. Ein **deterministischer endlicher Automat** (DEA) ist ein Fünftupel

$$A = (Q, \Sigma, \delta, q_0, F)$$

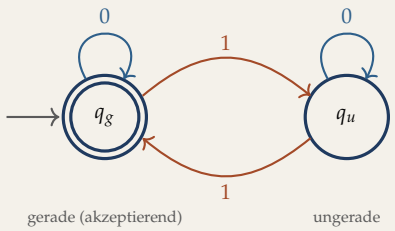
mit

Q	endliche Menge von **Zuständen**
Σ	endliches **Eingabealphabet**
δ	**Übergangsfunktion** $Q \times \Sigma \to Q$
$q_0 \in Q$	**Startzustand**
$F \subseteq Q$	Menge der **akzeptierenden** Zustände

Wie er arbeitet. *Er liest die Eingabe zeichenweise von links nach rechts und wechselt dabei nach δ den Zustand.* **Steht er am Ende in einem Zustand aus F, so akzeptiert er die Eingabe, sonst verwirft er sie.**

Der entscheidende Punkt ist das Wort „endlich“. Der Automat hat kein Papier, keinen Speicher, keine Möglichkeit zurückzuspringen. **Alles, was er sich merken kann, steckt in seinem *momentanen Zustand*** – und davon gibt es nur endlich viele.

Ein Beispiel: gerade Anzahl von Einsen.



Doppelter Kreis = akzeptierend. Der Pfeil von außen markiert den Startzustand.

Probe mit der Eingabe 10110*:*

$$q_g \xrightarrow{1} q_u \xrightarrow{0} q_u \xrightarrow{1} q_g \xrightarrow{1} q_u \xrightarrow{0} q_u.$$

Endzustand q_u *–* ***verworfen.*** **Und tatsächlich enthält** 10110 **drei Einsen, also eine ungerade Anzahl.** ✓

Man beachte, was der Automat ***nicht*** *tut: Er zählt die Einsen nicht.* **Er merkt sich nur die *Parität*** – ein einziges Bit. *Mehr braucht er nicht, und mehr könnte er auch nicht.*

9.1 Was ein Automat kann

Reguläre Sprachen

Definition 9.2. Eine Menge von Wörtern (eine **Sprache**) heißt **regulär**, wenn es einen endlichen Automaten gibt, der genau sie akzeptiert.

Sprache	regulär?	Zustände
gerade Anzahl Einsen	ja	2
durch 3 teilbare Binärzahlen	ja	3
enthält `abc` als Teilwort	ja	4
gültige Gleitkommazahl	ja	≈ 8
gültige E-Mail-Adresse (grob)	ja	viele
$a^n b^n$ (gleich viele a wie b)	**nein**	—
korrekt geklammerte Ausdrücke	**nein**	—
Palindrome	**nein**	—

Warum $a^n b^n$ nicht geht – ein sauberer Beweis.
Angenommen, ein Automat A mit k Zuständen akzeptiere genau die Wörter $a^n b^n$.
Schritt 1. *Wir füttern ihn mit a^{k+1}, also mit mehr a's, als er Zustände hat. Dabei durchläuft er $k+2$ Zustände (einschließlich des Starts).*

Schritt 2. *Nach dem* ***Schubfachprinzip*** (Mathematik-Band 15, Kapitel 1) *muss er einen Zustand* ***zweimal*** *besuchen: Es gibt $i < j$ mit*

$$\delta^*(q_0, a^i) = \delta^*(q_0, a^j).$$

Schritt 3. *Der Automat kann diese beiden Situationen nicht unterscheiden – er ist im selben Zustand, und mehr weiß er nicht.* **Also verhält er sich beim Rest der Eingabe identisch:**

$$a^i b^i \text{ akzeptiert} \Longrightarrow a^j b^i \text{ ebenfalls akzeptiert.}$$

Aber $j \neq i$, also gehört $a^j b^i$ nicht zur Sprache. **Widerspruch.** ✓

> ***Die Argumentationsfigur ist zentral:*** *Ein endlicher Automat kann nicht beliebig weit zählen, weil er nur endlich viele Zustände hat und daher irgendwann anfangen muss, sich zu wiederholen.*

Die allgemeine Fassung heißt Pumping-Lemma. *Sie besagt: Jedes hinreichend lange Wort einer regulären Sprache enthält ein Teilstück, das man beliebig oft wiederholen („aufpumpen") darf, ohne die Sprache zu verlassen.* **Band 2 führt das aus.**

Die berühmteste praktische Folgerung

Man kann HTML nicht mit regulären Ausdrücken parsen.

> *Reguläre Ausdrücke – in der theoretischen, nicht der erweiterten Perl-Bedeutung – beschreiben* ***genau*** *die regulären Sprachen* (Satz von KLEENE, 1956). **Verschachtelte Klammerstrukturen sind aber nicht regulär.**

Ein `<div>` *kann ein* `<div>` *enthalten, das ein* `<div>` *enthält, beliebig tief.* **Genau wie bei $a^n b^n$ müsste der Automat die Schachtelungstiefe zählen – und das kann er nicht.**

Das ist kein Mangel der verfügbaren Werkzeuge, sondern ein ***Satz.*** **Man braucht einen Kellerautomaten** (Band 2 und 6) *– also einen Automaten mit einem Stapelspeicher.*

Mealy und Moore

Bisher hatte der Automat nur ein Urteil am Ende. Für Steuerungen braucht man laufend Ausgaben.

	Moore-Automat	**Mealy-Automat**
Ausgabe hängt ab von	**nur vom Zustand**	Zustand **und** Eingabe
Formel	$\lambda : Q \to \Gamma$	$\lambda : Q \times \Sigma \to \Gamma$
Ausgabe wechselt	nur bei Taktflanke	sofort bei Eingabeänderung
Zustände	meist mehr	meist weniger
Beispiel	**Ampel**	Serienaddierer

*Die Ampel aus Kapitel 8 war ein **Moore**-Automat: Die Lampen hingen nur vom Zustand ab.* **Und das ist auch richtig so** – *man möchte nicht, dass die Lampe flackert, wenn sich ein Eingangssignal ändert.*

Satz 9.3. *Zu jedem Mealy-Automaten gibt es einen äquivalenten Moore-Automaten und umgekehrt.*

Man erkauft die Umwandlung mit mehr Zuständen: Man spaltet jeden Zustand nach den möglichen Ausgaben auf. ✓

Der Serienaddierer als Mealy-Automat.

*Zwei Zahlen sollen bitweise von rechts addiert werden. Der Automat braucht nur **zwei** Zustände* – **„kein Übertrag“ und „Übertrag“.**

Zustand	Eingabe *ab*	neuer Zustand	Ausgabe
$c{=}0$	00	$c{=}0$	0
$c{=}0$	01, 10	$c{=}0$	1
$c{=}0$	11	$c{=}1$	0
$c{=}1$	00	$c{=}0$	1
$c{=}1$	01, 10	$c{=}1$	0
$c{=}1$	11	$c{=}1$	1

Ein Addierer für beliebig lange Zahlen mit genau zwei Zuständen. Er ist langsam – ein Takt pro Bit – aber er braucht nur einen einzigen Volladdierer und ein Flipflop, unabhängig von der Wortbreite.

Frühe Rechner arbeiteten so, weil Bauteile teuer waren. **Die Z3 von Konrad Zuse (1941) addierte seriell** – *und brauchte für eine Addition etwa drei Sekunden.*

9.2 Minimierung

Der kleinstmögliche Automat

Satz 9.4 (Myhill-Nerode). *Zu jeder regulären Sprache gibt es einen **eindeutig bestimmten** minimalen Automaten. Seine Zustandszahl ist die Zahl der Äquivalenzklassen der Relation*

$$x \sim y \iff \big(\text{für alle } z:\ xz \in L \Leftrightarrow yz \in L\big).$$

> *In Worten:* ***Zwei Eingabepräfixe sind gleichwertig, wenn keine Fortsetzung sie unterscheiden kann.*** Der minimale Automat hat für jede solche Klasse genau einen Zustand – er merkt sich exakt das, was er noch braucht, und nichts sonst.

Das ist eine bemerkenswert starke Aussage. *Bei den meisten Berechnungsmodellen gibt es **keinen** eindeutigen minimalen Vertreter* – **man kann nicht einmal entscheiden, ob ein Programm minimal ist** (Band 2 und 3).

Endliche Automaten sind hier die große Ausnahme, und deshalb sind sie so gut beherrschbar.

Das Verfahren. *Man startet mit zwei Klassen – akzeptierend und nicht akzeptierend – und spaltet sie so lange auf, bis sich nichts mehr ändert.*

1. Teile Q in F und $Q \setminus F$.
2. Zwei Zustände einer Klasse werden getrennt, wenn ein Eingabezeichen sie in **verschiedene** Klassen führt.
3. Wiederhole, bis keine Trennung mehr möglich ist.

Der Algorithmus von HOPCROFT *(1971) leistet das in* $O(n \log n)$. ✓

Aufgabe 9.1.

(a) Bauen Sie einen Automaten, der Binärzahlen auf Teilbarkeit durch 3 prüft, und beweisen Sie seine Korrektheit.

(b) Minimieren Sie einen gegebenen Automaten vollständig.

(c) Zeigen Sie, dass die Sprache der Palindrome nicht regulär ist, und ordnen Sie das Ergebnis ein.

Lösung Schritt für Schritt.

(a) Teilbarkeit durch 3.
Schritt 1 – Die Idee.

Der Automat liest die Binärzahl von links (höchstwertiges Bit zuerst) und merkt sich den ***Rest modulo 3.*** **Drei mögliche Reste, also drei Zustände.**

Schritt 2 – Die Übergangsregel herleiten.
Hat man bisher den Wert x gelesen und liest nun das Bit b, so ist der neue Wert

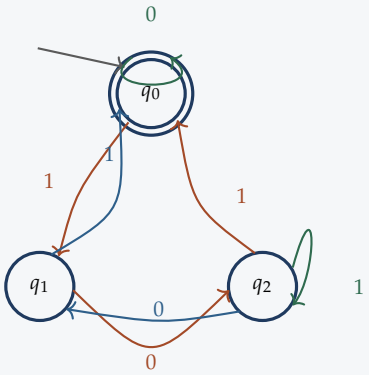
$$x' = 2x + b$$

(genau das Hornerschema aus Kapitel 2). **Modulo 3 gerechnet:**

$$x' \equiv 2x + b \pmod 3.$$

Schritt 3 – Die Übergangstabelle.

Zustand r	bei Bit 0	bei Bit 1
q_0 $(r = 0)$	$2 \cdot 0 + 0 = 0 \to q_0$	$2 \cdot 0 + 1 = 1 \to q_1$
q_1 $(r = 1)$	$2 \cdot 1 + 0 = 2 \to q_2$	$2 \cdot 1 + 1 = 3 \equiv 0 \to q_0$
q_2 $(r = 2)$	$2 \cdot 2 + 0 = 4 \equiv 1 \to q_1$	$2 \cdot 2 + 1 = 5 \equiv 2 \to q_2$



Schritt 4 – Probe mit $(1100)_2 = 12$.

$$q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_0 \xrightarrow{0} q_0 \xrightarrow{0} q_0.$$

Endzustand q_0 – ***akzeptiert.*** **Und** $12 = 3 \cdot 4$**.** ✓

Gegenprobe mit $(1011)_2 = 11$*:*

$$q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_2 \xrightarrow{1} q_2 \xrightarrow{1} q_2.$$

Endzustand q_2*, also Rest 2 –* **und** $11 = 3 \cdot 3 + 2$**.** ✓

Schritt 5 – Der Korrektheitsbeweis.

Wir beweisen durch Induktion über die Wortlänge:

> ***Behauptung:*** *Nach dem Lesen von* w *befindet sich der Automat im Zustand* q_r *mit* $r = \mathit{Wert}(w) \bmod 3$*.*

Anfang: *Das leere Wort hat den Wert 0, und der Startzustand ist* q_0*.* ✓
Schritt: *Gelte die Behauptung für* w*, sei also der Automat in* q_r *mit* $r \equiv \mathit{Wert}(w)$*. Beim Lesen von* b *geht er nach Konstruktion in* $q_{(2r+b) \bmod 3}$*. Und*

$$\mathrm{Wert}(wb) = 2 \cdot \mathrm{Wert}(w) + b \equiv 2r + b \pmod 3. \qquad ✓$$

Damit ist die Behauptung für wb *bewiesen.*

Schritt 6 – Die Verallgemeinerung.

> *Dasselbe Verfahren liefert einen Automaten für Teilbarkeit durch* ***jedes*** m *– mit genau* m *Zuständen.* **Er ist minimal**, *denn zwei verschiedene Reste lassen sich stets durch eine Fortsetzung unterscheiden.*

Vergleicht man das mit der Schaltnetzlösung aus Kapitel 7, so fällt auf: **Der Automat verarbeitet Zahlen *beliebiger* Länge mit konstantem Aufwand**, *während das Schaltnetz für jede Wortbreite neu gebaut werden muss.*

Das ist der Gewinn des Zustandsbegriffs: Zeit gegen Platz, wieder einmal.

(b) Minimierung.
Schritt 1 – Der Ausgangsautomat.

Zustand	bei 0	bei 1	akzeptierend?
A (Start)	B	C	nein
B	A	D	nein
C	E	F	**ja**
D	E	F	**ja**
E	C	F	**ja**
F	F	F	nein

Schritt 2 – Erste Aufteilung.

$$K_1 = \{A, B, F\}, \qquad K_2 = \{C, D, E\}.$$

Schritt 3 – K_1 prüfen.

	bei 0 landet in	bei 1 landet in
A	$B \in K_1$	$C \in K_2$
B	$A \in K_1$	$D \in K_2$
F	$F \in K_1$	$F \in K_1$

A und B verhalten sich gleich, F weicht bei Eingabe 1 ab. **Also spalten:**

$$K_1 \to \{A, B\} \text{ und } \{F\}.$$

Schritt 4 – K_2 prüfen.

	bei 0	bei 1
C	$E \in K_2$	$F \in \{F\}$
D	$E \in K_2$	$F \in \{F\}$
E	$C \in K_2$	$F \in \{F\}$

Alle drei verhalten sich gleich – **keine Spaltung.** ✓

Schritt 5 – Zweite Runde.
Nun ist die Aufteilung $\{A, B\}$, $\{C, D, E\}$, $\{F\}$. Wir prüfen $\{A, B\}$ erneut:

	bei 0	bei 1
A	$B \in \{A, B\}$	$C \in \{C, D, E\}$
B	$A \in \{A, B\}$	$D \in \{C, D, E\}$

Weiterhin gleich. **Das Verfahren ist stabil und bricht ab.**

Schritt 6 – Der minimale Automat.

Zustand	bei 0	bei 1	akzeptierend?
$\{A, B\}$ (Start)	$\{A, B\}$	$\{C, D, E\}$	nein
$\{C, D, E\}$	$\{C, D, E\}$	$\{F\}$	**ja**
$\{F\}$	$\{F\}$	$\{F\}$	nein

Sechs Zustände auf drei reduziert.

Schritt 7 – Was der Automat eigentlich erkennt.
*Man liest es jetzt ab: Der Automat akzeptiert Wörter, die **genau eine** Eins enthalten.*

> *$\{A, B\}$ heißt „noch keine Eins“, $\{C, D, E\}$ heißt „genau eine“, $\{F\}$ ist der Fangzustand für „zu viele“.* **Die ursprünglichen sechs Zustände enthielten Unterscheidungen, die für die Sprache bedeutungslos waren.**
>
> *Das ist der eigentliche Wert der Minimierung: Sie legt offen, was der Automat **tatsächlich** unterscheidet* – **und damit, was die Sprache eigentlich ist.**

(c) Palindrome.
Schritt 1 – Die Sprache.

$$L = \{w \in \{a, b\}^* : w = w^{\mathrm{rev}}\}.$$

Schritt 2 – Der Beweis mit Myhill-Nerode.
*Wir zeigen, dass es **unendlich viele** paarweise unterscheidbare Präfixe gibt. Betrachte die Wörter*

$$x_n := a^n b \qquad (n = 1, 2, 3, \dots).$$

***Behauptung:** Für $m \neq n$ sind x_m und x_n nicht äquivalent.*
Als Unterscheider wähle man $z := a^n$. Dann

$$x_n z = a^n b a^n \in L \quad \text{(Palindrom!)},$$

$$x_m z = a^m b a^n \notin L \quad \text{für } m \neq n,$$

denn ein Palindrom der Form $a^i b a^j$ verlangt $i = j$ – **es gibt nur ein b, das in der Mitte stehen muss.** ✓
Schritt 3 – Der Schluss.

> *Es gibt unendlich viele paarweise nicht äquivalente Präfixe, also nach Myhill-Nerode **unendlich viele Zustände**.* **Kein endlicher Automat kann die Sprache erkennen.**

Schritt 4 – Die Einordnung.

Modell	Speicher	erkennt
endlicher Automat	**nur Zustand**	reguläre Sprachen
Kellerautomat	+ **Stapel**	kontextfreie Sprachen
linear beschränkt	+ Band, begrenzt	kontextsensitive
Turingmaschine	+ **unbegrenztes Band**	aufzählbare Sprachen

*Das ist die **Chomsky-Hierarchie** (1956), und sie ist die Landkarte von Band 2.*

Bemerkenswert ist, wie schmal die Stufen sind.

> *Der Unterschied zwischen „kann nicht zählen“ und „kann alles Berechenbare“ besteht in genau zwei Zusätzen: einem Stapel und dann einem unbegrenzten Band.* **Mehr braucht es nicht, um von der Ampel zur universellen Maschine zu kommen.**

Und ein Rechner ist genau genommen ein endlicher Automat.
Ein Rechner mit 16 GB Speicher hat höchstens $2^{1{,}37 \cdot 10^{11}}$ *Zustände – eine unvorstellbar große, aber **endliche** Zahl.*

> ***Theoretisch kann er also nichts, was ein endlicher Automat nicht könnte.*** Praktisch ist die Zustandszahl so groß, dass diese Feststellung wertlos ist – **und deshalb modelliert man Rechner als Turingmaschinen mit unbegrenztem Band**, *obwohl das physikalisch falsch ist.*

Ein Modell muss nicht wahr sein, sondern nützlich. **Band 2 nimmt diesen Faden auf.** Sipser[23] entwickelt die Automatentheorie mit vorbildlicher Klarheit; Kohavi und Jha[16] verbinden sie mit dem Schaltungsentwurf; Harel und Feldman[8] ordnen sie in das Gesamtbild ein.

Part IV

Die Maschine

10 Der Von-Neumann-Rechner

Der Gedanke, der alles änderte

> *Die Rechner vor 1945 wurden **verdrahtet**, nicht programmiert. Wollte man die ENIAC eine andere Aufgabe rechnen lassen, so steckte man Kabel um und stellte Drehschalter – eine Arbeit von Tagen.*

Der Ausweg ist so einfach, dass er heute selbstverständlich erscheint:

> ***Man legt das Programm in denselben Speicher wie die Daten.***

*Das Programm ist dann eine Folge von Zahlen wie jede andere – und die Maschine kann diese Zahlen genauso laden, speichern und **verändern** wie Messwerte.*

Man nennt das das Prinzip des gespeicherten Programms (*stored program*), *und es steht im „First Draft of a Report on the EDVAC", den* John von Neumann *1945 verfasste.*

Zur Namensgebung

Der Name „Von-Neumann-Architektur" ist historisch unglücklich. *Der Bericht war eine Zusammenfassung der Diskussionen der gesamten EDVAC-Gruppe – vor allem von* J. Presper Eckert *und* John Mauchly*, die die Idee des gespeicherten Programms bereits 1944 in einem Aktenvermerk festgehalten hatten.*

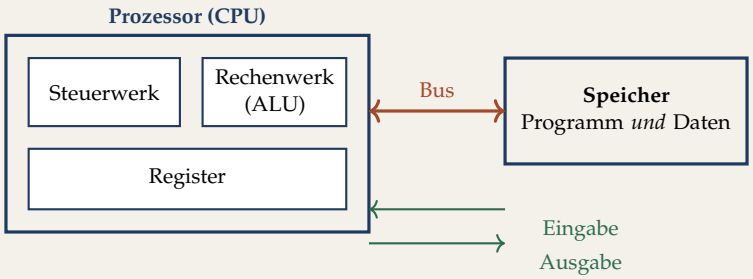
> *Der Entwurf zirkulierte mit **nur** von Neumanns Namen auf dem Deckblatt – offenbar ohne dessen Zutun, aber auch ohne seinen Widerspruch.* **Die Folge war ein jahrzehntelanger Prioritätsstreit und, weil die Veröffentlichung den Stand der Technik öffentlich machte, das Scheitern der Patentansprüche von Eckert und Mauchly.**

Unabhängig davon hatte Konrad Zuse 1936 in einem Patentantrag Ähnliches beschrieben, *und* Alan Turing *hatte 1936 mit der universellen Maschine das theoretische Gegenstück formuliert* – **eine Maschine, die die Beschreibung einer anderen Maschine als Eingabe liest.**

Von Neumann kannte Turings Arbeit gut; er hatte ihn 1937 in Princeton erlebt und ihm eine Assistentenstelle angeboten.

10.1 Die fünf Bauteile

Der Aufbau



Steuerwerk	Holt Befehle, entschlüsselt sie, steuert alles Übrige
Rechenwerk	Führt die Operationen aus (Kapitel 7)
Speicher	Enthält **Befehle und Daten in derselben Form**
Eingabewerk	Bringt Daten hinein
Ausgabewerk	Bringt Ergebnisse hinaus

Die wichtigsten Register.

Register	Kürzel	Aufgabe
Befehlszähler	PC	Adresse des **nächsten** Befehls
Befehlsregister	IR	der **gerade** bearbeitete Befehl
Adressregister	MAR	Adresse für den Speicherzugriff
Datenregister	MDR	Daten vom/zum Speicher
Akkumulator	ACC	Zwischenergebnis der ALU
Statusregister	FLAGS	Z, N, C, V (Kapitel 7)

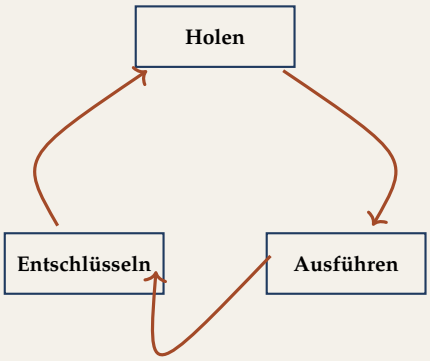
Der Befehlszähler ist das Herz der Sache. Er ist nichts als ein Zähler (Kapitel 8), der bei jedem Befehl weiterrückt – **und wenn man ihn stattdessen mit einem anderen Wert lädt, hat man einen *Sprung*.**

Das ist die ganze Kontrollstruktur eines Rechners: *Schleifen, Verzweigungen, Funktionsaufrufe – alles ist Manipulation des Befehlszählers.*

10.2 Der Befehlszyklus

Holen, Entschlüsseln, Ausführen

Der Rechner tut, solange er läuft, immer wieder dasselbe.



Der Zyklus im Einzelnen.

1	MAR ← PC	Adresse bereitstellen
2	MDR ← Speicher[MAR]	Befehl holen
3	IR ← MDR	in das Befehlsregister
4	**PC ← PC +1**	**schon jetzt weiterzählen**
5	Steuerwerk entschlüsselt IR	Opcode auswerten
6	Operanden holen	falls nötig
7	ALU ausführen	
8	Ergebnis speichern	Register oder Speicher

Schritt 4 verdient Beachtung: *Der Befehlszähler wird erhöht,* ***bevor*** *der Befehl ausgeführt wird.* **Deshalb bezieht sich ein relativer Sprung immer auf den *folgenden* Befehl** – *eine Quelle vieler Verwirrung beim Assemblerprogrammieren.*

Und ein Sprungbefehl ist nichts als:

$$\text{PC} \leftarrow \text{Zieladresse}.$$

*Ein **bedingter** Sprung prüft zuvor ein Flag:* **„Wenn Z gesetzt ist, lade PC mit der Zieladresse, sonst nicht."**

Der Flaschenhals

*Befehle **und** Daten gehen durch dieselbe Leitung zum selben Speicher. Der Prozessor kann also nicht gleichzeitig einen Befehl holen und einen Datenwert lesen.*

John Backus nannte das 1977 in seiner Turing-Award-Rede den Von-Neumann-Flaschenhals *und beschrieb ihn drastisch: Der Bus sei ein Rohr, durch das man Daten Wort für Wort hin- und herschiebe, während der eigentliche Zustand des Problems im Speicher liege und nur zäh durch dieses Rohr bewegt werden könne.*

Backus, der Erfinder von FORTRAN, benutzte die Rede, um für funktionale Programmierung zu werben – **ein Vorstoß, der damals wenig Wirkung hatte und heute umso mehr.**

	Von Neumann	**Harvard**
Speicher	**einer** für beides	**getrennt**
Bus	einer	zwei
Gleichzeitig?	nein	**ja**
Selbstmodifikation	möglich	unmöglich
Verwendung	Universalrechner	Signalprozessoren, Mikrocontroller

Moderne Prozessoren sind ein Zwitter: ***Nach außen Von Neumann****, damit Programme wie Daten geladen werden können –* ***innen Harvard****, mit getrennten Zwischenspeichern für Befehle und Daten.*

*Man nennt das **modifizierte Harvard-Architektur**, und praktisch jeder heutige Prozessor arbeitet so.* **Band 4 behandelt die Folgen.**

Die tiefere Bedeutung: Befehle sind Daten

Der Flaschenhals ist der Preis. Was man dafür bekommt, ist unbezahlbar: *Ein Programm kann ein Programm lesen, schreiben und verändern – auch sich selbst.*

Übersetzer	Ein Programm, das Programme erzeugt (Band 6)
Betriebssystem	Ein Programm, das Programme lädt und startet (Band 5)
Interpreter	Ein Programm, das ein Programm ausführt
Virtuelle Maschine	Ein Programm, das einen Rechner nachbildet
Virus	Ein Programm, das Programme verändert
Debugger	Ein Programm, das ein Programm beobachtet

Nichts davon wäre möglich, wenn Programme und Daten in getrennten Welten lebten.

Und die Konsequenz reicht weiter: *Wenn ein Programm ein Programm verarbeiten kann, kann es auch* ***über*** *Programme sprechen* – **und damit über sich selbst.**

Genau hier liegt die Wurzel des Selbstbezugs, der die ganze Informatik durchzieht (Kapitel 14). **Turings universelle Maschine, Gödels Diagonalisierung und der Quine sind alle Kinder desselben Gedankens:** *Beschreibung und Beschriebenes sind aus demselben Stoff.*

Selbstmodifizierender Code – einst Kunst, heute Gefahr

In den 1950er und 1960er Jahren war es üblich, dass Programme sich selbst umschrieben – *es sparte Speicher, und Indexregister gab es anfangs nicht. Wollte man ein Feld durchlaufen, so* ***addierte man 1 zur Adresse im Ladebefehl selbst****.*

Heute ist das aus drei Gründen verpönt:

Lesbarkeit	Der Code, den man liest, ist nicht der, der läuft
Zwischenspeicher	Der Befehlscache muss aufwendig ungültig gemacht werden
Sicherheit	**Speicherseiten sind entweder beschreibbar *oder* ausführbar**

Die letzte Zeile heißt $W \oplus X$ (write xor execute) *und ist heute Standard* – **sie soll verhindern, dass eingeschleuste Daten als Befehle ausgeführt werden.**

> *Bemerkenswert:* ***Man baut nachträglich die Harvard-Trennung wieder ein****, die von Neumann bewusst aufgegeben hatte – weil sich herausgestellt hat, dass die Verschmelzung von Code und Daten nicht nur mächtig, sondern auch gefährlich ist.*

Ganz gelingt es nicht: Übersetzer, die zur Laufzeit Code erzeugen (JIT), brauchen die Fähigkeit weiterhin – **und sind deshalb ein bevorzugtes Angriffsziel.**

Aufgabe 10.1.

(a) Führen Sie einen vollständigen Befehlszyklus mit allen Registerinhalten durch.

(b) Entwerfen Sie den Befehlssatz einer minimalen Maschine.

(c) Wie viele Befehle braucht eine Maschine mindestens, um universell zu sein?

Lösung Schritt für Schritt.

(a) Ein Befehlszyklus im Detail.

Schritt 1 – Der Speicherinhalt.

Wir nehmen eine einfache Maschine mit 8-Bit-Befehlen: 3 Bit Opcode, 5 Bit Adresse.

Adresse	Inhalt	als Befehl	Bedeutung
0	$(001\,10000)_2$	LOAD 16	ACC ← M[16]
1	$(010\,10001)_2$	ADD 17	ACC ← ACC + M[17]
2	$(011\,10010)_2$	STORE 18	M[18] ← ACC
3	$(111\,00000)_2$	HALT	
16	$(00000101)_2$	—	Datenwert 5
17	$(00000111)_2$	—	Datenwert 7
18	$(00000000)_2$	—	Ergebnisplatz

Man beachte: *Die Adressen 0–3 und 16–18 sehen im Speicher* ***gleich*** *aus. Ob eine Zahl Befehl oder Datum ist, entscheidet allein, ob der Befehlszähler auf sie zeigt.*

Schritt 2 – Der erste Zyklus, Takt für Takt.

Takt	Aktion	PC	IR	ACC
—	Start	0	—	—
1	MAR ← PC = 0	0	—	—
2	MDR ← M[0]	0	—	—
3	IR ← MDR	0	LOAD 16	—
4	PC ← PC+1	**1**	LOAD 16	—
5	Decodieren: Opcode 001	1	LOAD 16	—
6	MAR ← 16	1	LOAD 16	—
7	MDR ← M[16] = 5	1	LOAD 16	—
8	ACC ← MDR	1	LOAD 16	**5**

Schritt 3 – Die weiteren Zyklen, verkürzt.

Zyklus	Befehl	ACC danach	M[18] danach
2	ADD 17	$5 + 7 = \mathbf{12}$	0
3	STORE 18	12	**12**
4	HALT	12	12

Das Programm hat $5 + 7 = 12$ gerechnet und das Ergebnis abgelegt.

Schritt 4 – Die Zählung.
Vier Befehle, je etwa 8 Takte – ***32 Takte*** *für eine Addition.*

> *Ein moderner Prozessor führt in derselben Zeit einige Dutzend Befehle aus.* **Der Unterschied liegt nicht an schnelleren Gattern, sondern an *Fließbandverarbeitung***: *Während Befehl n ausgeführt wird, wird Befehl $n + 1$ schon entschlüsselt und $n + 2$ geholt.*

Band 4 behandelt das – *und die Schwierigkeiten, die entstehen, wenn ein Sprung alle vorgeholten Befehle wertlos macht.*

(b) Ein minimaler Befehlssatz.
Schritt 1 – Was man wirklich braucht.

Gruppe	**Befehle**	**wofür**
Datentransport	LOAD, STORE	Speicher $\leftrightarrow$ Register
Arithmetik	ADD, SUB	Rechnen
Logik	AND, NOT	Bitoperationen
Kontrolle	JMP, JZ	**Verzweigung, Schleife**
Ende	HALT	

Schritt 2 – Warum JZ unentbehrlich ist.

> *Ohne bedingten Sprung läuft ein Programm stur von vorn nach hinten.* **Es kann dann nur Berechnungen fester Länge durchführen** – *und ist damit nichts als ein großes Schaltnetz* (Kapitel 7).

Erst der bedingte Sprung macht Schleifen möglich – *und damit Programme, deren Laufzeit von den Daten abhängt.* **Genau darin liegt der Sprung von der endlichen zur universellen Maschine.**

Und genau daraus folgt auch das Halteproblem (Kapitel 14): **Wo eine Schleife möglich ist, ist auch eine Endlosschleife möglich.**

Schritt 3 – Eine Schleife schreiben.
Aufgabe: Summiere die Zahlen von 1 bis n.

Adr.	Befehl	Bedeutung
0	LOAD #0	Summe $\leftarrow 0$
1	STORE S	
2	LOAD N	**Schleifenbeginn**
3	JZ 10	wenn $N = 0$: fertig
4	ADD S	
5	STORE S	$S \leftarrow S + N$
6	LOAD N	
7	SUB #1	
8	STORE N	$N \leftarrow N - 1$
9	JMP 2	**zurück**
10	HALT	

Ablauf für $N = 3$:

Durchlauf	N	S
Start	3	0
1	2	3
2	1	5
3	0	6
Ende	0	**6**

Und $1 + 2 + 3 = 6$. ✓

(c) Wie wenige Befehle genügen?
Schritt 1 – Die überraschende Antwort.

Ein einziger Befehl genügt. Man nennt solche Maschinen **OISC** (*one instruction set computer*), und sie sind vollwertige Universalrechner.

Schritt 2 – Der Befehl `subleq`.

$$\texttt{subleq}\ a,\ b,\ c$$

1.	$M[b] \leftarrow M[b] - M[a]$
2.	Ist das Ergebnis ≤ 0, so springe nach c; sonst weiter

*Der Name: „**sub**tract and branch if **l**ess than or **eq**ual to zero“.*
Schritt 3 – Alles Übrige nachbauen.
Nulle eine Zelle:

`subleq` a, a, weiter

denn $M[a] - M[a] = 0$. ✓
Unbedingter Sprung:

`subleq` Z, Z, c

mit einer Zelle Z, die 0 enthält – das Ergebnis ist stets 0, also wird stets gesprungen. ✓
Kopieren $M[b] \leftarrow M[a]$***:***

1	`subleq b, b, +1`	$M[b] \leftarrow 0$
2	`subleq a, T, +1`	$T \leftarrow -M[a]$ (nach $T = 0$)
3	`subleq T, b, +1`	$M[b] \leftarrow 0 - (-M[a])$

Drei Befehle für eine Zuweisung. ✓

Addition $M[b] \leftarrow M[b] + M[a]$***: Man negiert*** $M[a]$ *über eine Hilfszelle und subtrahiert.* ✓
Schritt 4 – Der Preis.

Operation	**Befehle**	**Bemerkung**
Zuweisung	3	
Addition	4	
Multiplikation	≈ 40	Schleife
Division	≈ 100	

Die Maschine ist universell, aber grauenhaft langsam – etwa hundertmal langsamer als eine gewöhnliche. **Ihr Wert liegt nicht in der Anwendung, sondern in der Einsicht.**

Schritt 5 – Was man daraus lernt.

Die Grenze zwischen „kann rechnen“ und „kann alles Berechenbare“ ist erstaunlich niedrig. Sie liegt nicht bei einem reichen Befehlssatz, sondern bei zwei Fähigkeiten: **einen Wert verändern** und **abhängig vom Ergebnis springen**.

Genau dasselbe Phänomen begegnet uns wieder:

Ein Gatter	NAND genügt für alle Logik (Kapitel 7)
Ein Befehl	`subleq` genügt für alle Berechnung
Zwei Symbole	0 und 1 genügen für alle Information (Kapitel 1)
Drei Regeln	Lambda-Kalkül genügt für alle Funktionen (Band 6)

*Man nennt dieses Phänomen die **Schwelle der Universalität*** – **und ihre Niedrigkeit ist eine der merkwürdigsten Tatsachen des Fachs.**

Stephen Wolfram hat daraus die These gemacht, *dass Universalität in der Natur eher die Regel als die Ausnahme sei:* **Fast jedes hinreichend nichttriviale System sei berechnungsuniversell.**

*Ein Beleg: Die zellulare **Regel 110** – ein eindimensionaler Automat mit acht Regeln, dessen Zellen nur von ihren beiden Nachbarn abhängen – ist universell.* **Matthew Cook bewies das 2004.**

Auch das Kartenspiel „Magic: The Gathering“ ist es, ebenso wie das Spiel des Lebens von Conway *und – unter geeigneter Formalisierung – die Powerpoint-Animationen.* **Universalität ist billig zu haben; sie zu *vermeiden* ist die schwierigere Aufgabe.**

Und das ist keine Spielerei: **Wo Universalität auftritt, tritt auch Unentscheidbarkeit auf** (Kapitel 14). *Deshalb kann kein Werkzeug allgemein prüfen, ob eine Konfigurationsdatei terminiert – sobald ihre Sprache mächtig genug ist.* von Neumann[26] ist die Originalquelle und erstaunlich gut lesbar; Tanenbaum[24] und Patterson und Hennessy[20] entwickeln die Architektur systematisch; Ceruzzi[3] erzählt die Geschichte.

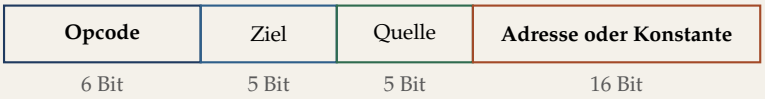
Zuse[28] schildert in seiner Autobiographie, wie er dieselben Grundgedanken unabhängig und ohne Kenntnis der angelsächsischen Arbeiten fasste.

11 Maschinensprache und Assembler

Ein Befehl ist eine Zahl

> *Das ist der Gedanke, auf dem alles Weitere ruht. Ein Befehl steht im selben Speicher wie die Daten, hat dieselbe Wortbreite und besteht aus denselben Bits.* **Was ihn zum Befehl macht, ist ausschließlich, dass das Steuerwerk ihn gerade holt.**

Der Aufbau eines Befehlsworts.
Man teilt die Bits eines Wortes in Felder auf:



Opcode	*operation code* – **was** zu tun ist
Operandenfelder	**womit** es zu tun ist

Ein Beispiel, ganz konkret. *Das Bitmuster*

$$(001000\,00001\,00010\,0000\,0000\,0110\,0100)_2$$

zerlegt sich in

Feld	Wert	Bedeutung
Opcode	$(001000)_2 = 8$	`ADDI` (addiere Konstante)
Ziel	2	Register 2
Quelle	1	Register 1
Konstante	100	

$\Longrightarrow$ `r2 := r1 + 100.`

Dieselben 32 Bit, als Zahl gelesen, ergeben 537 001 060 – **und als Text gelesen vier sinnlose Zeichen.** *Der Speicher kennt den Unterschied nicht.*

11.1 Der Assembler

Die erste Abstraktionsschicht

Niemand schreibt Bitmuster von Hand – man hat es in den 1940er Jahren getan und es gehasst. **Ein Assembler ist ein Programm, das eine Buchstabenschreibweise in Bitmuster übersetzt.**

`ADDI r2, r1, 100`	$(001000\,00001\,00010\,\ldots)_2$

*Die Zuordnung ist **eins zu eins**: Jede Assemblerzeile wird zu genau einem Maschinenbefehl.* **Deshalb ist der Assembler keine Programmiersprache im eigentlichen Sinn, sondern eine *Schreibweise*.**

Was der Assembler zusätzlich leistet:

Namen	`schleife:` statt Adresse $(0040\,1A38)_{16}$
Symbolische Register	`sp`, `ra` statt `r29`, `r31`
Pseudobefehle	`MOVE a,b` wird zu `ADD a,b,r0`
Adressberechnung	Sprungweiten werden ausgerechnet

Der wichtigste Punkt ist der erste. ***Ohne symbolische Namen müsste man bei jeder eingefügten Zeile sämtliche Sprungziele von Hand neu ausrechnen*** – der Grund, warum vor 1950 geschriebene Programme fast nicht änderbar waren.

Kathleen Booth schrieb 1947 den ersten Assembler, Nathaniel Rochester *1954 den ersten mit symbolischen Adressen für die IBM 701.* **Der Begriff „assemble" meint das Zusammensetzen des Programms aus Bruchstücken.**

Adressierungsarten

Woher kommt der Wert, mit dem ein Befehl rechnet? Dafür gibt es mehrere Verfahren, und ihre Zahl ist ein Kennzeichen der Architektur.

Art	Schreibweise	Wert ist
unmittelbar	`ADDI r1, 5`	die Zahl 5 selbst
Register	`ADD r1, r2`	Inhalt von `r2`
direkt	`LOAD r1, 1000`	Speicherzelle 1000
indirekt	`LOAD r1, (r2)`	Zelle, deren Adresse in `r2` steht
indiziert	`LOAD r1, 8(r2)`	Zelle `r2`+8

Die letzten beiden sind die entscheidenden. **Ohne indirekte Adressierung gäbe es keine Zeiger, keine verketteten Listen, keine Bäume – und keine dynamischen Datenstrukturen überhaupt.**

Die indizierte Adressierung ist der Feldzugriff: *`a[i]` bei vier Byte großen Elementen wird zu `LOAD r1, 0(r2)` mit vorher berechnetem `r2 = a + 4*i`.*

Das erklärt eine oft gestellte Frage: **Warum zählen Feldindizes bei null?**

Weil dann die Adresse schlicht „Basis + Index × Elementgröße“ lautet. **Bei Zählung ab eins müsste jeder Zugriff zusätzlich eine Eins abziehen** – *eine Rechnung, die in den 1960er Jahren tatsächlich ins Gewicht fiel.*

EDSGER DIJKSTRA *begründete die Nullzählung 1982 zusätzlich mit einem Eleganzargument: Bei Halbintervallen $[a, b)$ ist die Länge genau $b - a$, es gibt keine Sonderfälle beim leeren Bereich, und aneinandergrenzende Bereiche haben keine Lücken.*

11.2 Sprünge, Schleifen, Unterprogramme

Wie Kontrollfluss entsteht

*Der Befehlszähler zeigt normalerweise auf den nächsten Befehl. Alles, was ein Programm interessant macht, entsteht daraus, dass man ihn **überschreibt**.*

Befehl	Wirkung	Hochsprache
`JMP ziel`	`PC := ziel`	`goto`
`BEQ r1,r2,ziel`	Sprung, falls gleich	`if`
`BLT r1,r2,ziel`	Sprung, falls kleiner	`if`, `while`
`CALL ziel`	Rücksprungadresse merken, springen	Funktionsaufruf
`RET`	zur gemerkten Adresse	`return`

Eine Schleife in Maschinensprache.
Die Hochsprachenfassung

```
summe := 0;  i := 1;  while i <= n do { summe := summe + i;
                                        i := i+1 }
```

wird zu

Marke	Befehl	Kommentar
	`MOVI r1, 0`	summe
	`MOVI r2, 1`	i
`test:`	`BGT r2, r3, ende`	falls i > n: raus
	`ADD r1, r1, r2`	summe += i
	`ADDI r2, r2, 1`	i++
	`JMP test`	
`ende:`	...	

Es gibt in der Maschine keine Schleife. Es gibt nur Vergleiche und Sprünge. Die Schleife ist ein Begriff der Hochsprache, den der Übersetzer in dieses Muster auflöst (Kapitel 12).

Warum `goto` verpönt ist – und was das mit der Maschine zu tun hat

Die Maschine kennt nur `goto`. *Warum gilt der Befehl in Hochsprachen dann als schlechter Stil?*

> DIJKSTRAS *Argument von 1968 („Go To Statement Considered Harmful") ist* ***kein*** *ästhetisches: Bei beliebigen Sprüngen kann man dem Programmtext nicht mehr ansehen, in welchem Zustand das Programm an einer Stelle ist.*

Bei geschachtelten Schleifen und Verzweigungen kann man das. *Man kann für jede Stelle eine Aussage über die Variablen angeben, die dort gilt – eine* ***Schleifeninvariante*** (Kapitel 13).

Der ***Satz von Böhm und Jacopini*** *(1966) zeigt, dass man* ***nie*** *ein `goto` braucht:*
Jedes Programm lässt sich allein mit Hintereinanderausführung, Verzweigung und Schleife schreiben.

Der Preis kann hoch sein – *die Umformung braucht manchmal Hilfsvariablen und macht den Text länger.* **Aber sie ist immer möglich, und das ist der Punkt.**

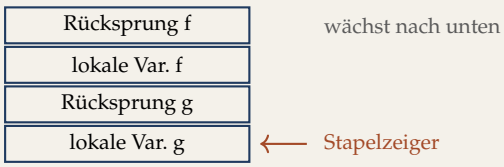
Der Stapel

Ein Unterprogramm muss wissen, wohin es zurückspringen soll. Bei geschachtelten Aufrufen genügt ein Register dafür nicht.

Definition 11.1. Ein **Stapel** (*stack*) ist ein Speicherbereich, auf dem nur zwei Operationen erlaubt sind:

`PUSH x`	Stapelzeiger verschieben, x ablegen
`POP`	oberstes Element holen, Zeiger zurück

Es gilt **last in, first out**: Was zuletzt abgelegt wurde, wird zuerst geholt.



Der Stapel ist genau die Struktur, die zur Schachtelung passt. Ruft f die Funktion g auf und g die Funktion h, so muss h zuerst zurückkehren – und ihr Rücksprungeintrag liegt obenauf.

Und damit ist auch die Rekursion erklärt: *Ruft eine Funktion sich selbst auf, so entsteht einfach ein weiterer Stapelrahmen.* **Die Maschine merkt gar nicht, dass es dieselbe Funktion ist.**

Das ist ein bemerkenswerter Punkt: ***Rekursion braucht keinen eigenen Mechanismus*** **– sie fällt als Nebenprodukt des Stapels an.**

Der Stapelüberlauf und seine Ausnutzung

Der Stapel hat eine feste Größe. *Eine Rekursion ohne Abbruchbedingung füllt ihn und stürzt ab* **– der stack overflow, nach dem eine bekannte Website benannt ist.**

> *Weit gefährlicher ist der* ***Pufferüberlauf****: Auf dem Stapel liegen lokale Variablen* ***und*** *die Rücksprungadresse nebeneinander.* **Wer über das Ende einer lokalen Zeichenkette hinausschreibt, überschreibt die Rücksprungadresse – und bestimmt damit, wohin die Maschine als Nächstes springt.**

Der „Morris-Wurm" legte am 2. November 1988 auf diesem Weg etwa 6000 Rechner lahm *– rund ein Zehntel des damaligen Internets.* **Robert Tappan Morris war Doktorand in Cornell; er wurde als Erster nach dem neuen US-Computerbetrugsgesetz verurteilt und ist heute Professor am MIT.**

Die Schwachstellenklasse ist seit 1988 bekannt und war 2026 noch immer unter den häufigsten. **Gegenmaßnahmen: Stapelschutzwerte, nicht ausführbarer Stapel, Adressraumverwürfelung** *– und Sprachen, die Feldgrenzen prüfen.*

Aufgabe 11.1.

(a) Übersetzen Sie eine Multiplikation durch wiederholte Addition vollständig in Assembler und zählen Sie die Takte.

(b) Verfolgen Sie den Stapel bei einer rekursiven Fakultätsberechnung Schritt für Schritt.

(c) Vergleichen Sie CISC und RISC an einem konkreten Beispiel.

Lösung Schritt für Schritt.

(a) Multiplikation ohne Multiplikationsbefehl.
Schritt 1 – Die einfache Fassung.
Berechne $a \cdot b$ durch b-maliges Addieren von a.

Marke	Befehl	Kommentar
	`MOVI r3, 0`	Ergebnis
`lp:`	`BEQ r2, r0, ende`	b = 0? fertig
	`ADD r3, r3, r1`	Ergebnis += a
	`SUBI r2, r2, 1`	b−−
	`JMP lp`	
`ende:`		

Schritt 2 – Der Aufwand.
Der Schleifenrumpf hat 4 Befehle und läuft b-mal:

$$T(b) \approx 4b + 2 \text{ Befehle.}$$

b	**Befehle**	**bei 1 GHz**
10	42	42 ns
1000	4002	4 µs
10^6	$4 \cdot 10^6$	4 ms

Die Laufzeit hängt vom ***Wert*** *ab, nicht von der Wortbreite* – **das ist inakzeptabel.**

Schritt 3 – Die russische Bauernmultiplikation.

Der bessere Weg beruht auf der Binärdarstellung:

$$a \cdot b = \sum_{i:\ b_i = 1} a \cdot 2^i.$$

Man verdoppelt a **und halbiert** b**; wo** b **ungerade ist, addiert man auf.**

Beispiel $13 \cdot 11$*:*

a	b	b ungerade?	aufaddieren
13	11	**ja**	+13
26	5	**ja**	+26
52	2	nein	—
104	1	**ja**	+104

$$13 + 26 + 104 = 143 = 13 \cdot 11. \qquad \checkmark$$

Schritt 4 – Als Assembler.

Marke	Befehl	Kommentar
	`MOVI r3, 0`	
`lp:`	`BEQ r2, r0, ende`	
	`ANDI r4, r2, 1`	unterstes Bit
	`BEQ r4, r0, weiter`	gerade? überspringen
	`ADD r3, r3, r1`	
`weiter:`	`SLL r1, r1, 1`	a verdoppeln
	`SRL r2, r2, 1`	b halbieren
	`JMP lp`	

Schritt 5 – Der Aufwand jetzt.
Die Schleife läuft so oft, wie b Bits hat, also $\lceil \log_2 b \rceil$-mal:

b	**naiv**	**binär**	**Faktor**
10	42	≈ 28	1,5
1000	4002	≈ 70	57
10^6	$4 \cdot 10^6$	≈ 140	28 600

> ***Von linear auf logarithmisch*** – und das allein durch eine andere Zerlegung derselben Aufgabe. **Das ist der Kern des algorithmischen Denkens** (Kapitel 13).

Das Verfahren ist über 3500 Jahre alt: *Es steht im Papyrus Rhind (um 1650 v. Chr.), wo die ägyptischen Schreiber genau so multiplizierten – durch Verdoppeln und Addieren.* **Dieselbe Idee liegt der schnellen Potenzierung zugrunde, auf der die RSA-Verschlüsselung beruht** (Band 8).

(b) Rekursion und Stapel.
Wir verfolgen `fak(3)` mit

```
fak(n):  if n <= 1 then return 1 else return n * fak(n-1)
```

Schritt 1 – Der Abstieg.

Aufruf	Stapel (oben zuletzt)	Zustand
`fak(3)`	$[n{=}3]$	braucht `fak(2)`
`fak(2)`	$[n{=}3, n{=}2]$	braucht `fak(1)`
`fak(1)`	$[n{=}3, n{=}2, n{=}1]$	**Abbruch: liefert 1**

Schritt 2 – Der Aufstieg.

Rückkehr aus	Stapel danach	Rechnung
`fak(1)` $\to 1$	$[n{=}3, n{=}2]$	$2 \cdot 1 = 2$
`fak(2)` $\to 2$	$[n{=}3]$	$3 \cdot 2 = 6$
`fak(3)` $\to 6$	$[\,]$	**Ergebnis 6**

Der Stapel ist das Gedächtnis der unerledigten Multiplikationen. Bei `fak(3)` muss die Maschine sich merken, dass sie das Ergebnis von `fak(2)` noch mit 3 malnehmen muss – und das steht im Stapelrahmen.

Schritt 3 – Die Tiefe.
`fak(n)` braucht n Stapelrahmen. Bei etwa 32 Byte pro Rahmen und 8 MB Stapel:

$$\frac{8 \cdot 10^6}{32} = 250\,000.$$

Ab etwa $n = 250\,000$ stürzt das Programm ab – **lange bevor die Zahl selbst ein Problem wäre.**

Schritt 4 – Die endrekursive Fassung.

```
fakh(n, acc):  if n <= 1 then return acc else return
                   fakh(n-1, n*acc)
```

*Hier ist der rekursive Aufruf die **letzte** Handlung – danach passiert nichts mehr.* **Also muss man sich auch nichts merken**, *und der Stapelrahmen kann wiederverwendet werden.*

Der Übersetzer verwandelt das in eine Schleife (**Endrekursionsoptimierung**):

```
lp:     BLE r1, 1, ende
        MUL r2, r2, r1      acc *= n
        SUBI r1, r1, 1      n--
        JMP lp              kein CALL!
```

Der Stapel wächst nicht mehr. **In Scheme und ML ist diese Optimierung von der Sprachnorm *vorgeschrieben*;** *in C und Java ist sie es nicht.*

Das ist der Grund, warum in Scheme Schleifen überhaupt keine eigene Sprachkonstruktion brauchen – **man schreibt Endrekursion und erhält eine Schleife** (Abelson und Sussman[1]).

(c) CISC gegen RISC.

	CISC	**RISC**
Befehlszahl	viele hundert	wenige Dutzend
Befehlslänge	**variabel** (1–15 Byte)	**fest** (4 Byte)
Speicherzugriff	in fast jedem Befehl	nur `LOAD/STORE`
Register	wenige (8)	viele (32)
Takte pro Befehl	schwankend	**gleichmäßig**
Beispiel	x86	ARM, RISC-V, MIPS

Ein Vergleich an derselben Aufgabe: *Addiere den Inhalt einer Speicherzelle zu einem Register.*

CISC	`ADD eax, [ebx+4]`	**ein** Befehl
RISC	`LW r5, 4(r6)`	laden
	`ADD r4, r4, r5`	addieren

Die historische Begründung beider Seiten.
Für CISC (1970er): *Speicher war teuer und langsam. Ein Befehl, der viel leistet, spart Speicherplatz und Ladezeit. Außerdem programmierte man noch in Assembler, und mächtige Befehle erleichterten das.*
Für RISC (ab 1980): DAVID PATTERSON *und* JOHN HENNESSY *beobachteten, dass Übersetzer die komplexen Befehle* ***fast nie benutzten.*** **Messungen zeigten: Etwa 20 Prozent der Befehle machten 80 Prozent der Ausführungen aus.**

Die Folgerung: ***Man streiche den Rest und mache die verbleibenden Befehle schnell.*** Gleichmäßig lange Befehle lassen sich in einer Fließbandverarbeitung überlappen – der Grund, warum RISC bei gleicher Technik höher takten kann.

Wie der Streit ausging.

Innen	Moderne x86-Prozessoren zerlegen jeden CISC-Befehl intern in RISC-artige **Mikrooperationen**
Außen	Der Befehlssatz bleibt CISC – wegen der Rückwärtskompatibilität

RISC hat die Architektur gewonnen, CISC den Befehlssatz behalten.

Stand August 2026 ist die Lage verschoben: *ARM beherrscht mobile Geräte vollständig und hat mit Apples M-Reihe und den Serverprozessoren erhebliche Anteile bei Arbeitsplatzrechnern und Rechenzentren gewonnen.* **RISC-V – ein *offener*, lizenzfreier Befehlssatz aus Berkeley (2010) – wächst schnell in eingebetteten Systemen.**

> *Bemerkenswert daran ist weniger die Technik als das Lizenzmodell: Ein Befehlssatz ist eine* ***Schnittstelle****, und Schnittstellen ohne Eigentümer haben sich in der Informatik regelmäßig durchgesetzt* – **wie TCP/IP gegen die Herstellerprotokolle.**

Patterson und Hennessy[20] sind das Standardwerk für Befehlssätze; ihr zweites Buch[9] behandelt die quantitative Bewertung; Bryant und O'Hallaron[2] zeigen den Zusammenhang zur Hochsprache.

12 Der Turm der Abstraktionen

Die zentrale Denkfigur der Informatik

Zwischen einem Elektron in einem Transistor und dem Klick auf eine Schaltfläche liegen etwa zehn Schichten. ***Jede davon benutzt die darunterliegende, ohne zu wissen, wie sie arbeitet.***



Definition 12.1. Eine **Abstraktionsschicht** besteht aus einer **Schnittstelle** – der Menge der Dienste, die sie nach oben anbietet – und einer **Implementierung**, die diese Dienste mit den Mitteln der darunterliegenden Schicht erbringt.

> ***Das Entscheidende ist die Trennung dieser beiden Dinge.*** Wer die Schnittstelle benutzt, darf nichts über die Implementierung wissen müssen – und wer die Implementierung austauscht, darf nichts kaputtmachen.

Warum das überhaupt funktioniert. *Es funktioniert nur, weil jede Schicht ihre Unregelmäßigkeiten* ***verbirgt****:*

Schicht	Was sie verbirgt
Gatter	dass Spannungen in Wahrheit stetig sind
Schaltwerk	dass Signale Laufzeiten haben
Befehlssatz	dass innen ganz anders gerechnet wird
Betriebssystem	dass es nur einen Prozessor gibt
Hochsprache	dass es Register und Adressen gibt

*Jede Zeile ist eine **Lüge**, und jede ist nützlich.* **Das ist keine Nachlässigkeit, sondern die einzige Möglichkeit, Systeme dieser Größe überhaupt zu bauen.**

Die durchlässige Abstraktion

Jede Abstraktion leckt

JOEL SPOLSKY formulierte 2002 das Gesetz der durchlässigen Abstraktionen:

> ***Alle nichttrivialen Abstraktionen sind in gewissem Maße durchlässig.***

Beispiele, die jeder Programmierer kennt:

Abstraktion	Wo sie leckt
„Speicher ist gleich schnell“	Cache: Faktor 100 je nach Zugriffsmuster
„Zahlen rechnen exakt“	Gleitkomma (Kapitel 3)
„Das Netz überträgt zuverlässig“	Zeitüberschreitungen, Paketverlust
„Der Speicher ist unbegrenzt“	Auslagerung, Stapelüberlauf

> ***Die praktische Folgerung ist unangenehm:*** *Man kann eine Schicht benutzen, ohne die darunterliegende zu verstehen – aber man kann sie nicht* ***reparieren****.*

Genau deshalb existiert dieser Band. *Er soll nicht zum Bau von Prozessoren befähigen, sondern dazu, im Fehlerfall eine Schicht tiefer schauen zu können.*

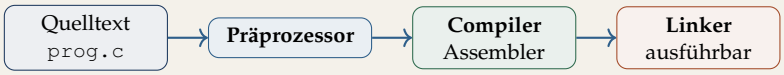
Ein Zahlenbeispiel für die erste Zeile.
Zwei Programme addieren dieselben 1024×1024 *Werte einer Matrix. Das eine geht zeilenweise, das andere spaltenweise.*

Reihenfolge	Zeit	Warum
zeilenweise	1	Daten liegen nebeneinander
spaltenweise	$\approx$ 8–20	jeder Zugriff eine neue Cachezeile

> *Beide Programme sind gleich lang, führen gleich viele Befehle aus und liefern dasselbe Ergebnis.* **Der Unterschied liegt vollständig in einer Schicht, die die Hochsprache verschweigt.**

12.1 Vom Quelltext zum laufenden Programm

Die vier Stationen



Präprozessor	Textersetzung: Einbindungen, Makros
Compiler	Übersetzung in Assembler; hier steckt die Optimierung
Assembler	Assembler in Maschinencode; erzeugt **Objektdatei**
Linker	fügt Objektdateien und Bibliotheken zusammen, löst offene Namen auf
Lader	bringt das Programm beim Start in den Speicher

Was der Compiler im Einzelnen tut.

Lexikalische Analyse	Zeichenfolge → Wortsymbole (*token*)
Syntaxanalyse	Wortsymbole → Baum
Semantische Analyse	Typprüfung, Namensauflösung
Zwischencode	maschinenunabhängige Darstellung
Optimierung	auf dem Zwischencode
Codeerzeugung	Zwischencode → Assembler

*Die Zweiteilung in **Vorderende** (die ersten drei Stufen) und **Hinterende** (die letzten drei) ist der Grund, warum es* `gcc` *für dreißig Sprachen und vierzig Prozessoren gibt* – **man braucht nicht** $30 \times 40 = 1200$ **Übersetzer, sondern** $30 + 40 = 70$ **Bausteine.**

Band 6 der Reihe behandelt den Compilerbau vollständig.

Compiler oder Interpreter

	Compiler	Interpreter
Übersetzt	**einmal, vorher**	bei jeder Ausführung
Geschwindigkeit	schnell	10–100-mal langsamer
Fehlermeldungen	vor dem Start	zur Laufzeit
Ausprobieren	umständlich	sofort
Beispiele	C, Rust, Go	Python, Ruby, Shell

*Der Unterschied ist **keine Eigenschaft der Sprache**, sondern der Implementierung.* **Es gibt C-Interpreter und Python-Compiler.** *Dass man von „Skriptsprachen" spricht, ist eine Gewohnheit, keine Klassifikation.*

Der heute übliche Mittelweg.

Bytecode	Übersetzung in eine *virtuelle* Maschinensprache (Java, Python, C#)
JIT	Zur Laufzeit werden die *heißen* Stellen in echten Maschinencode übersetzt

*Der **JIT-Übersetzer** (just in time) hat einen Vorteil, den kein statischer Compiler hat:* ***Er weiß, welche Werte tatsächlich vorkommen.***

Er kann etwa beobachten, dass eine Variable in der Praxis immer eine Ganzzahl ist, und darauf hin optimieren – mit einer Sicherheitsprüfung, die im Ausnahmefall zurückschaltet. **Deshalb kann JIT-Code in Einzelfällen *schneller* sein als vorübersetzter.**

Aufgabe 12.1.

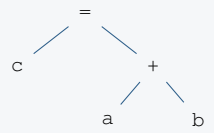
(a) Verfolgen Sie eine Zeile C durch alle Schichten bis zu den Gattern.
(b) Untersuchen Sie, was ein Übersetzer beim Optimieren tatsächlich tut.
(c) Erklären Sie, warum das Bootstrapping eines Compilers kein Zirkelschluss ist.

Lösung Schritt für Schritt.

(a) Eine Zeile durch den ganzen Turm.
Wir verfolgen

```
c = a + b;
```

Schicht 1 – Hochsprache. *Drei Namen, ein Operator. Der Programmierer denkt an Zahlen.*
Schicht 2 – Syntaxbaum.



Der Übersetzer prüft hier: Sind `a` *und* `b` *deklariert? Sind die Typen verträglich?*
Schicht 3 – Assembler.

`LW r1, -8(sp)`	`a` vom Stapel laden
`LW r2, -12(sp)`	`b` laden
`ADD r3, r1, r2`	addieren
`SW r3, -16(sp)`	in `c` zurückschreiben

Aus einer Zeile sind vier Befehle geworden. **Und die Namen sind verschwunden – es gibt nur noch Abstände vom Stapelzeiger.**
Schicht 4 – Maschinencode.

`ADD r3, r1, r2` $\longrightarrow (000000\,00001\,00010\,00011\,00000\,100000)_2$.

Schicht 5 – Befehlszyklus.

Holen	Speicherwort an Adresse `PC` lesen
Dekodieren	Opcode $(000000)_2$, Funktionsfeld $(100000)_2 \rightarrow$ ALU-Operation „addieren“
Ausführen	Registerinhalte an die ALU legen
Rückschreiben	Ergebnis in `r3`

Schicht 6 – Die ALU. *Ein 32-Bit-Addierer, aufgebaut aus 32 Volladdierern* (Kapitel 7).
Schicht 7 – Ein Volladdierer.

$$s = a \oplus b \oplus c_{\text{ein}}, \qquad c_{\text{aus}} = (a \wedge b) \vee (c_{\text{ein}} \wedge (a \oplus b)).$$

Schicht 8 – Gatter. *Ein Volladdierer besteht aus 2 XOR, 2 UND und 1 ODER – oder aus 9 NAND-Gattern.*
Schicht 9 – Transistoren. *Ein NAND braucht in CMOS-Technik 4 Transistoren.*
Die Bilanz.

Ebene	Anzahl	Rechnung
Zeile C	1	
Maschinenbefehle	4	
Volladdierer (nur `ADD`)	32	
NAND-Gatter	≈ 288	$32 \cdot 9$
Transistoren	≈ 1150	$288 \cdot 4$

> ***Über tausend Transistoren schalten, damit eine Zeile `c = a + b;` ausgeführt wird*** – und das ist die **sparsame** Schätzung, denn Laden und Speichern kommen hinzu.

Ein moderner Prozessor hat etwa 10^{10} *Transistoren und führt* 10^{10} *Befehle pro Sekunde aus.* **Die Zahl der Schaltvorgänge pro Sekunde liegt damit jenseits von** 10^{13} *– mehr, als es Sterne in tausend Galaxien gibt.*

(b) Was der Optimierer tut.
Gegeben sei

```
for (i = 0; i < n; i++) { s = s + a[i] * 4; }
```

Optimierung 1 – Stärkereduktion.

`a[i] * 4`	wird zu `a[i] « 2`

Eine Multiplikation kostet 3–5 Takte, ein Schieben einen. ✓
Optimierung 2 – Schleifeninvariante Ausdrücke herausziehen. *Die Adressberechnung `basis + 4*i` wird durch einen fortlaufend erhöhten Zeiger ersetzt* (**Induktionsvariablen**).
Optimierung 3 – Registerzuteilung. *`s` und `i` bleiben in Registern, statt bei jedem Durchlauf in den Speicher geschrieben zu werden.*

> *Das ist die **wirkungsvollste** aller Optimierungen – ein Registerzugriff kostet einen Takt, ein Hauptspeicherzugriff bis zu 300.* **Die zugrundeliegende Aufgabe ist ein Graphenfärbungsproblem** (Mathematik-Band 15, Kapitel 11) **und NP-schwer** – *man benutzt Näherungsverfahren.*

Optimierung 4 – Abrollen. *Vier Durchläufe werden zu einem zusammengefasst; das spart drei Viertel der Sprünge und Vergleiche.*
Optimierung 5 – Vektorisierung. *Vier Additionen werden zu **einem** SIMD-Befehl, der vier Werte gleichzeitig verarbeitet.*
Die Wirkung, gemessen.

Stufe	relative Zeit	Bemerkung
`-O0`	1,00	keine Optimierung
`-O1`	$\approx 0{,}45$	Register, einfache Umformungen
`-O2`	$\approx 0{,}35$	Standardeinstellung
`-O3`	$\approx 0{,}30$	Abrollen, Vektorisierung

Optimierung ändert das Verhalten – manchmal sichtbar

Der Übersetzer darf umformen, solange das *beobachtbare* Verhalten gleich bleibt (As-if-Regel).

Was „beobachtbar" heißt, legt die Sprachnorm fest – und sie ist großzügiger, als viele erwarten.

Drei berüchtigte Fälle:

Zeitmessung	Eine leere Schleife wird vollständig entfernt
Gleitkomma	$(a+b)+c \neq a+(b+c)$ (Kapitel 3); `-ffast-math` ignoriert das
Schlüssel löschen	Ein `memset` auf einen Puffer, der danach nicht mehr gelesen wird, gilt als *toter Code* und wird gestrichen

Der dritte Fall ist eine reale Sicherheitslücke: Kryptographische Schlüssel bleiben im Speicher stehen, obwohl der Programmierer sie ausdrücklich überschrieben hat. **Deshalb gibt es die Sonderfunktion `memset_s`, die nicht wegoptimiert werden darf.**

(c) Bootstrapping – der erste seltsame Kreis.

Die Frage: *Der C-Übersetzer ist selbst in C geschrieben. Wer hat ihn dann übersetzt?*

Schritt 1 – Warum das kein Zirkelschluss ist.
*Ein Zirkelschluss läge vor, wenn man den Übersetzer **brauchte**, um ihn zu erzeugen. Tatsächlich braucht man ihn nur, um ihn **bequem** zu erzeugen.*

Schritt 2 – Das Verfahren.

1. Man schreibt einen **sehr kleinen** Übersetzer für eine **Teilmenge** von C – in Assembler, von Hand.
2. Mit ihm übersetzt man einen etwas größeren Übersetzer, der in dieser Teilmenge geschrieben ist.
3. Mit diesem übersetzt man einen noch größeren.
4. Nach wenigen Runden hat man den vollen Übersetzer.

Man zieht sich buchstäblich an den eigenen Stiefelschlaufen hoch – **daher der Name Bootstrapping**, *nach den Lügengeschichten des Barons Münchhausen.*

Schritt 3 – Der historische Ablauf.

1969	THOMPSON schreibt B in Assembler
1971	B wird in B umgeschrieben
1972	RITCHIE entwickelt C aus B
1973	**Unix wird in C neu geschrieben** – das erste Betriebssystem in einer Hochsprache

Schritt 4 – Die Selbstübersetzungsprobe.
Ein Übersetzer, der sich selbst übersetzen kann, erlaubt eine bemerkenswerte Prüfung:

Lauf 1	Alter Übersetzer übersetzt den neuen Quelltext → Version A
Lauf 2	Version A übersetzt denselben Quelltext → Version B
Lauf 3	Version B übersetzt denselben Quelltext → Version C

B und C müssen Bit für Bit identisch sein. Denn beide wurden von einem Programm erzeugt, das aus demselben Quelltext hervorging und dieselben Optimierungen enthält. **Weicht etwas ab, liegt ein Fehler vor.**

Dieser Dreistufentest („bootstrap comparison“) gehört bei GCC seit Jahrzehnten zum Standardablauf des Übersetzens. **Er ist eine Selbstprüfung, die nur möglich ist, weil das Programm auf sich selbst anwendbar ist.**

Thompsons Hintertür – und warum der Test sie nicht findet

KEN THOMPSON beschrieb 1984 in seiner Turing-Award-Rede „Reflections on Trusting Trust" einen Angriff, der bis heute nicht restlos abwehrbar ist. *Man baue in den Übersetzer zwei Dinge ein:*

(1)	Erkennt er, dass er das Anmeldeprogramm übersetzt, so fügt er eine Hintertür ein.
(2)	Erkennt er, dass er **sich selbst** übersetzt, so fügt er **beide Regeln** wieder ein.

> *Jetzt entferne man beide Regeln aus dem* ***Quelltext*** *des Übersetzers.* **Der Quelltext ist sauber – und die Hintertür bleibt trotzdem in jedem erzeugten Programm.** *Denn der* binäre *Übersetzer fügt sie bei jeder Selbstübersetzung wieder ein.*

Der Dreistufentest schlägt nicht an, *weil B und C weiterhin identisch sind – beide enthalten die Hintertür.*

Thompsons Schlusssatz: **Man kann keinem Programm trauen, das man nicht vollständig selbst geschrieben hat** – *und das schließt den Übersetzer ein.*

Die praktische Gegenmaßnahme fand DAVID A. WHEELER 2005, *mit einem Verfahren namens* ***Diverse Double-Compiling****: Man übersetzt denselben Quelltext zusätzlich mit einem* ***zweiten, unabhängigen*** *Übersetzer und vergleicht die Ergebnisse.* **Beide müssten dieselbe Hintertür kennen, damit der Angriff unbemerkt bleibt.**

> *Dieser Angriff ist der erste vollständig ausgeführte* ***seltsame Kreis*** *dieses Bandes: Ein Programm, das sein eigenes Verhalten in seinen Nachkommen fortpflanzt, ohne dass es in seiner Beschreibung steht.* **Kapitel 14 nimmt den Faden auf.**

Tanenbaum[24] ist das klassische Werk über den Schichtenaufbau; Nisan und Schocken[18] lassen den Leser sämtliche Schichten selbst bauen – vom NAND-Gatter bis zum Übersetzer; Kernighan und Ritchie[13] zeigen die Sprache, mit der Unix aus dem Assembler herauswuchs.

Part V

Das Programm

13 Algorithmisches Denken

Was ein Algorithmus ist

Definition 13.1. Ein **Algorithmus** ist eine **endliche**, **eindeutige** Vorschrift, die aus einer Eingabe in **endlich vielen** Schritten eine Ausgabe erzeugt.

endlich beschrieben	Die Vorschrift selbst passt auf endlich viel Papier
eindeutig	In jedem Schritt steht fest, was als Nächstes geschieht
terminierend	Das Verfahren hört auf
allgemein	Es gilt für eine ganze Klasse von Eingaben

Die dritte Bedingung ist die heikelste

Woher weiß man, dass ein Verfahren aufhört?
Bei einer Schleife `while i <= n` *mit wachsendem* `i` *ist es klar.* **Bei der folgenden Vorschrift nicht:**

Ist n gerade: $n := n/2$

Ist n ungerade: $n := 3n + 1$

Wiederhole, bis $n = 1$ **Hört das immer auf?**

Für $n = 27$ dauert es 111 Schritte, und zwischendurch wird n auf 9232 hochgetrieben.

> *Das ist die **Collatz-Vermutung**. Sie wurde 1937 aufgestellt, für alle Startwerte bis etwa 2^{68} nachgerechnet* – **und ist unbewiesen.** PAUL ERDŐS *sagte dazu: „Die Mathematik ist für solche Probleme noch nicht reif."*

Kapitel 14 zeigt, dass die Schwierigkeit grundsätzlich ist: *Es gibt kein Verfahren, das für beliebige Programme entscheidet, ob sie terminieren.*

Die Herkunft des Wortes. *Es geht auf* MUHAMMAD IBN MUSA AL-CHWARIZMI *zurück (etwa 780–850), Gelehrter am Haus der Weisheit in Bagdad.* **Sein Name wurde im Lateinischen zu „Algorismi“ und bezeichnete zunächst das Rechnen mit indischen Ziffern.**

Aus dem Titel seines zweiten Buches – al-dschabr – *wurde übrigens das Wort „Algebra“.*
Zwei Grundbegriffe zweier Fächer aus einer Person.

13.1 Korrektheit

Die Schleifeninvariante

Ein Programm zu testen zeigt, dass es in den geprüften Fällen richtig läuft. **Ein Beweis zeigt, dass es in *allen* Fällen richtig läuft.** *Das Werkzeug dafür ist die Invariante.*

Definition 13.2. Eine **Schleifeninvariante** ist eine Aussage über die Variablen, die

- vor dem ersten Durchlauf gilt,
- bei jedem Durchlauf erhalten bleibt,
- zusammen mit der Abbruchbedingung das Gewünschte liefert.

Das sind genau die drei Teile eines Induktionsbeweises (Mathematik-Band 0, Kapitel 14)
– Anfang, Schritt, Schluss.

Ein vollständiges Beispiel.
Behauptung: Das folgende Programm berechnet $\sum_{k=1}^{n} k$.

```
s := 0;  i := 1;  while i <= n do { s := s + i;  i := i + 1
                                  }
```

Die Invariante:

$$\text{(I)} \quad s = \sum_{k=1}^{i-1} k \quad \text{und} \quad 1 \le i \le n+1$$

Beweis in drei Teilen.
Anfang. Vor dem ersten Durchlauf ist $s = 0$ und $i = 1$. Die Summe $\sum_{k=1}^{0}$ ist leer, also 0. ✓
Schritt. Gelte (I) zu Beginn eines Durchlaufs, und sei $i \le n$. Nach den beiden Zuweisungen ist

$$s_{\text{neu}} = s + i = \sum_{k=1}^{i-1} k + i = \sum_{k=1}^{i} k, \qquad i_{\text{neu}} = i + 1.$$

Also gilt $s_{neu} = \sum_{k=1}^{i_{neu}-1} k$ **– genau (I).** ✓

Schluss. Die Schleife endet, wenn $i > n$; zusammen mit $i \le n+1$ folgt $i = n+1$. Einsetzen in (I):

$$s = \sum_{k=1}^{n} k. \qquad \checkmark$$

> ***Damit ist bewiesen, dass das Programm für jedes*** $n \ge 0$ ***das Richtige liefert*** – ohne einen einzigen Testlauf.

Die Terminierung ist ein *zweiter* Beweis.

Man gibt eine ***Terminierungsfunktion*** *an – eine natürliche Zahl, die bei jedem Durchlauf echt kleiner wird.*

$$t := n - i + 1.$$

Sie fällt bei jedem Durchlauf um 1 und ist stets ≥ 0*, solange die Schleife läuft.* **Eine fallende Folge natürlicher Zahlen ist endlich** (Mathematik-Band 10, Wohlordnung). ✓

> ***Korrektheit und Terminierung sind getrennte Fragen.*** Ein Programm kann richtig rechnen und nie fertig werden – man nennt das **partielle Korrektheit**. Beides zusammen heißt **totale Korrektheit**.

C. A. R. HOARE *baute daraus 1969 einen vollständigen Kalkül* (**Hoare-Logik**): **Man schreibt Aussagen zwischen die Programmzeilen und leitet die Korrektheit formal ab.**

13.2 Aufwand

Warum man Konstanten weglässt

Die Laufzeit eines Programms hängt vom Rechner ab, vom Übersetzer, vom Cache. Diese Größen ändern sich alle paar Jahre.

> *Was sich* ***nicht*** *ändert, ist,* ***wie die Laufzeit mit der Eingabegröße wächst.*** **Genau das misst die** O**-Notation.**

Definition 13.3. $f(n) = O(g(n))$ bedeutet: Es gibt Konstanten $c > 0$ und n_0 mit

$$f(n) \le c \cdot g(n) \qquad \text{für alle } n \ge n_0.$$

In Worten: f ***wächst höchstens so schnell wie*** g*, bis auf einen konstanten Faktor und für hinreichend große* n*.*

Klasse	Name	Beispiel
$O(1)$	konstant	Feldzugriff
$O(\log n)$	logarithmisch	**binäre Suche**
$O(n)$	linear	Maximum finden
$O(n \log n)$	—	**gutes Sortieren**
$O(n^2)$	quadratisch	einfaches Sortieren
$O(n^3)$	kubisch	Matrixmultiplikation (naiv)
$O(2^n)$	exponentiell	alle Teilmengen
$O(n!)$	faktoriell	alle Anordnungen

Warum das kein akademisches Spiel ist.
Ein Rechner schaffe 10^9 *Grundschritte je Sekunde:*

n	$n \log_2 n$	n^2	2^n
10	33 ns	100 ns	1 µs
100	664 ns	10 µs	$4 \cdot 10^{13}$ Jahre
10^4	133 µs	100 ms	—
10^6	20 ms	**17 Minuten**	—
10^9	30 s	**31 Jahre**	—

Bei $n = 10^6$ ***trennt ein Faktor 50 000 die beiden Verfahren.*** Und dieser Faktor wächst mit n weiter – kein schnellerer Rechner holt ihn ein.

Konstanten sind nicht immer egal

Die O-Notation verschweigt Konstanten – und manchmal sind sie entscheidend.

Verfahren	Aufwand	praktisch
Einfügesortieren	$O(n^2)$	schnell für $n < 20$
Quicksort	$O(n \log n)$	Standard
Coppersmith-Winograd	$O(n^{2{,}376})$	nie eingesetzt

Die letzte Zeile ist berüchtigt: Die versteckte Konstante ist so groß, dass das Verfahren erst bei Matrizen jenseits jeder vorstellbaren Größe gewinnt. **Man nennt solche Verfahren galaktische Algorithmen.**

*Und die erste Zeile ist der Grund, warum praktische Sortierbibliotheken **umschalten**: Quicksort für große Bereiche, Einfügesortieren für die kleinen Reste.*

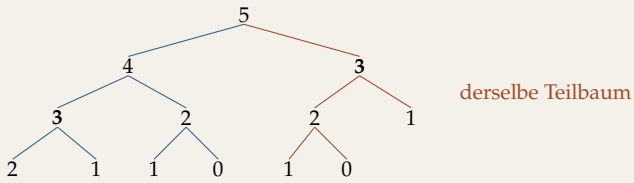
Die drei großen Entwurfsmuster

Muster	Idee	Beispiel
Teile und herrsche	Aufteilen, lösen, zusammensetzen	Mergesort, binäre Suche
Gierig	In jedem Schritt das lokal Beste	Wechselgeld, Dijkstra
Dynamisch	Teilergebnisse speichern statt neu rechnen	Fibonacci, Editierabstand

Zum dritten Muster, weil es das lehrreichste ist.
Die naive rekursive Fibonaccirechnung

```
fib(n) = fib(n-1) + fib(n-2)
```

berechnet `fib(5)` *so:*



*Der Teilbaum für 3 wird **zweimal** vollständig berechnet, der für 2 dreimal.*

Der Aufwand.

n	**naiv**	**mit Speicherung**
10	177 Aufrufe	19
30	2,7 Mio.	59
50	$4{,}0 \cdot 10^{10}$	99
100	10^{21}	199

Von exponentiell auf linear – durch das bloße Aufschreiben bereits berechneter Werte. Man nennt das **Memoisierung**, und es kostet drei Zeilen Code.

Der Aufwand der naiven Fassung ist tatsächlich $\approx \varphi^n$ mit dem goldenen Schnitt $\varphi = 1{,}618$ (Mathematik-Band 15, Kapitel 5) – **denn die Zahl der Aufrufe folgt selbst der Fibonaccirekursion.**

Aufgabe 13.1.

(a) Beweisen Sie die Korrektheit der binären Suche vollständig und bestimmen Sie ihren Aufwand.

(b) Zeigen Sie die untere Schranke $\Omega(n \log n)$ für vergleichsbasiertes Sortieren.

(c) Untersuchen Sie den euklidischen Algorithmus auf Korrektheit, Terminierung und Aufwand.

Lösung Schritt für Schritt.

(a) Binäre Suche.
*Gegeben: ein **sortiertes** Feld $a[0 \ldots n-1]$ und ein Wert x. Gesucht: ein Index mit $a[i] = x$, oder die Meldung „nicht vorhanden“.*
Das Verfahren.

```
lo := 0;  hi := n-1;
while lo <= hi do {
  m := (lo + hi) div 2;
  if a[m] = x then return m
  else if a[m] < x then lo := m+1
  else hi := m-1 }
return "nicht vorhanden"
```

Schritt 1 – Die Invariante.

(I) Kommt x im Feld vor, so liegt sein Index in $[lo,\ hi]$.

Schritt 2 – Anfang. $[0, n-1]$ *ist das ganze Feld.* ✓
Schritt 3 – Schritt.
Sei (I) gültig und $a[m] < x$*. Da das Feld sortiert ist, gilt*

$$a[j] \leq a[m] < x \qquad \text{für alle } j \leq m.$$

Also kann x *an keiner Stelle* $\leq m$ *stehen* – **das neue Intervall** $[m+1,\ hi]$ **enthält weiterhin alle Kandidaten.** ✓

Der Fall $a[m] > x$ *verläuft spiegelbildlich.* ✓
Schritt 4 – Schluss.
Die Schleife endet, wenn $lo > hi$ *– das Intervall ist leer. Nach (I) kommt* x *dann nicht vor.* ✓
Schritt 5 – Terminierung.
Terminierungsfunktion: $t := hi - lo + 1$*, die Intervalllänge.*
In jedem Durchlauf wird entweder zurückgekehrt oder das Intervall echt verkleinert:

Fall	**neues Intervall**	**neue Länge**
$a[m] < x$	$[m+1, hi]$	$\leq \lfloor t/2 \rfloor$
$a[m] > x$	$[lo, m-1]$	$\leq \lceil t/2 \rceil - 1$

In beiden Fällen wird t *mindestens halbiert* – **und da** t **ganzzahlig und** ≥ 0 **ist, endet die Folge.** ✓

Schritt 6 – Der Aufwand.
Nach k *Durchläufen ist die Intervalllänge höchstens* $n/2^k$*. Die Schleife endet, wenn sie unter 1 fällt:*

$$\frac{n}{2^k} < 1 \iff k > \log_2 n.$$

$$\boxed{T(n) = O(\log n)}$$

n	**Vergleiche**	**linear wären**
10^3	10	1000
10^6	20	10^6
10^9	30	10^9

*Eine Milliarde Einträge in **dreißig** Vergleichen.* **Das ist der Grund, warum Datenbanken sortierte Indexstrukturen führen** (Band 10).

Der Fehler, der zwanzig Jahre unentdeckt blieb

Die Zeile `m := (lo + hi) div 2` ist falsch.
Bei großen Feldern kann `lo + hi` den Zahlenbereich überschreiten (Kapitel 2) – **und liefert dann eine negative Zahl.**

Bei 32-Bit-Indizes tritt das ab etwa $n = 2^{30} \approx 10^9$ Elementen auf.

Richtig ist

```
m := lo + (hi - lo) div 2,
```

denn $hi - lo$ ist stets $\leq n$ und läuft nicht über.

> JOSHUA BLOCH *beschrieb 2006, dass genau dieser Fehler in der Java-Standardbibliothek stand – und im Programmbeispiel des Buches, aus dem Generationen ihn abgeschrieben hatten.* **Er stand seit 1986 in JON BENTLEYS „Programming Pearls“, dessen Autor die Korrektheit dort ausdrücklich *bewiesen* hatte.**

Warum der Beweis den Fehler nicht fand: *Er wurde über den **ganzen Zahlen** geführt, nicht über den Maschinenzahlen.* **Die Invariante stimmt – die Implementierung hält sie nur nicht ein.**

Ein lehrreicher Fall: Ein korrekter Beweis über dem falschen Modell schützt vor nichts.

(b) Die untere Schranke des Sortierens.

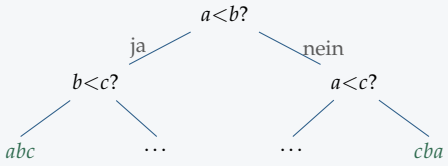
Satz 13.4. *Jedes Sortierverfahren, das Elemente nur **vergleicht**, braucht im schlechtesten Fall mindestens*

$$\lceil \log_2(n!) \rceil = \Omega(n \log n)$$

Vergleiche.

Schritt 1 – Der Entscheidungsbaum.

Ein vergleichsbasiertes Verfahren lässt sich als Baum darstellen: Jeder innere Knoten ist ein Vergleich, jeder Ast eine mögliche Antwort, jedes Blatt eine erzeugte Anordnung.



Schritt 2 – Wie viele Blätter braucht der Baum?
*Das Verfahren muss **jede** der $n!$ möglichen Anordnungen erzeugen können – sonst gäbe es eine Eingabe, die es nicht sortiert.*

$$\text{Blätter} \;\geq\; n!.$$

Schritt 3 – Wie hoch muss er sein?
Ein Binärbaum der Höhe h hat höchstens 2^h Blätter (Mathematik-Band 15, Kapitel 10).

$$2^h \geq n! \Longrightarrow h \geq \log_2(n!).$$

Schritt 4 – Die Größenordnung.
Mit der Stirlingformel (Mathematik-Band 15, Kapitel 2)

$$n! \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$$

folgt

$$\log_2(n!) \approx n\log_2 n - n\log_2 \mathrm{e} + \tfrac{1}{2}\log_2(2\pi n) \approx n\log_2 n - 1{,}44\,n.$$

$$\boxed{h = \Omega(n \log n)}$$

Schritt 5 – Zahlen.

n	$\log_2(n!)$	Mergesort	Verhältnis
10	22	34	1,5
100	525	665	1,3
1000	8530	9966	1,2

Mergesort ist also fast optimal – es liegt bei großen n nur noch etwa 20 Prozent über der theoretischen Schranke.

Schritt 6 – Und wie umgeht man die Schranke?

*Indem man **nicht vergleicht**.*

Zählsortieren *und* ***Radixsortieren*** *schauen sich die Werte selbst an – etwa ihre Ziffern – statt sie miteinander zu vergleichen.*

Verfahren	**Aufwand**	**Voraussetzung**
Vergleichsbasiert	$\Omega(n \log n)$	nur Ordnung nötig
Zählsortieren	$O(n + k)$	Werte aus $\{0, \ldots, k\}$
Radixsortieren	$O(d \cdot n)$	feste Stellenzahl d

> *Das ist kein Widerspruch zum Satz –* ***die Voraussetzung ist eine andere.*** Der Satz sagt nur, was mit Vergleichen allein möglich ist. **Wer mehr über die Daten weiß, kann mehr.**

Untere Schranken sind in der Informatik selten und kostbar. *Diese hier ist eine der wenigen, die man leicht beweisen kann – die meisten wichtigen (etwa für* P $\neq$ NP*) sind offen* (Band 3).

(c) Der euklidische Algorithmus.
Der älteste nichttriviale Algorithmus der Welt – aus Euklids Elementen*, Buch VII, um 300 v. Chr.*

```
ggT(a, b):   while b <> 0 do { (a, b) := (b, a mod b) };
                          return a
```

Schritt 1 – Ein Beispiel.

a	b	$a \bmod b$
1071	462	147
462	147	21
147	21	0
21	0	**fertig**

$$\mathrm{ggT}(1071, 462) = 21.$$

Probe: $1071 = 21 \cdot 51$, $462 = 21 \cdot 22$, und $\mathrm{ggT}(51, 22) = 1$. ✓
Schritt 2 – Korrektheit.
Die Invariante:

$$\boxed{\mathrm{ggT}(a, b) = \mathrm{ggT}(a_0, b_0) \text{ bleibt über alle Durchläufe erhalten.}}$$

Zu zeigen ist also: $\mathrm{ggT}(a, b) = \mathrm{ggT}(b,\ a \bmod b)$.
Sei $a = qb + r$ *mit* $r = a \bmod b$.

***Ist** d **ein Teiler von** a **und** b, so teilt d auch $r = a - qb$* – **denn beide Summanden sind durch** d **teilbar.** ✓
***Ist umgekehrt** d **ein Teiler von** b **und** r, so teilt d auch $a = qb + r$.* ✓

*Die beiden Paare haben also **dieselben gemeinsamen Teiler** –* **und damit denselben größten.** ✓

Beim Abbruch ist $b = 0$, und $\mathrm{ggT}(a, 0) = a$. ✓
Schritt 3 – Terminierung.
Terminierungsfunktion: b selbst. Da $a \bmod b < b$, wird b in jedem Durchlauf echt kleiner und bleibt ≥ 0. ✓
Schritt 4 – Der Aufwand.

Wie schnell schrumpft b? Die Antwort ist überraschend genau.

Satz 13.5 (Lamé, 1844)**.** *Der euklidische Algorithmus braucht für Zahlen bis N höchstens*

$$\approx \frac{\ln N}{\ln \varphi} \approx 4{,}785 \log_{10} N + 1{,}67$$

Schritte, wobei $\varphi = \frac{1+\sqrt{5}}{2}$ der goldene Schnitt ist.

> ***Der schlechteste Fall tritt bei aufeinanderfolgenden Fibonaccizahlen ein*** – denn dort ist der Quotient q stets 1, und b schrumpft so langsam wie möglich.

Beispiel $\mathrm{ggT}(89, 55)$*:*

$$(89, 55) \to (55, 34) \to (34, 21) \to (21, 13) \to (13, 8)$$

$$\to (8, 5) \to (5, 3) \to (3, 2) \to (2, 1) \to (1, 0).$$

Neun Schritte für zweistellige Zahlen – und die Zwischenwerte sind lauter Fibonaccizahlen. ✓

Stellen von N	**max. Schritte**	**Bemerkung**
10	≈ 49	
100	≈ 480	
1000	≈ 4790	RSA-Größenordnung

> ***Der Aufwand ist logarithmisch in den Zahlen, also linear in ihrer Stellenzahl.*** Deshalb ist der Algorithmus nach 2300 Jahren immer noch der praktisch benutzte – er steckt in jeder RSA-Implementierung (Band 8).

Lamés Satz von 1844 gilt als das erste Ergebnis der Komplexitätstheorie überhaupt – *eine Aussage über die Laufzeit eines Algorithmus, hundert Jahre bevor es Rechner gab.* Knuth[14] widmet dem euklidischen Algorithmus über zwanzig Seiten; Harel und Feldman[8] sowie Dewdney[5] führen das algorithmische Denken anschaulich ein; Wirth[27] verbindet es mit der Programmierpraxis.

14 Der Selbstbezug

Der Gedanke, der alles trägt

Wir haben in Kapitel 10 gesehen: ***Befehle und Daten stehen im selben Speicher.*** *Ein Programm ist also selbst ein Datum – eine Folge von Bits, eine Zahl.*

Daraus folgt unmittelbar:

Ein Programm kann Daten verarbeiten	(offensichtlich)
Ein Programm **ist** ein Datum	(weil es im Speicher steht)
Also kann ein Programm ein Programm verarbeiten	Übersetzer, Interpreter
Also kann ein Programm *sich selbst* verarbeiten	**hier wird es interessant**

Die ersten drei Zeilen sind Handwerk und stehen in Kapitel 12. ***Die vierte ist der Punkt, an dem die Informatik aufhört, eine Ingenieurwissenschaft zu sein.***

Dieses Kapitel geht drei Stufen des Selbstbezugs durch:

1.	**Der Quine**	Ein Programm druckt sich selbst
2.	**Die Universalmaschine**	Ein Programm führt jedes Programm aus
3.	**Die Diagonale**	Ein Programm, das es nicht geben kann

Die dritte Stufe wird in Band 2 vollständig ausgeführt; hier zeige ich nur, wie man dorthin kommt – **und dass der Weg kurz ist.**

14.1 Ein Programm, das sich selbst schreibt

Warum das schwierig ist

> *Ein Programm soll seinen eigenen Quelltext ausgeben – ohne ihn aus einer Datei zu lesen.* **Solche Programme heißen Quine, nach dem Logiker WILLARD VAN ORMAN QUINE.**

Der naive Versuch scheitert sofort.
Man schreibt

```
print("Hallo")
```

und will, dass die Ausgabe der Quelltext ist. Also

```
print("print(...)")
```

– aber dann muss in den Anführungszeichen wieder das ganze Programm stehen, also auch die Anführungszeichen, also…

> ***Man gerät in einen unendlichen Regress.*** Jeder Versuch, den Text wörtlich einzubauen, macht den Text länger – und der Zuwachs muss wieder eingebaut werden.

Die Lösung besteht darin, den Text *zweimal* zu verwenden.

einmal	als **Anweisung**, die etwas tut
einmal	als **Zeichenkette**, die beschrieben wird

Und der Kunstgriff lautet: ***Die Zeichenkette beschreibt die Anweisung, und die Anweisung druckt die Zeichenkette – zweimal, einmal roh und einmal in Anführungszeichen.***

Die Grundidee an einem Satz der Umgangssprache

> *Ein Satz, der sich selbst hervorbringt – QUINES berühmtes Beispiel:*
>
> **„ergibt einen falschen Satz, wenn man es seinem eigenen Zitat nachstellt“ ergibt einen falschen Satz, wenn man es seinem eigenen Zitat nachstellt.**

Man betrachte den Aufbau.
Der Satz besteht aus zwei Teilen:

Teil A	ein Text **in Anführungszeichen** – also *erwähnt*
Teil B	derselbe Text **ohne** Anführungszeichen – also *gebraucht*

Teil B ist eine Anweisung: „Nimm den Text, der links steht, und stelle ihn seinem eigenen Zitat nach.“ **Führt man sie aus, erhält man genau den Satz zurück, der dasteht.**

Der Unterschied zwischen ***Gebrauch*** *und* ***Erwähnung*** *– zwischen einem Wort benutzen und über ein Wort reden – ist der ganze Mechanismus.* **Im Programm entspricht ihm der Unterschied zwischen Code und Zeichenkette.**

Und das Behauptete ist überdies wahr: Der Satz ist tatsächlich falsch **– eine Fassung des Lügnerparadoxons, aus der GÖDEL 1931 seinen Unvollständigkeitssatz baute.**

Warum das nicht paradox ist – im Programm

Bei Quines Satz entsteht ein Widerspruch, weil er über *Wahrheit* redet.

Ein Quine-Programm redet nur über ***Text****.* **Es behauptet nichts, es druckt.** *Deshalb ist es völlig harmlos – es existiert einfach.*

Der Widerspruch kehrt zurück, sobald das Programm anfängt, über ***sein eigenes Verhalten*** *zu reden* **– etwa: „Ich halte genau dann an, wenn ich nicht anhalte.“** *Genau daraus wird in Abschnitt 3 der Beweis.*

14.2 Die universelle Maschine

Der Interpreter als Universalprogramm

Ein ***Interpreter*** *ist ein Programm U, das zwei Eingaben nimmt: die Beschreibung eines Programms P und dessen Eingabe x. Es tut dann genau das, was P bei Eingabe x täte.*

$$\boxed{U(\langle P\rangle,\, x) = P(x)}$$

Dabei bezeichnet $\langle P\rangle$ die Kodierung von P als Datum **– eine Zahl, eine Bitfolge, ein Text.**

Dass es ein solches U gibt, ist keine Selbstverständlichkeit **– es ist TURINGS Satz von der universellen Maschine (1936).**

*Vor Turing war eine Rechenmaschine ein Gerät für **eine** Aufgabe: Babbages Difference Engine berechnete Polynomtafeln, die Hollerith-Maschine zählte Volkszählungskarten.* **Turing zeigte, dass *eine* Maschine genügt, wenn man ihr die Beschreibung der anderen als Eingabe gibt.**

vor 1936	Eine Maschine, eine Aufgabe
nach 1936	**Eine Maschine, alle Aufgaben** – die Aufgabe ist Eingabe

Das ist die theoretische Rechtfertigung des Computers als Universalgerät – *und der Grund, warum dasselbe Blech Texte schreibt, Musik abspielt und Wetter vorhersagt.*

Von Neumanns Architektur (Kapitel 10) *ist die technische Umsetzung dieses Satzes.* **Von Neumann kannte Turings Arbeit; er hatte ihn 1936–38 in Princeton betreut und ihm eine Assistentenstelle angeboten.**

Der Selbstbezug wird konstruktiv

Sobald es einen Interpreter gibt, kann man ihn auf sich selbst anwenden.

$$U(\langle U\rangle,\ (\langle P\rangle, x)) = U(\langle P\rangle, x) = P(x).$$

Der Interpreter interpretiert sich selbst, während er ein anderes Programm interpretiert. Das ist kein Kunststück, sondern täglich benutzte Technik: Ein in Python geschriebener Python-Interpreter läuft in einem Python-Interpreter.

In der Lisp-Welt heißt ein solcher Interpreter Meta-Zirkularer Interpreter, *und er ist bemerkenswert kurz:* John McCarthy *schrieb 1960 einen Lisp-Interpreter für Lisp auf einer halben Seite.*

Alan Kay *nannte diese halbe Seite später „Maxwells Gleichungen der Informatik“* – **eine kurze Formel, aus der sich die ganze Sprache ergibt.**

Der Zirkel ist keiner

Man könnte einwenden: Wenn der Lisp-Interpreter in Lisp geschrieben ist, was führt ihn dann aus?

Dieselbe Antwort wie beim Bootstrapping (Kapitel 12): **Irgendwo unten steht ein Interpreter, der *nicht* in Lisp geschrieben ist** – *ursprünglich in Maschinencode, von Hand.*

*Der meta-zirkulare Interpreter **erklärt** die Sprache, er **begründet** sie nicht.* **Er sagt: „Wenn du schon Lisp verstehst, dann verstehst du damit auch Lisp."** *Das ist nützlich und trotzdem kein Fundament.*

14.3 Die Grenze

Ein Programm, das es nicht geben kann

*Wir haben gesehen, wie viel der Selbstbezug ermöglicht. Jetzt zeigt sich, dass er zugleich eine **Grenze** setzt* – **und zwar mit demselben Kunstgriff.**

Definition 14.1. Das **Halteproblem** lautet: Gibt es ein Programm H, das für jedes Paar $(\langle P \rangle, x)$ entscheidet, ob P bei Eingabe x jemals anhält?

Satz 14.2 (Turing, 1936)**.**

Ein solches H gibt es nicht.

Der Beweis – er passt auf eine halbe Seite.
Schritt 1 – Annahme.
Angenommen, es gäbe ein Programm H mit

$$H(\langle P \rangle, x) = \begin{cases} \text{„hält“} & \text{falls } P(x) \text{ anhält,} \\ \text{„hält nicht“} & \text{sonst.} \end{cases}$$

*Wichtig: H soll **selbst immer anhalten** und eine Antwort geben.* **Sonst wäre es kein Entscheidungsverfahren.**

Schritt 2 – Ein Widerspenstiges bauen.
*Wir konstruieren daraus ein neues Programm D (für „Diagonale"), das **eine** Eingabe nimmt:*

```
D(w):
  if H(w, w) = "hält" then gehe in eine
Endlosschleife
  else halte an
```

*D fragt: „Hält das Programm w, wenn man ihm seinen **eigenen** Text als Eingabe gibt?“* **Und dann tut** D **genau das Gegenteil.**

Beachten Sie: D *ist ein völlig gewöhnliches Programm.* **Wenn** H **existiert, existiert auch** D *– man schreibt es in vier Zeilen hin.*

Schritt 3 – D **auf sich selbst anwenden.**

Nun setzen wir $w := \langle D \rangle$ *ein und fragen: Hält* $D(\langle D \rangle)$*?*

Fall 1: $D(\langle D \rangle)$ hält	Dann liefert $H(\langle D \rangle, \langle D \rangle)$ die Antwort „hält“. Also geht D nach seiner eigenen Vorschrift in die **Endlosschleife**. **Also hält es nicht. Widerspruch.**
Fall 2: $D(\langle D \rangle)$ hält nicht	Dann liefert H die Antwort „hält nicht“. Also **hält** D nach seiner eigenen Vorschrift an. **Also hält es. Widerspruch.**

Schritt 4 – Der Schluss.

Beide Fälle führen auf einen Widerspruch. Da D zwangsläufig entweder hält oder nicht hält, war die einzige Annahme falsch, die wir gemacht haben:

H existiert nicht.

Das ist kein Ergebnis über heutige Rechner, über Speichergrößen oder über unsere Findigkeit. Es gilt für jede denkbare Maschine, für alle Zeiten. **Es gibt keinen Algorithmus, der Terminierung entscheidet.**

Was daraus folgt

Kein perfekter Virenscanner	„Verhält sich dieses Programm bösartig?“ ist unentscheidbar
Kein perfekter Optimierer	„Wird dieser Code je erreicht?“ ist unentscheidbar
Keine vollständige Fehlersuche	„Kann hier eine Division durch null auftreten?“ ebenso
Kein Test auf Gleichheit	„Tun diese zwei Programme dasselbe?“ ebenso

*Der **Satz von Rice** (1951) verallgemeinert das drastisch: **Jede nichttriviale Eigenschaft des Verhaltens eines Programms ist unentscheidbar.***

„Nichttrivial“ heißt nur: Es gibt Programme mit und solche ohne die Eigenschaft.
Über die Eigenschaft selbst wird gar nichts vorausgesetzt.

*Entscheidbar bleiben nur Eigenschaften des **Textes*** – „Ist das Programm kürzer als 100 Zeichen?“, „Kommt der Buchstabe x vor?“

Und trotzdem gibt es Virenscanner

Der Satz sagt nicht, dass man *nichts* entscheiden kann. *Er sagt, dass kein Verfahren **alle** Fälle korrekt entscheidet.*

Ausweg 1	Antwort „weiß nicht“ erlauben
Ausweg 2	Nur eine **Teilklasse** von Programmen betrachten
Ausweg 3	Fehler in eine Richtung zulassen (lieber falscher Alarm)
Ausweg 4	Eine Zeitschranke setzen

Die gesamte statische Programmanalyse lebt von Ausweg 1 und 3. Ein Typprüfer weist manche korrekten Programme zurück – er ist **konservativ**. Das ist der Preis dafür, dass er überhaupt terminiert.

Ausweg 2 ist der Grund für totale Programmiersprachen (Coq, Agda, Idris): *Sie erlauben nur Rekursionen, deren Terminierung der Übersetzer nachweisen kann.* **Damit ist das Halteproblem für sie trivial – alles hält an.** *Der Preis: Man kann in ihnen keinen Interpreter für sich selbst schreiben.*

*Die Universalität und die Unentscheidbarkeit sind **dasselbe Merkmal**, von zwei Seiten gesehen.* **Wer das eine will, bekommt das andere dazu.**

Aufgabe 14.1.

(a) Konstruieren Sie einen Quine Schritt für Schritt und verstehen Sie den Mechanismus.
(b) Zeigen Sie, dass das Halteproblem und Cantors Diagonalargument dieselbe Struktur haben.
(c) Untersuchen Sie die Fleißige-Biber-Funktion und was sie über Beweisbarkeit sagt.

Lösung Schritt für Schritt.

(a) Ein Quine, konstruiert.
Schritt 1 – Der erste Versuch und sein Scheitern.
Wir versuchen es in einer Pseudosprache mit `print`*:*

```
print("print(")
```

Ausgabe: `print(`*. Fehlt der Rest.*
Also länger:

```
print("print(\"print(\")")
```

*Die Ausgabe ist jetzt der **vorige** Versuch, nicht der jetzige.* **Man bleibt immer eine Runde zurück.**

Schritt 2 – Die Einsicht.

*Der Regress entsteht, weil wir den Text **einmal** verwenden. **Man muss ihn zweimal verwenden**: einmal als auszuführenden Code und einmal als Datum, das gedruckt wird.*

Schritt 3 – Die Konstruktion.
Wir brauchen zwei Hilfsmittel:

Eine Variable `s`	enthält den Text des Programms als Zeichenkette
Eine Funktion `q(s)`	liefert `s` **in Anführungszeichen**

Damit lautet das Programm:

```
s := "s := @; print(ersetze(s, '@', q(s)))"
print(ersetze(s, '@', q(s)))
```

Dabei ersetzt `ersetze` *das Zeichen* `@` *durch den zweiten Wert.*
Schritt 4 – Warum das funktioniert.

Zeile 1	Legt den Text fest – mit einem **Loch** an der Stelle, wo der Text selbst stehen müsste
Zeile 2	Füllt das Loch mit dem zitierten Text und druckt das Ergebnis

Der Trick besteht darin, das Loch nicht füllen zu wollen, bevor man druckt. Die Zeichenkette beschreibt das Programm bis auf sich selbst – und die Selbsteinsetzung geschieht erst zur Laufzeit.

Schritt 5 – Der Vergleich mit Quines Satz.

Quines Satz	Programm
Text in Anführungszeichen	die Zeichenkette `s`
Text ohne Anführungszeichen	der ausgeführte Code
„seinem eigenen Zitat nachstellen“	`ersetze(s, '@', q(s))`

*Es ist **dieselbe** Konstruktion.* **Und es ist dieselbe, die Gödel 1931 benutzte, um in der Arithmetik einen Satz zu bauen, der seine eigene Unbeweisbarkeit behauptet.**

> *Man nennt sie das* ***Rekursionstheorem*** *(Kleene, 1938):* ***Zu jedem Programm*** F***, das Programmtexte verarbeitet, gibt es ein Programm*** P***, das sich genau so verhält wie*** F ***angewandt auf*** $\langle P \rangle$***.***

In Worten: *Jedes Programm darf seinen eigenen Quelltext benutzen, als hätte es ihn geschenkt bekommen.* **Das ist ein Satz, kein Kunstgriff** *– und aus ihm folgen Quines, Thompsons Hintertür und die Existenz selbstreproduzierender Programme gleichermaßen.*

(b) Halteproblem und Diagonalargument.
Schritt 1 – Cantors Argument, kurz wiederholt.
GEORG CANTOR *zeigte 1891: Die reellen Zahlen in* $[0,1]$ *lassen sich nicht durchnummerieren* (Mathematik-Band 10, Kapitel 5).
Man nimmt an, es gäbe eine Liste $r_1, r_2, r_3, \ldots$ *aller dieser Zahlen, und schreibt sie untereinander:*

r_1	**3**	1	4	1	…
r_2	2	**7**	1	8	…
r_3	1	4	**1**	4	…
r_4	5	7	7	**2**	…

Dann baut man eine neue Zahl d*, die sich an der* n*-ten Stelle von* r_n *unterscheidet.* d **steht in keiner Zeile – Widerspruch.** ✓
Schritt 2 – Dieselbe Tabelle für Programme.
Man ordne alle Programme (es sind abzählbar viele, denn jedes ist ein endlicher Text) in einer Liste an und trage ein, ob P_i *bei Eingabe* $\langle P_j \rangle$ *hält:*

	$\langle P_1 \rangle$	$\langle P_2 \rangle$	$\langle P_3 \rangle$	…
P_1	**hält**	hält nicht	hält	…
P_2	hält	**hält**	hält	…
P_3	hält nicht	hält	**hält nicht**	…

Schritt 3 – Die Diagonale umkehren.
D ist genau das Programm, das die Diagonale umkehrt:

$$D(\langle P_i \rangle) \text{ hält} \iff P_i(\langle P_i \rangle) \text{ hält } \textbf{nicht}.$$

*D unterscheidet sich also von **jedem** P_i an der Stelle i –* **genau wie Cantors Zahl d sich von jedem r_n an der Stelle n unterscheidet.**

Schritt 4 – Der Unterschied im Schluss.

	Cantor	**Turing**
Liste enthält	alle reellen Zahlen	**alle Programme**
d bzw. D	steht nicht in der Liste	**steht sehr wohl darin**
Folgerung	Liste unvollständig	H **existiert nicht**

Der zweite Punkt ist der entscheidende Unterschied. Bei Cantor darf d fehlen – daraus folgt, dass die Liste unvollständig war. **Bei Turing *muss* D in der Liste stehen, denn es ist ein gewöhnliches Programm.** *Also muss stattdessen H falsch sein – die einzige andere Zutat.*

Turing kannte Cantors Argument und sagte es ausdrücklich. *Seine Arbeit von 1936 heißt „On Computable Numbers" – er untersuchte zunächst, welche **reellen Zahlen** sich berechnen lassen, und stellte fest: nur abzählbar viele.*

Fast alle reellen Zahlen sind unberechenbar. Man kann keine einzige von ihnen hinschreiben – denn jede Beschreibung wäre ein endlicher Text und damit ein Programm. **Sie existieren, und sie sind unerreichbar.**

(c) Der fleißige Biber.

Definition 14.3. $\Sigma(n)$ (**Busy-Beaver-Funktion**) ist die größte Anzahl von Einsen, die eine anhaltende Turingmaschine mit n Zuständen auf ein anfangs leeres Band schreiben kann.

TIBOR RADÓ *führte sie 1962 ein.*

Schritt 1 – Die bekannten Werte.

n	$\Sigma(n)$	**Status**
1	1	trivial
2	4	
3	6	1965
4	13	1983
5	**4098**	**2024 bewiesen**
6	$> 10^{10^{15}}$	offen
7	$> 10^{10^{10^{10^{18705353}}}}$	offen

Der Wert $\Sigma(5) = 4098$ *wurde im Juli 2024 von einem Zusammenschluss von Freiwilligen („bbchallenge") bewiesen* – **und der Beweis ist in Coq formal verifiziert.** *Er erforderte die Analyse von rund 88 Millionen Maschinen.*

Schritt 2 – Warum Σ nicht berechenbar ist.
Angenommen, ein Programm B berechnete $\Sigma(n)$. Dann könnte man das Halteproblem lösen:

1. Gegeben eine Maschine M mit n Zuständen.
2. Man berechne $\Sigma(n)$ und daraus die maximale Schrittzahl $S(n)$.
3. Man lasse M genau $S(n)$ Schritte laufen.
4. Hält sie bis dahin nicht, so hält sie **nie** – denn $S(n)$ ist das Maximum über alle anhaltenden Maschinen dieser Größe.

Damit wäre das Halteproblem entschieden – **Widerspruch.** ✓

Σ ***wächst schneller als jede berechenbare Funktion.*** Sie überholt die Fakultät, die Ackermannfunktion und jede Türme-von-Potenzen-Konstruktion, die man hinschreiben kann – **ab irgendeinem n und für immer.**

Schritt 3 – Der wirklich merkwürdige Teil.

> YEDIDIA *und* AARONSON *konstruierten 2016 eine Turingmaschine mit 7918 Zuständen (später auf 748 verkleinert), die* ***genau dann anhält, wenn die Zermelo-Fraenkel-Mengenlehre widersprüchlich ist.***

Daraus folgt: *Der Wert* $\Sigma(748)$ *ist innerhalb von ZFC* ***nicht bestimmbar*** **– denn wüsste man ihn, könnte man die Widerspruchsfreiheit von ZFC in ZFC beweisen**, *was Gödels zweiter Unvollständigkeitssatz ausschließt* (Mathematik-Band 10, Kapitel 12).

$\Sigma(5)$	bewiesen: 4098
$\Sigma(6)$	offen, aber vermutlich prinzipiell erreichbar
$\Sigma(748)$	**in ZFC nicht beweisbar**

> ***Hier stößt eine schlichte kombinatorische Frage – „Wie viele Striche kann eine kleine Maschine malen?“ – an die Grenzen der Beweisbarkeit selbst.***

Auch andere offene Vermutungen lassen sich so kodieren: *Es gibt Maschinen mit wenigen Zuständen, die genau dann anhalten, wenn die Goldbachsche Vermutung falsch ist (27 Zustände) oder die Riemannsche Vermutung falsch ist (744 Zustände).*

Wer Σ *für diese Größen kennen würde, hätte die Vermutungen entschieden.* **Man kennt sie nicht, und man wird sie nicht kennen.**

> ***Das ist der Punkt, an dem dieser Band endet und der Begleitband beginnt.*** *Der seltsame Kreis – ein System, das über sich selbst spricht – erzeugt zugleich die Universalität des Rechners und die Grenze dessen, was er wissen kann.* **Beides ist dieselbe Eigenschaft.**

Turings Originalarbeit[25] ist erstaunlich zugänglich; Davis[4] erzählt die Geschichte von Leibniz bis Turing; Hofstadter[11] macht den Selbstbezug zum Gegenstand eines ganzen Buches – und Sipser[23] führt die Theorie streng aus.

Part VI

Aktueller Stand

15 Wo die Grundlagen heute stehen

Was sich ändert und was nicht

Unverändert seit	
1847	Boolesche Algebra
1936	Berechenbarkeit, Halteproblem
1937	Schaltalgebra (Shannon)
1945	Von-Neumann-Architektur
1948	Informationstheorie
1972	Zweierkomplement als Standard
1985	IEEE 754
In Bewegung	
seit 2005	Ende der Taktfrequenzsteigerung
seit 2016	Ende der klassischen Strukturverkleinerung
seit 2018	Sicherheitslücken *in* der Mikroarchitektur
seit 2020	Rückkehr zu spezialisierter Hardware

Die obere Hälfte ist der Inhalt dieses Bandes. *Sie wird in fünfzig Jahren unverändert gelten.* **Die untere Hälfte ändert sich, während dieser Satz gedruckt wird.**

15.1 Das Ende der Skalierung

Drei Gesetze und ihr Schicksal

Gesetz	Aussage	Status
Moore (1965)	Transistorzahl verdoppelt sich alle 2 Jahre	stark verlangsamt
Dennard (1974)	Leistungsdichte bleibt bei Verkleinerung gleich	**seit 2005 gebrochen**
Amdahl (1967)	Grenze der Parallelisierbarkeit	gilt weiterhin

Warum das Dennard-Gesetz zuerst brach.
Bis etwa 2005 konnte man mit jeder Verkleinerung auch die Spannung senken – die Verlustleistung blieb pro Fläche gleich. **Unterhalb etwa 1 Volt geht das nicht mehr: Der Leckstrom durch das Gate wird dominant.**

> *Die Folge war der abrupte Wechsel zu **Mehrkernprozessoren**.* **Man konnte die Taktrate nicht mehr steigern, also verdoppelte man stattdessen die Kerne** – *und schob damit das Problem auf die Programmierer.*

Jahr	Takt	Kerne	Struktur
1995	100 MHz	1	350 nm
2005	3,8 GHz	1–2	90 nm
2015	4,0 GHz	4–8	14 nm
2026	≈ 5,5 GHz	16–128	2–3 nm

Die Taktrate ist in zwanzig Jahren um 45 Prozent gestiegen, die Kernzahl um das Fünfzigfache.

„2 Nanometer“ ist ein Marketingname

Die Bezeichnungen der Fertigungsknoten entsprechen seit etwa 2010 keiner gemessenen Länge mehr.

Der kleinste Abstand in einem „3-nm“-Prozess liegt bei etwa 20 nm; die Gatelänge bei etwa 16 nm. **Ein Siliziumatom misst** 0,22 **nm.**

„2 nm“ bedeutet in Wahrheit: „die Generation, die etwa so viel Verbesserung bringt, wie eine Verkleinerung auf 2 nm gebracht hätte“. **Es ist eine Generationsnummer, keine Größenangabe.**

Die physikalische Grenze ist trotzdem in Sicht: *Bei Kanallängen unter etwa 5 nm tunneln Elektronen durch die Sperrschicht, und der Transistor lässt sich nicht mehr ausschalten* – **ein quantenmechanischer Effekt, den keine Fertigungskunst behebt.**

Das Gesetz von Amdahl

Wenn mehr Kerne die Antwort sind, wie viel bringen sie?

Satz 15.1 (Amdahl, 1967)**.** *Ist p der parallelisierbare Anteil eines Programms, so beträgt die Beschleunigung mit N Kernen höchstens*

$$\boxed{S(N) = \frac{1}{(1-p) + \frac{p}{N}}}$$

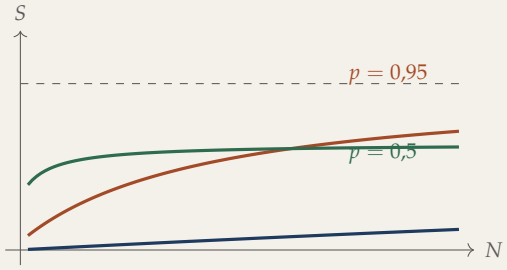
und im Grenzfall $N \to \infty$ nur $\dfrac{1}{1-p}$.

Die Herleitung ist eine Zeile. *Die Gesamtzeit sei 1. Der serielle Teil $(1-p)$ bleibt, der parallele Teil p wird auf N Kerne verteilt:*

$$T(N) = (1-p) + \frac{p}{N}. \qquad \checkmark$$

p	$S(8)$	$S(1000)$	$S(\infty)$
0,50	1,8	2,0	**2**
0,90	4,7	9,9	**10**
0,95	6,0	19,6	**20**
0,99	7,5	91	**100**
0,999	7,9	500	**1000**

Bei 90 Prozent parallelisierbarem Anteil bringen unendlich viele Kerne den Faktor 10. Nicht mehr. Das restliche Zehntel bestimmt die Grenze – gleichgültig, wie viel Hardware man aufwendet.



Die Gegenposition. JOHN GUSTAFSON *bemerkte 1988, dass man bei mehr Rechenleistung meist* ***größere Probleme*** *rechnet, nicht dieselben schneller.* **Wächst der parallele Anteil mit der Problemgröße, so ist die Beschleunigung nicht beschränkt.**

Beide Gesetze sind richtig – sie beantworten verschiedene Fragen. **Amdahl: „Wie schnell wird *dieses* Problem?“ Gustafson: „Wie groß darf das Problem werden?“**

Was seit 2018 die Architektur erschüttert
Im Januar 2018 wurden ***Spectre*** *und* ***Meltdown*** *veröffentlicht – Sicherheitslücken, die nicht in einem Programm steckten, sondern in der* ***Mikroarchitektur*** *praktisch aller Prozessoren seit etwa 1995.*

Der Mechanismus in einem Satz.

Moderne Prozessoren führen Befehle ***spekulativ*** *aus – sie raten den Ausgang einer Verzweigung und rechnen weiter, bevor sie es wissen.* **Rät der Prozessor falsch, verwirft er die Ergebnisse – aber die Spuren im Cache bleiben.**

Aus diesen Spuren lassen sich fremde Speicherinhalte rekonstruieren – Passwörter, Schlüssel, alles.

Warum das für diesen Band bedeutsam ist.

Es ist die folgenreichste ***durchlässige Abstraktion*** *der Informatikgeschichte* (Kapitel 12). **Der Befehlssatz – die stabile Schicht, auf die sich alles verlässt – sagt nichts über Spekulation.** *Ein Programm, das* ***gemessen an der Schnittstelle*** *korrekt und sicher ist, ist es* ***unterhalb*** *der Schnittstelle nicht.*

Stand August 2026: *Meltdown ist durch Speicherumbau behoben. Spectre ist es im Grundsatz* ***nicht*** **– es folgt aus der spekulativen Ausführung selbst,** *und die aufzugeben würde Prozessoren um den Faktor 3 bis 5 verlangsamen.*

Man behilft sich mit Einzelmaßnahmen, und in regelmäßigen Abständen erscheinen neue Varianten. **Der Preis der bisherigen Gegenmaßnahmen liegt je nach Last bei 5 bis 30 Prozent Leistung.**

15.2 Wohin es geht

Vier Richtungen

Erstens: Spezialisierung statt Universalität.

Wenn der Allzweckprozessor nicht mehr schneller wird, baut man Schaltungen für einzelne Aufgaben. **Das ist eine bemerkenswerte Umkehrung: Turings Universalität war der Fortschritt – und jetzt geht man teilweise zurück.**

Baustein	Aufgabe	Vorteil
GPU	Vektorrechnung	Faktor 10–100
TPU/NPU	Matrixmultiplikation	Faktor 10–50
Videokodierer	Kompression	Faktor 100 bei Energie
Kryptobefehle	AES, SHA	Faktor 5–10

*Die Universalität geht dabei nicht verloren – die Spezialbausteine sitzen **neben** dem Allzweckprozessor, der sie ansteuert.*

Zweitens: Andere Rechenmodelle.

Ansatz	Idee	Stand 2026
Quantenrechner	Superposition, Verschränkung	Fehlerkorrektur im Aufbau (Band 15)
Neuromorph	Ereignisgesteuerte Neuronen	Forschung, sehr energiesparsam (Band 14)
Analogrechnen	Physik rechnet mit	Nischen, KI-Beschleuniger
Reversibel	keine Information vernichten	theoretisch, Landauer-Grenze

Keiner dieser Ansätze bricht die Grenze aus Kapitel 14. Ein Quantenrechner kann das Halteproblem so wenig lösen wie ein Abakus. **Er ändert, was *effizient* berechenbar ist – nicht, was berechenbar ist.**

Drittens: Die Landauer-Grenze.

ROLF LANDAUER *zeigte 1961: Das **Löschen** eines Bits kostet mindestens*

$$E_{\min} = k_B \, T \, \ln 2$$

an Energie – bei Zimmertemperatur etwa $2{,}9 \cdot 10^{-21}$ *Joule.*

Vorgang	Energie je Bit	Faktor
Landauer-Grenze	$2{,}9 \cdot 10^{-21}$ J	1
Transistor 2026	$\approx 10^{-17}$ J	≈ 3000
Transistor 1990	$\approx 10^{-12}$ J	$\approx 3 \cdot 10^{8}$

Es bleiben also noch etwa drei Größenordnungen – aber die Grenze ist ***thermodynamisch****, nicht technisch.* **Sie folgt aus dem zweiten Hauptsatz und ist mit keiner Bauweise zu unterbieten.**

Der Ausweg wäre ***reversibles Rechnen****: Wer keine Information löscht, muss keine Energie aufwenden.* **Der Preis ist, dass man alle Zwischenergebnisse aufheben muss.**

Bemerkenswerterweise ist Quantenrechnen von Natur aus reversibel – **jedes Quantengatter ist umkehrbar** (Band 15).

Viertens: Formale Verifikation.

System	Was bewiesen ist	Jahr
seL4	Betriebssystemkern	2009
CompCert	C-Übersetzer	2008
CakeML	ML-Übersetzer	2014
Kernel-Teile	Speicherverwaltung	laufend

Bei CompCert lässt sich beweisen, dass der erzeugte Maschinencode dasselbe tut wie der Quelltext. Eine Untersuchung von 2011 fand mit automatisierten Tests in jedem verbreiteten C-Übersetzer hunderte Fehler **– und in CompCerts verifiziertem Teil keinen einzigen.**

Das ist die praktische Antwort auf den Satz von Rice (Kapitel 14): **Man kann nicht *jedes* Programm prüfen – aber man kann *ein bestimmtes* Programm beweisen.**

Stand August 2026 – eine Momentaufnahme

Frage	Status	Bemerkung
Halteproblem	1936 entschieden	unlösbar, endgültig
P gegen NP	**offen**	Millenniumsproblem (Band 3)
Moore'sches Gesetz	stark verlangsamt	Kosten je Transistor steigen wieder
Spectre	grundsätzlich offen	folgt aus Spekulation selbst
$\Sigma(6)$	offen	$> 10^{10^{15}}$
$\Sigma(5) = 4098$	2024 bewiesen	in Coq verifiziert
Fehlertolerante Quantenrechner	im Aufbau	Band 15

Drei Beobachtungen zum Zustand des Fachs.
Erstens: Die Grundlagen sind älter als die Technik. *Boole 1847, Shannon 1937, Turing 1936, von Neumann 1945* – **alles Wesentliche dieses Bandes war 1950 fertig gedacht.** *Was danach kam, war Ingenieurskunst in gewaltigem Maßstab.*
Zweitens: Die Abstraktionen leaken zunehmend. *Cache, Spekulation, Energieverbrauch, Mehrkernigkeit – alles Dinge, die die Schnittstelle verschweigt und die trotzdem über Korrektheit und Geschwindigkeit entscheiden.*

> *Der Trend geht dahin, dass Programmierer wieder mehr über die unteren Schichten wissen müssen als in den 1990er Jahren* – **eine Umkehrung der Entwicklung von fünfzig Jahren.**

Drittens: Der Beweis kehrt zurück. *Verifizierte Übersetzer, verifizierte Betriebssystemkerne, formalisierte mathematische Beweise* – **Techniken, die dreißig Jahre lang als akademisch galten, sind im praktischen Einsatz angekommen.**

Schlussgedanke: drei Einsichten

Erstens: Alles ist auf eine einzige Unterscheidung zurückgeführt.

Wir sind vom Schalter zum Gatter, vom Gatter zum Addierer, vom Addierer zum Rechenwerk, über die Rückkopplung zum Gedächtnis und über den gespeicherten Befehl zur Universalmaschine gekommen. **An keiner Stelle kam etwas hinzu, das nicht aus dem Vorigen gebaut war.**

Die Kette aus dem Vorwort ist geschlossen – **und der einzige Baustein war ein Ding, das an- oder ausgeschaltet ist.**
Zweitens: Jede Schicht ist eine nützliche Lüge.

Behauptung	Wahrheit
„Bits sind 0 oder 1“	Spannungen sind stetig
„Zahlen rechnen exakt“	Gleitkomma rundet
„Speicher ist gleich schnell“	Faktor 100 durch Cache
„Ein Befehl nach dem anderen“	Spekulation, Parallelität

Ohne diese Vereinfachungen könnte niemand ein System mit 10^{10} *Transistoren überblicken.* **Mit ihnen kann man es benutzen – aber nur, wer sie durchschaut, kann es reparieren.**

Drittens: Universalität und Grenze sind dieselbe Eigenschaft.

Weil ein Programm ein Datum ist, kann eine einzige Maschine alle anderen nachbilden – das ist Turings Universalität und der Grund, warum es überhaupt Computer gibt.

Und weil ein Programm ein Datum ist, kann es auf sich selbst angewandt werden – das ist Turings Diagonale und der Grund, warum es Dinge gibt, die kein Computer je entscheiden wird.

Man kann das eine nicht ohne das andere haben. **Eine Maschine, die nicht über sich selbst reden kann, ist auch nicht universell.**

DOUGLAS HOFSTADTER *hat 1979 die Vermutung geäußert, dass genau dieser Kreis – ein System, das reich genug ist, um sich selbst zu beschreiben – auch das ist, was ein Gehirn zu einem Ich macht.* **Ob das stimmt, weiß niemand.** *Dass die Struktur dieselbe ist, lässt sich zeigen* – **und der Begleitband dieser Reihe versucht es.**

Wir können nur eine kurze Strecke voraussehen,
aber wir sehen dort viel, was getan werden muss.

— Alan Turing (1912–1954),
letzter Satz von „Computing Machinery
and Intelligence“, 1950.

Part VII

Anhang

A Formelsammlung und Übersicht

A.1 Die Kette vom Schalter zur Maschine

Der Aufbau in einer Tabelle

Baustein	Wird daraus gebaut	Kapitel
Transistor	Schalter	7
NAND-Gatter	4 Transistoren (CMOS)	7
alle Booleschen Funktionen	NAND allein genügt	6
Halbaddierer	XOR + UND	7
Volladdierer	2 Halbaddierer + ODER	7
n-Bit-Addierer	n Volladdierer	7
ALU	Addierer + Multiplexer	7
Flipflop	**2 rückgekoppelte NANDs**	8
Register	n Flipflops	8
Schaltwerk	Register + Schaltnetz + Takt	9
Rechner	**+ Speicher für Befehle**	10

Die fett gesetzten Zeilen sind die beiden Stellen, an denen etwas qualitativ Neues entsteht: **das Gedächtnis und die Universalität.**

A.2 Zahlen und Kodierung

Was man im Kopf haben sollte

Stellenwertsystem:

$$(z_{n-1} \dots z_0)_2 = \sum_{i=0}^{n-1} z_i \, 2^i.$$

Zweierkomplement (Wert eines n-Bit-Musters):

$$\text{Wert} = -z_{n-1} 2^{n-1} + \sum_{i=0}^{n-2} z_i 2^i, \qquad -x = \overline{x} + 1.$$

$$\text{Bereich: } -2^{n-1} \dots 2^{n-1} - 1.$$

Gleitkomma (IEEE 754):

$$x = (-1)^s \cdot 1{,}m \cdot 2^{e-B}.$$

Format	Bits	Exp.	Mant.	Bias B
`float`	32	8	23	127
`double`	64	11	52	1023

$$\varepsilon_{\text{double}} = 2^{-52} \approx 2{,}22 \cdot 10^{-16} \quad \text{(ca. 16 Dezimalstellen).}$$

Information und Entropie:

$$I(p) = \log_2 \frac{1}{p}, \qquad H = -\sum_i p_i \log_2 p_i, \qquad H_{\max} = \log_2 n.$$

Zweierpotenzen:

2^{10}	1024	2^{32}	$4{,}29 \cdot 10^9$
2^{16}	65 536	2^{64}	$1{,}84 \cdot 10^{19}$
2^{20}	$1{,}05 \cdot 10^6$	2^{80}	$1{,}21 \cdot 10^{24}$
2^{24}	$1{,}68 \cdot 10^7$	2^{128}	$3{,}40 \cdot 10^{38}$

Merkregel: $2^{10} \approx 10^3$, *also* $2^{10k} \approx 10^{3k}$. **Der Fehler beträgt 2,4 % je Zehnerschritt.**

Boolesche Algebra

$a \vee 0 = a, \quad a \wedge 1 = a$	neutrale Elemente
$a \vee 1 = 1, \quad a \wedge 0 = 0$	dominierende Elemente
$a \vee a = a, \quad a \wedge a = a$	**Idempotenz**
$a \vee \overline{a} = 1, \quad a \wedge \overline{a} = 0$	Komplement
$\overline{\overline{a}} = a$	Involution
$a \vee (a \wedge b) = a$	**Absorption**
$\overline{a \wedge b} = \overline{a} \vee \overline{b}$	**de Morgan**
$\overline{a \vee b} = \overline{a} \wedge \overline{b}$	**de Morgan**
$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$	Distributivität
$a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$	**gilt hier auch!**

Die vorletzte und letzte Zeile zusammen sind der Unterschied zur gewöhnlichen Algebra – **dort gilt nur eine der beiden Richtungen.**

Vollständige Systeme:

$\{\wedge, \vee, \neg\}$	Standard
$\{\wedge, \neg\}$ und $\{\vee, \neg\}$	über de Morgan
$\{\overline{\wedge}\}$	**allein vollständig**
$\{\underline{\vee}\}$	**allein vollständig**

$$\neg a = a \,\overline{\wedge}\, a, \qquad a \wedge b = (a \,\overline{\wedge}\, b) \,\overline{\wedge}\, (a \,\overline{\wedge}\, b),$$
$$a \vee b = (a \,\overline{\wedge}\, a) \,\overline{\wedge}\, (b \,\overline{\wedge}\, b).$$

A.3 Aufwandsklassen

Die Größenordnungen

Klasse	$n = 10^6$	**Beispiel**
$O(1)$	sofort	Feldzugriff, Hashtabelle
$O(\log n)$	20 Schritte	binäre Suche
$O(n)$	10^6	Maximum, Summe
$O(n \log n)$	$2 \cdot 10^7$	Mergesort, FFT
$O(n^2)$	10^{12}	einfaches Sortieren
$O(n^3)$	10^{18}	Matrixmultiplikation
$O(2^n)$	unvorstellbar	alle Teilmengen

Untere Schranken (bewiesen):

Vergleichsbasiertes Sortieren	$\Omega(n \log n)$
Suche in unsortierter Folge	$\Omega(n)$
Unterscheidung von N Fällen	$\lceil \log_2 N \rceil$ Ja-Nein-Fragen
Löschen eines Bits	$k_B T \ln 2$ Joule (Landauer)

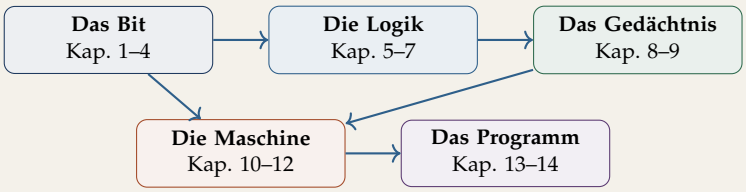
Typische Zugriffszeiten (Größenordnungen, 2026):

Ebene	**Zeit**	**maßstäblich**
Register	0,3 ns	1 Sekunde
L1-Cache	1 ns	3 Sekunden
L2-Cache	4 ns	13 Sekunden
Hauptspeicher	80 ns	**4 Minuten**
SSD	50 µs	**2 Tage**
Festplatte	5 ms	**6 Monate**
Netz (interkontinental)	150 ms	**16 Jahre**

Die rechte Spalte skaliert alles so, als dauerte ein Registerzugriff eine Sekunde. **Sie macht anschaulich, warum die Speicherhierarchie über die Laufzeit entscheidet – und nicht die Zahl der Befehle.**

A.4 Die Landkarte des Bandes

Aufbau und Querverbindungen



Querverbindungen zur Mathematikreihe:

Math. Band 0	Logarithmus, Stellenwertsysteme, Induktion
Math. Band 1	Modulare Arithmetik, ggT, Primzahlen
Math. Band 3	Grenzwerte, Monotonie, dichte Mengen
Math. Band 4	Wahrscheinlichkeit, Entropie
Math. Band 10	Berechenbarkeit, Gödel, Auswahlaxiom
Math. Band 11	Boolesche Algebra als Verband, endliche Körper
Math. Band 15	Graphen, Abzählung, untere Schranken

Ausblick auf die weiteren Bände dieser Reihe:

1	Algorithmen und Datenstrukturen
2	**Berechenbarkeit** – die Grenze aus Kapitel 14 vollständig
3	Komplexität, P gegen NP
4	Rechnerarchitektur – Pipeline, Cache, Parallelität
5	Betriebssysteme und Nebenläufigkeit
6	Programmiersprachen und Compiler
7	Informationstheorie und Kompression
8	Kryptographie
9	Netzwerke und verteilte Systeme
10	Datenbanken und Retrieval
11	Graphen und Netzwerkanalyse
12	Symbolische künstliche Intelligenz
13	Lernende Systeme
14	Computational Neuroscience
15	Quanteninformatik
B	**Der seltsame Kreis** – Selbstbezug, Gödel, Bewusstsein

A.5 Zeittafel

Von Leibniz bis zur Gegenwart

1679	Leibniz	Binärsystem
1801	Jacquard	Lochkartensteuerung
1837	Babbage	Analytical Engine entworfen
1843	**Lovelace**	**erstes Programm; universelle Symbolverarbeitung**
1847	**Boole**	**Boolesche Algebra**
1854	Boole	*Laws of Thought*
1904	Fleming	Vakuumröhre
1928	Hilbert	Entscheidungsproblem gestellt
1936	**Turing**	**Turingmaschine, Universalmaschine, Halteproblem**
1936	Church	Lambda-Kalkül
1937	**Shannon**	**Schaltalgebra = Boolesche Algebra**
1938	Zuse	Z1
1941	Zuse	**Z3** – erster funktionierender programmgesteuerter Rechner
1943	McCulloch, Pitts	Neuron als logisches Gatter
1945	von Neumann	EDVAC-Bericht: gespeichertes Programm
1945	Colossus, ENIAC	Röhrenrechner im Einsatz
1948	**Shannon**	**Informationstheorie; das Wort „Bit“**
1948	Manchester Baby	erstes gespeichertes Programm ausgeführt
1947/48	Bardeen u. a.	**Transistor**
1951	Rice	Satz von Rice
1954	Rochester	Assembler mit symbolischen Adressen
1956	Buchholz	das Wort „Byte“
1957	Backus	FORTRAN – erste weit verbreitete Hochsprache
1958	Kilby, Noyce	integrierter Schaltkreis
1958–65	Brusenzow	Setun (ternär)
1960	McCarthy	Lisp: meta-zirkularer Inter-

A.6 Glossar

Die Begriffe dieses Bandes

Abstraktionsschicht	Schnittstelle plus Implementierung
Adressierungsart	Woher der Operand eines Befehls kommt
Assembler	Buchstabenschreibweise für Maschinenbefehle
Befehlszähler	Register mit der Adresse des nächsten Befehls
Bit	Informationsmenge einer Ja-Nein-Frage; auch: Speicherstelle
Bootstrapping	Ein Übersetzer übersetzt sich selbst
Byte	Acht Bit (normsprachlich: Oktett)
Entropie	Mittlerer Informationsgehalt einer Quelle
Flipflop	Rückgekoppelte Gatter, die ein Bit halten
Gatter	Schaltung für eine Boolesche Grundfunktion
Halteproblem	Terminiert dieses Programm? – unentscheidbar
Invariante	Aussage, die jeder Schleifendurchlauf erhält
Maschinenepsilon	Kleinstes ε mit $1 + \varepsilon \neq 1$
NAND	Nicht-Und; allein vollständig
Normalform	Kanonische Darstellung Boolescher Ausdrücke
***O*-Notation**	Wachstumsordnung, Konstanten ignoriert
Opcode	Feld eines Befehlsworts, das die Operation angibt
Quine	Programm, das seinen eigenen Text ausgibt
Schaltnetz	Schaltung ohne Gedächtnis
Schaltwerk	Schaltung mit Zustand
Stapel	Speicher nach dem Prinzip *last in, first out*
Universalmaschine	Programm, das jedes Programm ausführt
Von-Neumann-Arch.	Befehle und Daten im selben Speicher
Zweierkomplement	Darstellung negativer Zahlen; $-x = \overline{x} + 1$

B Kommentierte Literatur

Achtundzwanzig Titel

Die Auswahl deckt alle Teile des Bandes ab – *Information, Zahldarstellung, Logik und Schaltungen, Architektur, Algorithmen und die Grundlagenfragen.* **Die Reihenfolge ist alphabetisch.**

[1] Abelson, H.; Sussman, G. J.: *Structure and Interpretation of Computer Programs.* 2. Aufl., MIT Press 1996.

Das legendäre Lehrbuch des MIT-Grundkurses. Es geht nicht um eine Sprache, sondern um Abstraktion, Zustand und Interpretation – mit einem meta-zirkularen Lisp-Interpreter als Höhepunkt. Frei verfügbar; anspruchsvoll und lohnend.

[2] Bryant, R. E.; O'Hallaron, D. R.: *Computer Systems: A Programmer's Perspective.* 3. Aufl., Pearson 2015.

Der beste Brückenschlag zwischen Hochsprache und Maschine. Zeigt an echten x86-Beispielen, was der Übersetzer erzeugt und warum Gleitkomma, Cache und Verknüpfung sich so verhalten, wie sie es tun.

[3] Ceruzzi, P. E.: *A History of Modern Computing.* 2. Aufl., MIT Press 2003.

Nüchterne, quellengesättigte Technikgeschichte von ENIAC bis zum Internet. Ceruzzi war Kurator am Smithsonian und interessiert sich für das, was tatsächlich gebaut und benutzt wurde – nicht für Legenden.

[4] Davis, M.: *The Universal Computer: The Road from Leibniz to Turing.* 3. Aufl., CRC Press 2018.

Die Ideengeschichte in sieben Porträts – Leibniz, Boole, Frege, Cantor, Hilbert, Gödel, Turing. Davis war selbst Berechenbarkeitstheoretiker und erzählt die Mathematik mit, statt sie zu umgehen.

[5] Dewdney, A. K.: *The New Turing Omnibus.* Holt 1993.

Sechsundsechzig kurze Kapitel über je ein Thema der Informatik – von Zellularautomaten über Kompression bis zur Unentscheidbarkeit. Zum Querlesen; jedes Kapitel ist in einer halben Stunde zu schaffen.

[6] Feynman, R. P.: *Feynman Lectures on Computation.* Addison-Wesley 1996.

Feynmans Vorlesung von 1983–86, aus Mitschriften rekonstruiert. Behandelt Berechenbarkeit, Thermodynamik des Rechnens und reversibles Rechnen – und enthält frühe Überlegungen zum Quantencomputer. Eigenwillig und erhellend.

[7] Goldberg, D.: *What Every Computer Scientist Should Know About Floating-Point Arithmetic.* ACM Computing Surveys 23(1), 1991.

Fünfzig Seiten, die man gelesen haben muss, bevor man ernsthaft numerisch rechnet. Erklärt IEEE 754 lückenlos – Rundung, Auslöschung, Sonderwerte. Frei verfügbar.

[8] Harel, D.; Feldman, Y.: *Algorithmics: The Spirit of Computing.* 3. Aufl., Addison-Wesley 2004.

Erklärt algorithmisches Denken fast ohne Code und ohne Formalismus, dafür mit ungewöhnlich klaren Bildern. Reicht bis zu Unentscheidbarkeit und Kryptographie.

[9] Hennessy, J. L.; Patterson, D. A.: *Computer Architecture: A Quantitative Approach.* 6. Aufl., Morgan Kaufmann 2017.

Das Standardwerk der Rechnerarchitektur, von den beiden Turing-Preisträgern des Jahres 2017. Argumentiert konsequent mit Messungen statt mit Meinungen. Für Band 4 dieser Reihe die Hauptquelle.

[10] Hillis, W. D.: *The Pattern on the Stone.* Basic Books 1998.

Die eleganteste Kurzdarstellung der ganzen Kette vom Schalter zum Computer – unter 200 Seiten, ohne eine einzige Formel. Hillis baute die Connection Machine; er weiß, wovon er schreibt.

[11] Hofstadter, D. R.: *Gödel, Escher, Bach: ein Endloses Geflochtenes Band.* Klett-Cotta 1985 (engl. 1979).

Das Buch über den seltsamen Kreis. Verknüpft Gödels Unvollständigkeitssatz, Eschers Bilder und Bachs Kanons zu einer These über Selbstbezug und Bewusstsein. Der Begleitband dieser Reihe nimmt den Faden auf.

[12] Hyde, R.: *Write Great Code, Volume 1: Understanding the Machine.* 2. Aufl., No Starch 2020.

Sehr praxisnah zu Zahldarstellung, Zeichenkodierung und Speicherlayout – genau die Themen der Kapitel 2 bis 4, mit vielen Fallstricken aus der Praxis.

[13] Kernighan, B. W.; Ritchie, D. M.: *The C Programming Language.* 2. Aufl., Prentice Hall 1988.

Zweihundert Seiten, die eine Sprache vollständig beschreiben. Als Muster technischer Prosa unerreicht; zugleich das Dokument der Sprache, in der Unix aus dem Assembler herauswuchs.

[14] Knuth, D. E.: *The Art of Computer Programming, Vol. 1: Fundamental Algorithms.* 3. Aufl., Addison-Wesley 1997.

Der erste Band des Jahrhundertwerks. Enthält die MIX-Maschine, die Analyse des euklidischen Algorithmus und die Grundlagen der Datenstrukturen – mit einer historischen Gründlichkeit, die es sonst nirgends gibt.

[15] Knuth, D. E.: *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms.* 3. Aufl., Addison-Wesley 1997.

Die Referenz für Arithmetik: Zahldarstellungen, Multiplikationsverfahren, Gleitkomma, Zufallszahlen. Das Kapitel über Zufallsgeneratoren hat die Praxis dauerhaft verändert.

[16] Kohavi, Z.; Jha, N. K.: *Switching and Finite Automata Theory.* 3. Aufl., Cambridge 2009.

Das gründlichste Werk zu Schaltalgebra, Minimierung und Automaten – die Kapitel 5 bis 9 dieses Bandes in voller Tiefe, einschließlich Zustandsminimierung und Testbarkeit.

[17] Muller, J.-M. u. a.: *Handbook of Floating-Point Arithmetic.* 2. Aufl., Birkhäuser 2018.

Wenn Goldberg [7] nicht reicht: sechshundert Seiten über alles, was IEEE 754 betrifft – einschließlich der Beweise für Rundungseigenschaften und der Fallstricke bei Compileroptimierung.

[18] Nisan, N.; Schocken, S.: *The Elements of Computing Systems.* 2. Aufl., MIT Press 2021.

Das „Nand2Tetris"-Projekt: Der Leser baut in zwölf Projekten alles selbst – vom NAND-Gatter über ALU, Prozessor und Assembler bis zum Übersetzer und Betriebssystem. Wer diesen Band *tun* statt lesen will, nimmt dieses Buch dazu.

[19] Overton, M. L.: *Numerical Computing with IEEE Floating Point Arithmetic.* SIAM 2001.

Ein schmaler Band, der die Norm anhand konkreter Rechnungen erklärt. Besonders gut bei den Sonderwerten und beim Zusammenhang zwischen Rundung und Stabilität.

[20] Patterson, D. A.; Hennessy, J. L.: *Computer Organization and Design.* 6. Aufl., Morgan Kaufmann 2020.

Das Einführungswerk der beiden – Datenpfad, Steuerwerk, Pipeline und Speicherhierarchie, entwickelt an MIPS beziehungsweise RISC-V. Die natürliche Fortsetzung der Kapitel 10 bis 12.

[21] Petzold, C.: *Code: The Hidden Language of Computer Hardware and Software.* 2. Aufl., Microsoft Press 2022.

Die schönste Darstellung dieses Stoffes überhaupt. Petzold beginnt mit Taschenlampen und Morsezeichen und baut über Relais, Gatter und Flipflops einen vollständigen Rechner – ohne je etwas zu überspringen. Meine erste Empfehlung.

[22] Shannon, C. E.: *A Mathematical Theory of Communication.* Bell System Technical Journal 27, 1948.

Die Gründungsarbeit der Informationstheorie – fünfundfünfzig Seiten, erstaunlich gut lesbar. Ebenso lesenswert seine Masterarbeit von 1937 über Relaisschaltungen. Beide frei verfügbar.

[23] Sipser, M.: *Introduction to the Theory of Computation.* 3. Aufl., Cengage 2012.

Das Standardlehrbuch für Automaten, Berechenbarkeit und Komplexität. Ungewöhnlich klar geschrieben, mit sorgfältig ausgeführten Beweisen. Die Grundlage für Band 2 und 3 dieser Reihe.

[24] Tanenbaum, A. S.: *Structured Computer Organization.* 6. Aufl., Pearson 2012.

Entwickelt den Rechner konsequent als Schichtenmodell – von der Digitallogik über Mikroarchitektur und Befehlssatz bis zum Betriebssystem. Das Buch, das die Denkfigur aus Kapitel 12 geprägt hat.

[25] Turing, A. M.: *On Computable Numbers, with an Application to the Entscheidungsproblem.* Proc. London Math. Soc. 42, 1936.

Sechsunddreißig Seiten, die die Informatik begründeten: die Maschine, die Universalmaschine, das Halteproblem. Erstaunlich zugänglich; man sollte sie einmal im Original gelesen haben.

[26] von Neumann, J.: *First Draft of a Report on the EDVAC*. 1945.

Der Bericht, der die Architektur festlegte – und zugleich eines der umstrittensten Dokumente der Fachgeschichte, da Eckert und Mauchly wesentliche Ideen beisteuerten, aber nicht genannt wurden. Historisch wie technisch aufschlussreich.

[27] Wirth, N.: *Algorithmen und Datenstrukturen*. 5. Aufl., Teubner 1999.

Wirths berühmter Titel „Algorithmen + Datenstrukturen = Programme“ fasst das Programm des Buches zusammen. Knapp, streng und ohne jede Redundanz – der Stil des Pascal-Erfinders.

[28] Zuse, K.: *Der Computer – Mein Lebenswerk*. 5. Aufl., Springer 2010.

Zuses Autobiographie. Zeigt, wie jemand ohne Kenntnis von Turing, Shannon oder von Neumann im Berliner Wohnzimmer der Eltern auf dieselben Grundgedanken kam – Binärsystem, Gleitkomma, Programmsteuerung.

Wenn Sie nur drei Bücher kaufen

[21]	**Petzold**	zum Lesen – die ganze Kette vom Schalter zur Maschine, ohne Formeln
[18]	**Nisan, Schocken**	zum Tun – man baut alles selbst, vom NAND bis zum Compiler
[11]	**Hofstadter**	zum Nachdenken – der seltsame Kreis als Thema

Für die Fortsetzung: [2] Bryant/O’Hallaron *zur Verbindung von Hochsprache und Maschine*, **[23] Sipser** *für die Theorie*, **[20] Patterson/Hennessy** *für die Architektur*.

Und einmal im Leben sollte man ***[25] Turing 1936*** *im Original gelesen haben* – **es ist kürzer, als man denkt, und klarer, als man fürchtet.**